

Министерство науки и высшего образования Российской Федерации
КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

А.Н. ПОЛЕТАЙКИН, Н.Ю. ДОБРОВОЛЬСКАЯ

МЕТОДОЛОГИЯ И ТЕХНОЛОГИЯ
РАЗРАБОТКИ ПРОГРАММНЫХ СИСТЕМ:
МЕТОДЫ И МОДЕЛИ ПРОГРАММНОЙ
ИНЖЕНЕРИИ

Учебное пособие

Краснодар
2025

УДК 004.41(075.8)

ББК 32.973я73

П 497

Рецензенты:

Доктор физико-математических наук, профессор

А.В. Павлова

Кандидат технических наук, доцент

П.А. Приставка

Полетайкин, А.Н., Добровольская, Н.Ю.

П 497 Методология и технология разработки программных систем: методы и модели программной инженерии: учебное пособие / А.Н. Полетайкин, Н.Ю. Добровольская; Министерство науки и высшего образования Российской Федерации, Кубанский государственный университет. – Краснодар: Кубанский гос. ун-т, 2025. – 229 с. – 500 экз.
ISBN 978-5-8209-2573-3

Учебное пособие посвящено теории и инструментальной поддержке программной инженерии. Рассматриваются общий подход к программной инженерии, модели жизненного цикла и методологии создания программного обеспечения. Анализируются вопросы управления требованиями к программному обеспечению, конфигурационного управления и обеспечения качества программных продуктов.

Адресуется студентам высших учебных заведений направлений подготовки 01.03.02 «Прикладная математика и информатика», 02.03.02 «Фундаментальная информатика и информационные технологии», 02.03.03 «Математическое обеспечение и администрирование информационных систем».

УДК 004.41(075.8)

ББК 32.973я73

ISBN 978-5-8209-2573-3

© Кубанский государственный университет, 2025

© Полетайкин А.Н.,
Добровольская Н.Ю., 2025

ВВЕДЕНИЕ

Развитие информационных технологий определяет возрастание сложности и функциональных возможностей информационных систем, разрабатываемых в различных сферах деятельности. Современные крупные проекты создания программных продуктов предполагают наличие описания большого числа подпроцессов и взаимосвязей между ними, моделирование поведения системы, анализ данных, выявление взаимодействующих компонентов системы, имеющих локальные цели и задачи. К особенностям подобных проектов относятся: отсутствие готовых шаблонных решений, необходимость расширения спроектированного решения путем интеграции существующих и новых компонентов, функционирование на различных аппаратных платформах, неоднородность квалификации различных групп разработчиков, временные рамки проекта. Современное программирование представлено трансформацией от технологий индивидуального программирования отдельных небольших систем к коллективной разработке крупных программных комплексов, использующей инженерные методы проектирования и современные технологии разработки.

Технология традиционно определяется как комплекс организационных мер, операций и приемов, направленных на изготовление, обслуживание, ремонт и/или эксплуатацию изделия с номинальным качеством и оптимальными затратами и обусловленных текущим уровнем развития науки, техники и общества в целом. В контексте организации проектной деятельности технология объединяет определённым образом упорядоченную систему условий, критериев, форм, методов и средств решения поставленной задачи и достижения поставленной цели – разработку программного продукта высокого качества в рамках отведенного бюджета и в срок.

Для успешного решения проблемы создания крупного программного продукта, информационной системы требуется

тщательная подготовка проекта, определение функциональных и нефункциональных требований к будущему продукту, исследование структуры системы, определение логических взаимосвязей между компонентами. Другими словами, необходимо качественное проектирование программной системы, основанное на методологии и современных технологиях проектирования. Этот процесс предполагает использование методик и технологий управления процессами жизненного цикла программной системы: формулирование требований, проектирование, разработка, тестирование и развертывание системы.

Методология в самом общем смысле определяется как наука об организации деятельности.

Методология разработки программных продуктов ориентирует команду на технологические характеристики разрабатываемых систем, на применение следующих принципов: результаты программной деятельности должны соответствовать заявленным требованиям в условиях ограниченных ресурсов (эффективность), программный продукт должен быть рассчитан на конкретного пользователя (практичность), базироваться на знаниях фундаментальных наук (фундаментальность), запущенный в эксплуатацию программный продукт должен обслуживаться (сопровождаемость).

Основным предметом исследования программной инженерии является процесс создания программного обеспечения. Он охватывает большое разнообразие видов деятельности, методов, шагов, методик, используемых для разработки и улучшения программного обеспечения и связанных с ним продуктов.

Программное обеспечение (ПО) – неотъемлемая часть современной жизни. Оно используется в различных сферах деятельности, таких как образование, наука, бизнес, медицина, развлечения и т.д. Программное обеспечение представляет собой совокупность программ и данных, которые управляют работой компьютера или другого устройства. Качество, эффективность и

безопасность программного обеспечения зависят от того, как оно разрабатывается и сопровождается.

Программная инженерия – это наука и искусство создания качественного программного обеспечения с использованием систематических, дисциплинированных и измеримых подходов. Программная инженерия охватывает все стадии жизненного цикла программного обеспечения, такие как анализ требований, проектирование, реализация, тестирование, развертывание и сопровождение. Программная инженерия также изучает принципы, методы, технологии и инструменты для разработки программных продуктов.

Целью данного пособия является рассмотрение происхождения и развития программной инженерии как научной дисциплины и профессии.

Программная инженерия – учебная и научная дисциплина, которая изучает методологию и технологии разработки программного обеспечения.

Разработка программного обеспечения здесь понимается в широком смысле как весь жизненный цикл ПО от начала и до конца – от возникновения идеи создания программного продукта и до его снятия с эксплуатации и стирания.

Программная инженерия – область компьютерной науки и технологии, которая занимается построением программных систем, настолько больших и сложных, что для этого требуется участие слаженных команд разработчиков различных специальностей и квалификаций. Обычно такие системы применяются долгие годы, развиваясь от версии к версии, претерпевая на своем жизненном цикле множество изменений и обновлений – улучшение существующих функций, добавление новых или удаление устаревших возможностей, адаптация для работы в новой среде, устранение дефектов и ошибок, поддержка пользователей. Суть методологии программной инженерии

состоит в применении систематизированного, научного и предсказуемого процесса.

Все эти этапы (от анализа предметной области до внедрения и эксплуатации через постановку задачи и проектирование, кодирование, тестирование, обеспечение качества и внедрение) образуют программный процесс, или разработку в широком смысле. В программной инженерии разработке подлежат не только коды программ, но и требования, проектные решения, тесты, планы внедрения и документация. На всех этапах жизненного цикла осуществляются разные разработки, на поздних этапах – мелкие и средние доработки. В узком же смысле разработка ПО – это кодирование, рабочее проектирование ПО.

Предлагаемое учебное пособие ориентировано на студентов компьютерных направлений подготовки бакалавриата и магистратуры, изучающих программную инженерию и её разделы. В издании дается полное и практичное представление об основных и дополнительных процессах жизненного цикла ПО, инженерных работах, осуществляемых в этих процессах, и особенностях их реализации в настоящих программных проектах. В пособии представлено более 70 определений базовых понятий программной инженерии. Для удобства восприятия эти понятия выделены в тексте *разреженным курсивом*, а сами определения выделены отступами, как показано выше на примере определений основных понятий данного учебного пособия: «технология», «методология», «программная инженерия» и «разработка программного обеспечения».

Пособие рекомендуется студентам, изучающим учебные дисциплины «Методы программирования», «Программная инженерия», «Технология проектирование программных систем», «Тестирование ПО», «Коллективная разработка приложений» и другие дисциплины, непосредственно связанные с разработкой ПО на разных этапах его жизненного цикла.

1. ОСНОВНЫЕ ПОНЯТИЯ ПРОГРАММНОЙ ИНЖЕНЕРИИ

1.1. ПРОИСХОЖДЕНИЕ ПРОГРАММНОЙ ИНЖЕНЕРИИ

В конце 1960-х – начале 1970-х гг. произошел так называемый кризис программирования. Это был первый из нескольких кризисов, когда затраты на разработку и сопровождение программного обеспечения впервые опередили рост его качества и функциональности. Кризис был вызван рядом факторов, таких как увеличение сложности и размера компьютерных программ, несоответствие между ожиданиями заказчиков и результатами разработчиков, недостаток квалифицированных специалистов и эффективных методологий разработки.

На фоне этого кризиса стоимость программного обеспечения стала приближаться к стоимости аппаратуры (период времени появления операционных систем) и в настоящее время намного опережает стоимость аппаратуры (рис. 1.1). Тогда и возникла необходимость в разработке технологии программирования как в некоторой дисциплине, цель которой – сокращение стоимости программ. В дальнейшем эта дисциплина получила название программной инженерии.

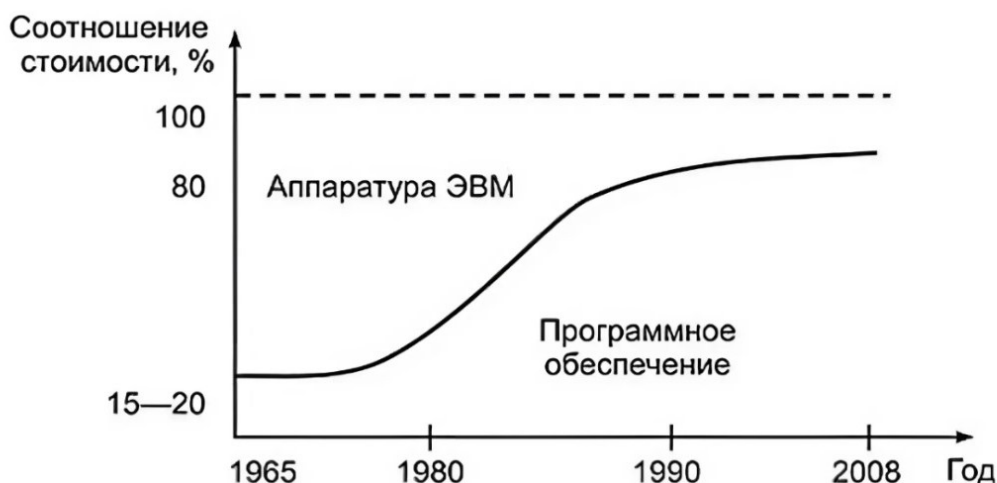


Рис. 1.1. Соотношение стоимости аппаратного и программного обеспечения ЭВМ

Каждый этап программной инженерии связан с появлением (или осознанием) очередной проблемы и нахождением путей и способов её решения. Рассмотрим ряд основных проблем разработки программ и найденных методов их решения.

Проблема 1 – повторное использование кода.

Решение – модульное программирование.

На первых этапах становления программной инженерии отмечалась связь высокой стоимости программ с разработкой подобных фрагментов кода в различных программах. Использование при создании новых программ ранее написанных фрагментов сулило существенное снижение сроков и стоимости разработки. Решения оформлялись в виде модулей, которые тиражировались для создания программных продуктов. Описательная часть (интерфейс) модуля определяла его назначение и взаимодействие с другими программами. Тем не менее повторное использование модулей со сложными интерфейсами является актуальной задачей и по сей день. Для ее решения разрабатываются специальные формы (структуры) представления модулей и организации их интерфейсов.

Проблема 2 – рост сложности программ.

Решение – структурное программирование.

Проблема возникла с появлением потребности в сложных программах (системы управления космическими объектами, управления оборонным комплексом, автоматизации крупных финансовых учреждений, производств и т.д.). Сложность программ оценивалась следующими показателями:

- большой объем кода (миллионы строк);
- большое количество связей между элементами кода;
- большое количество разработчиков (сотни человек);
- большое количество пользователей (сотни и тысячи);
- длительное время использования.

Все это весьма характерно для компьютерных программ, назначением которых является управление реальными объектами и процессами.

Управление – информационный процесс, связанный со сбором, передачей, накоплением информации о состоянии объекта управления, переработкой этой информации, выработкой управляющей информации и передачей ее на объект с целью принятия решений об изменении состояния объекта и приведения объекта управления в заданное (нормальное) состояние.

Оказалось, что основная часть таких сложных программ стоимости приходится не на создание программ, а на их внедрение и эксплуатацию. Возникло понятие о жизненном цикле программного продукта как о последовательности определенных этапов: проектирования, разработки, тестирования, внедрения и сопровождения.

Здесь заслуживает внимания этап сопровождения, который включал действия по исправлению ошибок в работе программы и внесению изменений в соответствии с изменившимися требованиями пользователей. Основная причина высокой стоимости (а порой и невозможности выполнения) этапа сопровождения состояла в том, что программы были плохо документированы (документация была не понятна и не соответствовала программному коду), а сам программный код был очень сложен и запутан. Это потребовало создания технологии корректного проектирования и рационального кодирования. Таким образом, возникла технология структурного программирования, основные принципы которой следующие:

- нисходящее функциональное проектирование, когда в системе выделяются основные функциональные подсистемы, которые потом разбиваются на функции (принцип декомпозиции);
- применение специальных языков проектирования и средств автоматизации использования этих языков;
- дисциплина проектирования и разработки: планирование и документирование проекта, поддержка соответствия кода проектной документации;

– структурное кодирование без использования оператора goto.

Проблема 3 – модификация программ.

Решение – объектно-ориентированное программирование (ООП).

Следующая проблема роста стоимости программ была вызвана тем, что изменение требований к программе стало возникать также на стадии проектирования. Это проблема заказчика, который не знает, чего он хочет. Создание программного продукта превратилось в его перманентное перепроектирование. Возник вопрос: как проектировать и писать программы, чтобы обеспечить возможность внесения изменений в программу, не меняя ранее написанного кода.

Решением этой проблемы стало использование подхода, который называется объектно-ориентированным проектированием и программированием. Суть подхода состоит в том, что вводится понятие класса как развитие понятия модуля с определенными свойствами и поведением, характеризующими обязанности класса. Каждый класс может порождать объекты – экземпляры данного класса. При этом работают основные принципы (парадигмы) ООП:

- инкапсуляция – объединение в классе данных (свойств) и методов (процедур обработки);
- наследование – возможность вывода нового класса из старого с частичным изменением свойств и методов;
- полиморфизм – определение свойств и методов объекта по контексту их использования.

Однако это были лишь локальные решения. На рубеже 1980–1990-х гг. отмечается начало информационно-технологической революции, вызванной взрывным ростом использования новейших информационных средств, таких как персональный компьютер, локальные и глобальные вычислительные сети, мобильная связь, электронная почта, Интернет и т.д. При этом кризис программирования принимает хронические формы:

- США тратит ежегодно более 200 млрд дол. на более чем 170 тыс. проектов в сфере IT (в основном на разработку ПО);
- 31,1 % из них закрываются, так и не завершившись; 52,7 % проектов завершаются с превышением первоначальных оценок бюджета / сроков и ограниченной функциональностью;
- потери от недополученного эффекта внедрения ПО измеряются триллионами.

Статистика по 30 тыс. проектам по разработке ПО в американских компаниях показывает следующее распределение (рис. 1.2) между:

- успешными – вовремя и в рамках бюджета выполнен весь намеченный объем работы;
- проблемными (спорными) – нарушение сроков, перерасход бюджета и / или сделано не все, что требовалось;
- проваленными – не были доведены до конца из-за перерасхода средств и / или бюджета, неприемлемого качества.

	2004	2008	2012	2016	2020	2023
УСПЕШНЫЕ	29%	35%	32%	37%	39%	36%
ПРОВАЛЬНЫЕ	18%	19%	24%	21%	18%	16%
СПОРНЫЕ	53%	46%	44%	42%	43%	48%

Рис. 1.2. Диаграмма распределения долей проваленных, проблемных (спорных) и успешных проектов

Процент успешных проектов по созданию ПО и сейчас чрезвычайно низок. Рассмотрим основные причины.

1. Разработка ПО по-прежнему остается непредсказуемой. Очень незначительное число проектов (порядка 20–25 %) по созданию ПО оказываются успешными и укладываются в первоначальные бюджетные и временные рамки.

2. Управление определяет успех или неудачу программных проектов в большей степени, чем технологические преимущества.

3. Количество переделанного или отбракованного ПО показывает незрелость процесса его разработки.

Практика разработки указанных 30 тыс. проектов дает следующую статистику (табл. 1.1).

Таблица 1.1

Соотношение между параметрами проекта и оценкой его успеха

Бюджет проекта, млн дол.	Число разработчиков	Срок реализации (месяцы)	Оценка успеха, %
Менее 0,75	6	6	55
От 0,75 до 1,5	12	9	33
От 1,5 до 3	25	12	25
От 3 до 6	40	18	15
От 6 до 10	+250	+24	8
Более 10	+500	+36	0

Соответственно, чем больше команда, тем ниже процент успешности. Если в команде более 500 человек, то эта успешность стремится к нулю. Даже если в команде всего 40 человек, то статистика дает только 15 % успешности проекта.

1.2. ПОНЯТИЕ ПРОГРАММНОЙ ИНЖЕНЕРИИ

На сегодняшний день нет единого определения понятия «программная инженерия». Сам термин – software engineering (программная инженерия) – впервые был озвучен в октябре 1968 г. на конференции подкомитета НАТО “NATO Software Engineering”, г. Гармиш (ФРГ). На конференции присутствовали 50 профессиональных разработчиков ПО из 11 стран. Рассматривались проблемы проектирования, разработки, распространения и поддержки программ. Впервые был определен термин «программная инженерия» как некоторая дисциплина, которую надо создавать и которой надо руководствоваться в решении перечисленных проблем.

Для того чтобы получить представление о программной инженерии, следует разобраться, что такое программное

обеспечение. Международный стандарт ISO/IEC 12207 дает следующее определение ПО.

Программное обеспечение – это набор компьютерных программ, процедур и связанной с ними документации и данных.

Как видно, ПО тесно связано с данными, которые оно перерабатывает.

Данные – отдельные сведения и факты, характеризующие объекты, процессы и явления предметной области, а также их свойства; структурированная информация о некоторой предметной области.

Предметная область – фрагмент реального мира, подлежащий изучению и рассматриваемый как источник сведений для анализа и информатизации бизнес-процессов. Представляется множеством фрагментов, каждый из которых характеризуется множеством бизнес-процессов, а также множеством пользователей, характеризуемых различными взглядами на предметную область.

Пользователь – специалист предметной области, для которого предназначено ПО.

Поскольку программы поставляются в совокупности с документацией и конфигурационными данными, ПО также называют программным продуктом.

Программный продукт – это хорошо документированное ПО.

В зависимости от того, для кого разрабатываются программные продукты (конкретного заказчика или рынка), программные продукты бывают двух типов:

- 1) коробочные продукты (generic products – общие продукты или shrink-wrapped software – упакованное ПО);
- 2) заказные продукты (bespoke – сделанный на заказ или customized products – настроенный продукт).

Разница между ними в том, кто ставит задачу (определяет или специфицирует требования). В первом случае это делают

сами разработчики на основе анализа рынка – и при этом рискуют сами. Во втором – заказчик, который рискует, что разработчик не сможет реально выполнить все требования в срок и при выделенном бюджете.

На основании указанных особенностей ПО можно предложить следующее определение программной инженерии.

Программная инженерия – это инженерная дисциплина, которая связана со всеми аспектами производства ПО от начальных стадий анализа предметной области и создания спецификации требований до поддержки ПО после сдачи в эксплуатацию.

В этом определении есть две ключевые фразы: 1) «инженерная дисциплина»; 2) «все аспекты производства ПО».

Инженеры – это технические специалисты, которые выполняют практическую работу и добиваются результатов за счет применения технологий. Для решения задачи инженеры используют теории, методы и средства, пригодные для решения данной задачи, но они применяют их выборочно и всегда пытаются найти решения даже в случаях отсутствия известных теорий или методов, соответствующих данной задаче. В этом случае инженер ищет способ решения задачи, применяет его и несет ответственность за результат. Набор таких инженерных методов или способов, теоретически не обоснованных, но получивших неоднократное подтверждение на практике, играет большую практическую роль. В программной инженерии они получили название *лучших практик* (best practices).

Инженеры работают в условиях ограниченных ресурсов: материальных, временных, финансовых и организационных. Хотя регламент разработки в первую очередь относится к созданию заказных продуктов (оговаривается в условиях контракта), при создании коробочных продуктов эти ограничения также актуальны, однако диктуются условиями рыночной конкуренции.

Программная инженерия занимается не только техническими вопросами производства ПО (специфицирование

требований, проектирование, кодирование и др.), но и управлением программными проектами, включая вопросы планирования, финансирования, управления коллективом и т.д. Кроме того, задача программной инженерии – разработка средств, методов и теорий для поддержки процесса производства ПО.

Программные инженеры применяют систематичные и организованные подходы к работе для достижения максимальной эффективности процесса разработки и качества ПО. Их задача состоит в адаптации существующих методов и подходов к решению своей конкретной проблемы.

Кроме того, в сферу программной инженерии попадают все вопросы и темы, связанные с организацией и улучшением процесса разработки ПО, в числе которого разработка и внедрение программных средств поддержки жизненного цикла разработки ПО.

Также программная инженерия использует в своей практике достижения информатики, тесно связана с системотехникой и предваряется бизнес-инжинирингом. Данные отношения показаны на рис. 1.3.

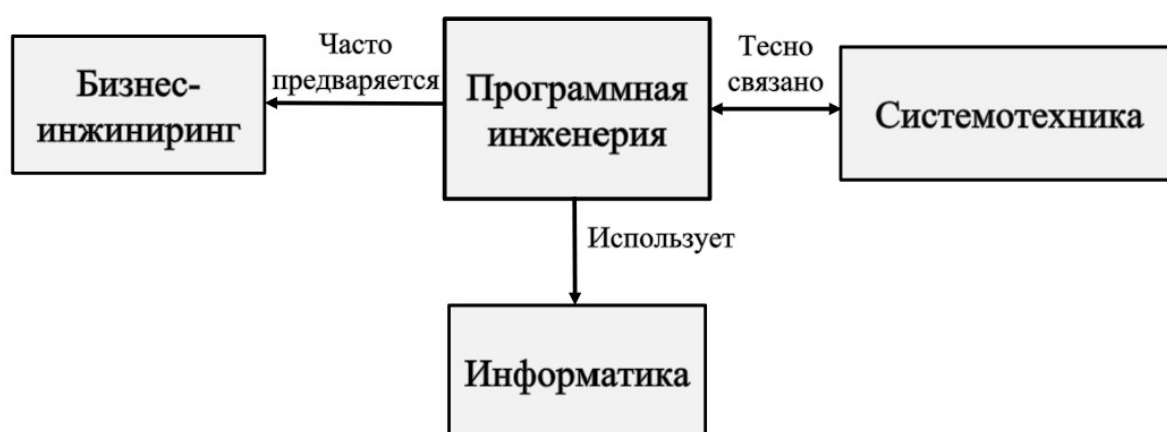


Рис. 1.3. Схема взаимодействия разных инженерий в программном процессе

Информатика (Information Science) занимается теорией и математическими методами вычислительных и программных

систем, а также их информационным обеспечением, в то время как программная инженерия является одной из компьютерных наук (Computer Science) и занимается практическими проблемами создания ПО. Трудно строго отделить программную инженерию от информатики, но в целом направленность этих дисциплин различна. Программная инженерия нацелена на решение проблем производства и обслуживания ПО, а информатика – на разработку формальных, математизированных подходов к программированию. В целом информатика составляет теоретические основы программной инженерии, и инженер по программному обеспечению должен знать информатику, так же как инженер по электронике – физику.

Системотехника (System Engineering) объединяет различные инженерные дисциплины по разработке всевозможных искусственных систем¹ – энергоустановок, ЭВМ, телекоммуникационных систем, компьютерных систем, встроенных систем реального времени и т.д. Достаточно заметить, что компьютерные программы создаются и работают на базе ЭВМ, что само по себе формирует тесную связь между системотехникой и программированием. К тому же часто ПО оказывается частью указанных систем, выполняя задачу управления соответствующим оборудованием. Такие системы называются *программно-аппаратными*, и, участвуя в их создании, программисты вынуждены глубоко разбираться в особенностях соответствующей аппаратуры. В этом аспекте программная инженерия тесно связана с системотехникой в контексте архитектуры проектируемых программно-аппаратных систем.

Бизнес-инжиниринг (Business Engineering) в широком смысле обозначает модернизацию бизнеса и внедрение новых практик, поддерживаемых информационными системами. При этом акцент может быть как на внутреннем переустройстве

¹ Широкоупотребительное понятие системы как совокупности взаимодействующих разнородных компонентов, объединенных для достижения общей цели, дано в параграфе 2.1.

компании, так и на разработке нового клиентского сервиса (как правило, эти вопросы взаимосвязаны). Бизнес-инжиниринг часто предваряет разработку и внедрение информационных систем на предприятии, так как требуется сначала навести определенный порядок в делопроизводстве, а лишь потом закрепить его информатизацией.

Программная инженерия принципиально отличается от других инженерий прежде всего нематериальностью своей продукции. В отличие от объектов других инженерий, компьютерная программа – это не материальный объект (не путать с материальным носителем программы). Фаза производства ПО состоит в копировании образца на другие носители, стоимость которой мала. Если кодирование считать элементом проектирования (в широком смысле), то отсутствует и фаза создания образца (строится компилятором и линковщиком) за считанные секунды. Отсюда следующие выводы.

1. Стоимость программы – это стоимость только ее создания. Типовое распределение стоимости между основными этапами (без сопровождения) представлено на рис. 1.4.

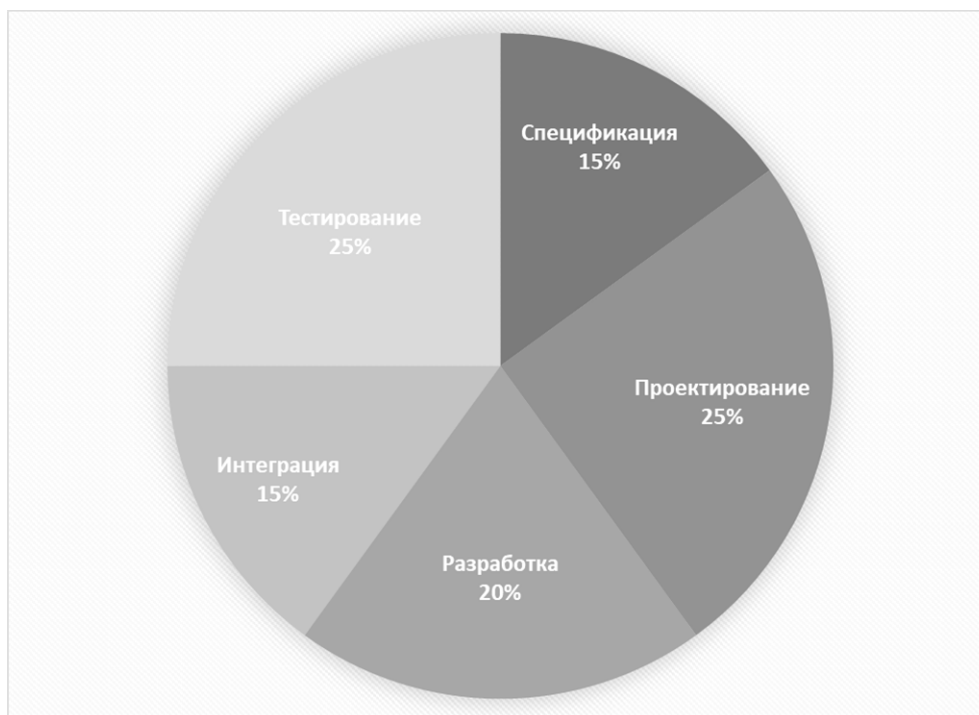


Рис. 1.4. Типовое распределение стоимости между основными этапами (без сопровождения)

2. Стоимость проектирования коробочных продуктов распределяется по копиям. Кроме того, для коробочного ПО характерна более высокая доля тестирования за счет сокращения прежде всего доли спецификации (до 5 %). Применение опробованных решений и готовых компонентов позволяет повысить качество и сократить сроки разработки.

3. Стоимость заказных продуктов (массово не копируемых) остается высокой. Распределение стоимости зависит от сложности ПО. При сложном ПО возрастает доля интеграции и тестирования, но за счет сокращения доли проектирования и разработки, а доля спецификаций может возрасти. Сокращение доли проектирования и разработки достигается за счет применения опробованных проектных решений и повторного использования готовых компонентов.

Также важно отметить, что *программа – искусственный объект*, и для программы нет объективных законов, которым бы подчинялось ее поведение. Например, у инженера-строителя есть объективные законы строительной механики: равновесия моментов и сил, устойчивости механических систем и т.д. У инженера-электронщика – свои базовые законы: Ома, Джоуля-Ленца, Кирхгофа, Фарадея и др. Эти инженеры-«материалисты» могут проверить свои решения на соответствие этим законам и тем самым обеспечить удачу проекта. Это законы природы, они объективны, они будут действовать всегда.

У программного инженера на первый взгляд также есть типовые, проверенные временем решения (например, спиральная модель Боэма, клиент-серверная архитектура и принципы кибернетики и парадигмы ООП). Но эти решения определяются уровнем развития программной и системной инженерий (и адекватным им уровнем требований). С постановкой более масштабных и комплексных задач либо появлением техники с принципиально новыми возможностями программному инженеру придется искать новые решения.

Прямым следствием этого недостатка является то, что тестирование продукта – это единственный способ убедиться в его качестве. Именно поэтому стоимость тестирования

составляет существенную стоимость ПО (рис. 1.4). Тогда как строительный инженер, как правило, лишен возможности настолько масштабного тестирования своего продукта перед сдачей его в эксплуатацию.

Программная инженерия – сравнительно молодая дисциплина, опыт которой насчитывает всего несколько десятков лет. По сравнению с опытом строительной инженерии (тысячелетия) это очень мало. Программную инженерию иногда сравнивают с ранней строительной, когда законы строительной механики еще не были известны и строительные инженеры действовали методом проб и ошибок, накапливая бесценный опыт. Программная инженерия также накопила определенный опыт, который позволяет (при разумном его применении) делать удачные проекты. Этот опыт выражен в основных принципах программной инженерии.

1.3. ПРОГРАММНЫЙ ПРОЦЕСС

Одним из основных понятий программной инженерии является понятие жизненного цикла программного продукта и программного процесса.

Процесс – совокупность действий и задач, упорядоченных во времени и имеющих целью достижение системно значимого результата.

Жизненный цикл ПО – непрерывный процесс, начинающийся с момента принятия решения о создании ПО и заканчивающийся снятием его с эксплуатации.

Фактически программный процесс представляет собой жизненный цикл (ЖЦ) ПО, который также разбивается на отдельные процессы. Основные процессы (иногда называют этапами или фазами) ЖЦ ПО, индифферентно к его назначению и другим особенностям, показаны в табл. 1.2.

Кроме основных, существует много организационных и вспомогательных процессов, связанных как с организацией работ (нефункциональные процессы): создание инфраструктуры,

управление конфигурацией, управление качеством, обучение, разрешение противоречий и др., так и с реализацией вспомогательных процессов, к которым относится, например, документирование программы. Прежде чем перейти к их изучению, рассмотрим некоторые особенности ПО с точки зрения применения программной инженерии.

Таблица 1.2

Соотношение между параметрами проекта и оценкой его успеха

Процесс	Результат
Разработка спецификации требований (Requirements Specification)	Описания требований к программе, которые обязательны для выполнения – описание того, что программа должна делать
Разработка проекта программы	Описание того, как программа будет работать (проект)
Кодирование	Исходный код и файлы конфигурации
Тестирование программы	Контроль соответствия программы требованиям
Внедрение	Действующая программа

Прежде всего, программный процесс должен быть установлен. Полное установление процесса предполагает:

- описание процесса – детальное описание действий и операций процесса;
- обучение процессу – проведение занятий с персоналом по освоению процесса;
- введение метрик – установление количественных показателей хода выполнения;
- контроль выполнения – способы измерения метрических показателей и оценка хода выполнения;
- усовершенствование – изменение процесса при меняющихся условиях применения.

Применение полных (тяжелых) процессов требует дополнительных ресурсов (зачастую существенных) и далеко не всегда окупается полученным результатом. Поэтому выбор состава процессов, степени (полноты) их установленности в

каждом конкретном случае может быть сделан по-разному в соответствии с выбранной моделью процесса. Более подробно понятие программного процесса будет рассмотрено во второй главе.

1.4. ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

Под программным обеспечением будем понимать множество развивающихся во времени логических предписаний, с помощью которых люди управляют объектами и процессами реального мира с использованием современных вычислительных устройств и систем.

Проясним некоторые составляющие этого определения.

1. Логические предписания – это не только сами программы, но и различная документация (например, по эксплуатации программ), а также инструкции пользователям программ в рамках определенного процесса деятельности.

2. Современное ПО предназначено, как правило, для одновременной работы со многими пользователями, которые могут быть значительно удалены друг от друга в физическом пространстве. Таким образом, вычислительная среда (персональные компьютеры, серверы и т.д.), в которой функционирует ПО, оказывается распределенной.

3. Задачи, решаемые современным ПО, часто требуют различных вычислительных ресурсов в силу различной специализации этих задач, из-за большого объема выполняемой работы, а также из соображений безопасности. Например, появляется сервер базы данных, сервер приложений, интернет-сервер и пр. Таким образом, вычислительная среда, в которой ПО функционирует, оказывается многопроцессорной.

4. ПО развивается во времени – исправляются ошибки, добавляются новые функции, выпускаются новые версии, меняется его аппаратная база.

Таким образом, ПО является сложной динамической системой, включающей технические, психологические и социальные аспекты. ПО заметно отличается от других видов

систем, создаваемых человеком (механических, социальных, научных и пр.), и, кроме нематериальности, имеет следующие особенности:

1) сложность программных объектов, которая существенно зависит от их размеров. Как правило, большее ПО (большее количество пользователей, большой объем обрабатываемых данных, более жесткие требования по быстродействию и пр.) с аналогичной функциональностью – это другое ПО;

2) согласованность – ПО основывается не на объективных посылах (подобно тому, как различные системы в классической науке основываются на постулатах и аксиомах), а должно быть согласовано с большим количеством интерфейсов, с которыми впоследствии оно должно взаимодействовать. Программные интерфейсы плохо поддаются стандартизации, поскольку основываются на многочисленных и плохо формализуемых человеческих соглашениях;

3) модифицируемость – ПО, вследствие его нематериальности, легко модифицировать и поэтому требования к нему меняются в процессе разработки. Это создает много дополнительных трудностей при его разработке и эволюции.

1.5. МЕТОДЫ ПРОГРАММНОЙ ИНЖЕНЕРИИ

Технология разработки ПО, будучи по своей природе комплексным организационно-техническим образованием, включает большое количество методов и построенных на их основе либо с их применением моделей.

Методы как таковые есть реализации способов теоретического исследования или практического осуществления чего-либо (от греч. μέθοδος – путь исследования, познания, теория, учение). В практической деятельности методы помогают организовать работу, повысить эффективность и достичь поставленных целей. Соответственно, методы программной инженерии есть не что иное, как формализованные способы изготовления ПО.

Метод программной инженерии – это структурный подход к созданию ПО, который способствует производству высококачественного продукта эффективным в экономическом аспекте способом.

В этом определении есть две основные составляющие:

- 1) создание высококачественного продукта;
- 2) экономически эффективный способ.

Иными словами, метод программной инженерии – это то, что обеспечивает решение ее основной задачи: создание качественного продукта при заданных ресурсах времени, бюджета, оборудования, людей и др.

Начиная с 1970-х гг. создано очень много методов разработки ПО. Наиболее известные из них:

- метод структурного анализа и проектирования (Том ДеМарко, инженер-программист, США, 1978);
- метод «сущность-связь» для проектирования ИС (Питер Чен, ученый информатик, США, 1976);
- метод объектно-ориентированного анализа (Гради Буч, инженер-программист, США, 1994; Рамбо, 1991) и др.

Методика (методический аппарат) программной инженерии основана на идее создания *моделей* ПО с поэтапным преобразованием этих моделей в программный код и далее – в программу – окончательную модель решаемой задачи в виде машинного кода, который, собственно, и исполняется центральным процессором ЭВМ. Так, на этапе спецификаций создается модель в форме описания требований, которая далее преобразуется в модель проекта ПО, проект – в структурированный программный код, который в свою очередь – в машинный код, исполняемый ЭВМ. При этом важно, чтобы модели представлялись графически с помощью некоторого языка представления моделей. Это делает их наглядными и удобными в построении и модификации.

Методы включают следующие компоненты:

- руководство по применению метода – описание последовательности работ (действий), которые надо выполнить

для построения моделей (например, все атрибуты должны быть задокументированы до определения операций, связанных с этим объектом);

- описание моделей системы и нотация, используемая для описания этих моделей (например, объектные модели, конечно-автоматные модели и т.д.);

- правила и ограничения, которые следует выполнять при разработке моделей (например, каждый объект должен иметь одинаковое имя);

- рекомендации – эвристики, характеризующие хорошие приемы проектирования в данном методе (например, рекомендация о том, что ни у одного объекта не должно быть больше семи подобъектов).

Не существует универсальных методов, все они применимы только для тех или иных случаев. Применяемые на практике методы могут включать элементы различных подходов и отражать один из множества вариантов реализации того или иного способа. Выбор метода составляет задачу специалиста по программной инженерии.

Однако метод сам по себе бесполезен. Реальную пользу в виде реализации способа он несёт только в процессе его применения. При этом ручной способ применения методов зачастую является весьма ресурсозатратным. Для оптимизации применения методов по этому критерию создаются соответствующие *инструменты*.

1.6. CASE-СРЕДСТВА

Инструмент (лат. *instrūmentum* – орудие производства) – технологическая оснастка, которая воздействует на предметы труда и изменяет их, предмет, орудие для производства каких-нибудь работ. Данное базовое определение однозначно относит инструменты к компонентам технологии.

Инструмент (в широком смысле) – целевое средство воздействия на объект, преобразования и создания объекта.

В программной инженерии инструменты представляют собой прикладные программы, имеющие целью реализацию какой-либо задачи программного процесса. Такие инструментальные программы называются CASE-средствами.

CASE – Computer Aided System Engineering – средства автоматизированного программирования, различного рода инструментальные программы, используемые для автоматизированной поддержки процесса создания компьютерных программ.

К первым CASE-средствам отнесли с определенной настороженностью. Со временем появились как восторженные отзывы об их применении, так и критические оценки их возможностей. Во-первых, ранние CASE-средства были простой надстройкой над СУБД. Визуализация процесса разработки концептуальной схемы БД имеет немаловажное значение. Однако она не решает проблем создания приложений других типов. Во-вторых, графическая нотация, реализованная в CASE-средстве, в отличие от строгого синтаксиса языков программирования, воспринималась далеко не однозначно.

CASE-средства могут быть классифицированы по нескольким признакам.

1. По уровню применения:
 - Upper CASE – средства анализа требований;
 - Middle CASE – средства проектирования;
 - Low CASE – средства разработки приложений.
2. По специализации:
 - средства проектирования баз данных;
 - средства процессного моделирования;
 - средства реинжиниринга (восстановления) модели (формирование ERD на основе анализа схем БД или формирования диаграмм на основе анализа текстов программ).
3. По видам деятельности:
 - средства управления требованиями;
 - средства планирования и управления проектом;
 - средства конфигурационного управления;

- средства управления качеством;
- средства управления выпуском и др.

4. По структурной организации: простые и интегрированные CASE – охватывают все этапы и процессы создания ПО от анализа требований до тестирования и выпуска документации. Интегрированные CASE выступают, как правило, в виде набора согласованных по интерфейсу средств, предназначенных для поддержки отдельных этапов процесса.

В настоящее время существует очень много CASE-средств и фирм, специализирующихся на их разработке. Это широкий спектр программ – инструментальных средств, применяемых на разных этапах разработки ПО: спецификации требований, проектирования, кодирования, тестирования, документирования и т.д.

2. СТАНДАРТНЫЙ ПРОЦЕСС РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1. ПОНЯТИЕ ПРОЦЕССА РАЗРАБОТКИ ПО

Центральным объектом изучения программной инженерии является процесс создания ПО – программный процесс.

Программный процесс – множество различных видов деятельности, методов, методик и шагов, используемых для разработки и эволюции ПО, а также связанных с ним продуктов (проектных планов, документации, программного кода, тестов, пользовательской документации и пр.).

На сегодняшний день не существует универсального процесса разработки ПО для любых компаний, для команд любой национальности. Каждый текущий процесс разработки, осуществляемый некоторой командой в рамках определенного проекта, имеет большое количество особенностей и индивидуальностей. Однако есть и общие моменты. Так, перед началом проекта целесообразно спланировать программный процесс, определив роли и обязанности в команде, рабочие продукты (промежуточные и финальные), структуру проекта, порядок участия в их разработке членов команды и т.д. Будем называть это предварительное описание *конкретным процессом*, отличая его от плана работ, проектных спецификаций и пр. Например, в системе коллективной разработки ПО Visual Studio Azure DevOps Server (СУ ЖЦ) используется шаблон процесса, создаваемый или адаптируемый (в случае использования стандартного) перед началом разработки. В СУ ЖЦ существуют заготовки для конкретных процессов на базе различных технологических подходов (CMMI, Scrum и др.).

Само понятие стандартного процесса недвусмысленно намекает на то, что процесс некоторым образом унифицирован и стандартизирован, как готовая постоянная структура. Как правило, это жизненный цикл, прежде всего этапы жизненного цикла, и пр. Помимо основных этапов, основных процессов,

олицетворяющих процесс создания ПО на всех этапах жизненного цикла, есть еще и вспомогательные процессы, управленческие процессы, организационные процессы, распределение ресурсов для реализации работ по проекту. Все эти 4 класса процессов должны быть определены для того, чтобы не только какая-то конкретная компания или страна могла эффективно осуществлять разработку ПО, но и другие компании / страны. Стандартный процесс делает возможной эффективную международную коллаборацию. А так как международные коллаборации довольно часто встречаются на мировом рынке ПО, актуальной задачей является стандартизация процесса разработки ПО. Поэтому во многом программный процесс стандартизирован и уже давно. Существует множество методик и стандартов, которые определяют структуру программного процесса и обуславливают его во времени и пространстве. Один из них – это унифицированный процесс разработки ПО, или Rational Unified Process (RUP), будет рассмотрен далее в параграфе 2.2.

В рамках компании-разработчика ПО возможна и полезна стандартизация всех текущих процессов, которые отражаются в некоторой базе данных, содержащей:

- информацию о средствах разработки (систем версионного контроля, средств контроля ошибок, средств программирования – различных IDE, СУБД и т.д.), в том числе правила их использования, документацию и инсталляционные пакеты;
- описание практик разработки – проектного менеджмента, правил работы с заказчиком и т.д.;
- шаблоны проектных документов – технических заданий, проектных и функциональных спецификаций, тестовых планов и пр.

Также возможна стандартизация процедуры разработки конкретного процесса как «вырезки» из стандартного. Основная идея стандартного процесса – циркуляция внутри компании передового опыта, а также унификация средств разработки. Множественность этих средств существенно затрудняет миграцию специалистов из проекта в проект, использование

результатов одного проекта в другом и т.д. Однако при организации стандартного процесса необходимо следить, чтобы он не оказался всего лишь формальным, бюрократическим аппаратом. Понятие стандартного процесса введено и подробно описано в подходе СММІ.

2.2. ЗРЕЛОСТЬ ПРОЦЕССА РАЗРАБОТКИ ПО

Для решения задачи стандартизации программного процесса американским университетом Карнеги-Меллон (Software Engineering Institute, SEI) по заказу Министерства обороны США разработана модель СММІ (Capability Maturity Model Integration), характеризующая уровни зрелости процесса разработки ПО. Конец 1980-х – Capability Maturity Model – модель зрелости возможностей создания ПО: эволюционная модель развития способности компании разрабатывать программное обеспечение, СММІ (Integration – обобщающий стандарт).

СММІ – модель зрелости возможностей (СММ) создания ПО: эволюционная модель развития способности компании разрабатывать программное обеспечение. Integration – обобщающий стандарт.

Модель СММІ представляет описание идеального программного процесса. Базовым понятием модели СММІ считается зрелость компании.

Незрелой называют компанию, где процесс создания ПО и принимаемые решения зависят только от таланта конкретных разработчиков. Результатом служит высокий риск превышения бюджета или срыва сроков окончания проекта.

В *зрелой* компании работают процедуры управления проектами и построения программных продуктов. По мере необходимости эти процедуры уточняются и развиваются. Оценки длительности и затрат разработки точны и основываются на накопленном опыте. Кроме того, в компании имеются и действуют корпоративные стандарты на такие процессы ЖЦ ПО:

- взаимодействие с заказчиком;

- анализ;
- проектирование;
- программирование (кодирование);
- тестирование;
- внедрение.

Все это создает среду, обеспечивающую качественную разработку ПО.

Данные значимые части процесса в СММІ называются *областями усовершенствования*. Различаются следующие их группы:

- управление процессами;
- управление проектами;
- инженерные области;
- служебные области.

При этом все области задаются в виде требований к процессам ЖЦ ПО, из чего имеется два следствия.

Следствие 1. СММІ допускает различные реализации и не является методологией разработки ПО, подобно MSF, Scrum, RUP и пр. Последние могут использоваться в его реализации. Так, существует, например, специальный шаблон процесса в СУ ЖЦ для СММІ под названием MSF for СММІ.

Следствие 2. СММІ используется для сертификации компаний на зрелость их процессов. Изначально на рубеже 1990-х гг. СММ (тогда еще не СММІ) создавался именно как средство сертификации федеральных субподрядчиков разработки ПО в США. И только позднее, получив широкое распространение в мире, он начал использоваться, а после и ориентироваться на совершенствование процессов.

СММІ имеет целью улучшение качества создаваемых продуктов и / или снижения стоимости и времени их разработки. Производителей ПО к этому побудили следующие обстоятельства (помимо удручающей статистики успешности программных проектов, рис. 1.2):

- происходит быстрая смена технологий разработки ПО, требуются изучение и внедрение новых средств разработки;

- наблюдается быстрый рост компаний и их выход на новые рынки, что требует новой организации работ;
- имеет место высокая конкуренция, которая требует поиска более эффективных, более экономичных способов разработки.

При этом возможны следующие практические решения, улучшающие производственный процесс компании-производителя ПО:

- 1) переход на новые средства разработки, языки программирования и т.д.;
- 2) улучшение отдельных управленческих и инженерных практик – тестирования, управления требованиями и пр.;
- 3) полная, комплексная перестройка всех процессов в проекте, департаменте, компании (в соответствии, например, с СММІ);
- 4) сертификация компании (СММ/СММІ, ISO 9000 и пр.).

Здесь п. 3 от п. 4 отделен потому, что на практике сертификация компании далеко не всегда означает действительную созидательную работу по улучшению процессов разработки ПО, а часто сводится к поддержанию соответствующего документооборота, необходимого для получения сертификации. Сертификат потом используется как средство конкурентной борьбы за заказы.

СММІ предназначен не только для разработки программных систем. Многие крупные компании выпускают не ПО, а целевые изделия, куда ПО входит как составная часть. Разработка ПО происходит вместе с инженерными работами иных видов. И часто бывает, что в одном проекте участвует более двух различных видов инженерии. Задача СММІ – предоставить таким проектам и компаниям единую платформу организации процесса разработки.

Главная трудность реального совершенствования процессов в компании заключается в том, что она при этом должна работать и создавать ПО. Отсюда вытекает идея непрерывного улучшения процесса малыми этапами. В числе прочего это обусловлено постоянным появлением новых и перманентным развитием

существующих технологий разработки, что нужно постоянно отслеживать. Эта стратегия, в частности, отражена в стандарте совершенствования процессов разработки СММІ.

Уровни зрелости процессов по СММІ. В отличие от классической модели СММ, которая была жестко иерархической и допускала только последовательное улучшение процессов по уровням, модель СММІ имеет два измерения – последовательное, такое же, как и в СММ, и непрерывное, допускающее совершенствование процессов в организации до некоторой степени в произвольном порядке. Здесь рассмотрим последовательную модель. Она имеет 5 уровней зрелости процессов, как показано в табл. 2.1.

Начальный уровень (уровень зрелости 1) – это уровень, на котором изначально находится любая компания. На этом уровне разработка ПО ведется более-менее хаотично.

Повторяемый уровень (уровень зрелости 2) – здесь уже появляются политики и процедуры организации процессов, утвержденные на уровне компании. Но в полной мере процессы существуют лишь в рамках отдельных проектов и многократно воспроизводятся откуда и название уровня – повторяемый.

Определенный уровень (уровень зрелости 3) – здесь появляется стандартный процесс на уровне всей компании в целом. Это большой и постоянно пополняющийся набор активов процесса – шаблонов документов, моделей жизненного цикла, программных средств, практик и пр.

Управляемый уровень (уровень зрелости 4) подразумевает появление системы измерений в компании, которые происходят на базе стандартного процесса и позволяют количественно управлять разработкой.

Оптимизирующийся уровень (уровень зрелости 5) подразумевает постоянное улучшение процессов разработки, как постепенных, пошаговых, так и революционных. При этом данные изменения оказываются не вынужденными, а упреждающими проблемы и трудности. Процесс совершенствуется сам и постоянно, для чего реализованы соответствующие механизмы.

Таблица 2.1

Области усовершенствования по СММІ

Уровень зрелости / фокус	Области усовершенствования
1. Начальный	–
2. Повторяемый. Управление проектами	Управление требованиями
	Планирование проекта
	Наблюдение за проектом и контроль
	Управление договоренностями с заказчиками
	Измерения и анализ
	Проверка процессов и продуктов на соответствие стандартам
	Конфигурационное управление
3. Определённый. Целостность процесса	Разработка требований
	Техническое решение
	Сборка и поставка продукта
	Проверка продукта на соответствие требованиям (верификация)
	Проверка продукта на соответствие предназначению (валидация)
	Фокусирование на процессах организации
	Организация обучения
	Комплексное управление проектом
	Управление рисками
	Управление командой
	Принятие решений: оценка альтернатив
	Создание в организации условий для совместной работы
4. Управляемый. Качество продукта и процесса	Установление показателей выполнения процессов организации
	Управление проектами на основе количественных показателей
5. Оптимизирующий. Постоянное улучшение процесса	Отбор и внедрение улучшений в организацию
	Анализ причин возникновения проблем и предотвращение их появления в будущем

2.3. Жизненный цикл ПО

Одним из базовых понятий методологии проектирования ПО является понятие ее жизненного цикла (ЖЦ ПО).

Жизненный цикл ПО – непрерывный процесс, начинающийся с момента принятия решения о создании ПО и заканчивающийся снятием его с эксплуатации.

Основным нормативным документом, регламентирующим ЖЦ ПО, является международный стандарт ISO/IEC 12207 (ISO – International Organization for Standardization, Международная организация по стандартизации, IEC – International Electrotechnical Commission, Международная комиссия по электротехнике). Он определяет структуру ЖЦ, содержащую процессы, действия и задачи, которые должны быть выполнены в период существования ПО.

Структура ЖЦ ПО по стандарту ISO/IEC 12207 базируется на четырех группах процессов (рис. 2.1):

- основные;
- вспомогательные, обеспечивающие выполнение основных процессов;
- процессы управления;
- организационные процессы.

Разработка включает все работы по созданию ПО и его компонентов в соответствии с заданными требованиями:

- анализ предметной области (как правило);
- создание компонентов ПО: функциональных и обеспечивающих (проектирование и реализацию);
- тестирование компонентов ПО разными видами тестов (подробнее см. главу 3);
- эскизное оформление проектной и эксплуатационной документации (основная работа по документированию ПО выполняется в рамках соответствующего вспомогательного процесса ЖЦ);
- подготовку материалов, необходимых для проверки работоспособности и качества компонентов ПО;



Рис. 2.1. Жизненный цикл ПО согласно стандарту ISO/IEC 12207

- подготовку обучающих материалов для персонала и т.д.
- Эксплуатация* включает работы по внедрению компонентов ПО в эксплуатацию. При этом предполагается:
- конфигурирование базы данных и рабочих мест пользователей;
 - обеспечение эксплуатационной документацией и ее актуализация;
 - обеспечение обучения персонала;

- локализация проблем и устранение причин их возникновения;
- подготовка предложений по совершенствованию, развитию и модернизации системы;
- модификация ПО в рамках установленного регламента и т.д.

Управление проектом связано с вопросами планирования и организации работ, создания коллективов разработчиков и контроля за сроками и качеством выполняемых работ.

Управление конфигурацией является одним из вспомогательных процессов, поддерживающим прежде всего такие основные процессы ЖЦ ПО, как разработка и сопровождение ПО. Позволяет организовать, систематически учитывать и контролировать внесение изменений в ПО на всех стадиях ЖЦ.

Обеспечение качества проекта связано с проблемами верификации, проверки и тестирования ПО.

Верификация – это процесс определения того, отвечает ли текущее состояние разработки, достигнутое на данном этапе, требованиям этого этапа.

Проверка позволяет оценить соответствие параметров разработки исходным требованиям к проекту. Проверка частично совпадает с тестированием, которое связано с идентификацией различий между действительными и ожидаемыми результатами.

В процессе реализации проекта важное место занимают вопросы идентификации, описания и контроля конфигурации отдельных компонентов и всей системы в целом. Каждый процесс характеризуется определенными задачами и методами их решения, исходными данными, полученными на предыдущем этапе, и результатами. Например, результатами анализа являются функциональные модели, информационные модели и соответствующие им диаграммы.

ЖЦ ПО носит итерационный характер: результаты очередного этапа часто вызывают изменения в проектных решениях, выработанных на более ранних этапах.

2.4. МОДЕЛЬ ПРОЦЕССА РАЗРАБОТКИ ПО

Модель процесса разработки ПО – упрощенное описание программного процесса, представленное с некоторой точки зрения, задается в виде практических этапов, необходимых для создания ПО; структура, содержащая процессы, действия и задачи, которые осуществляются в ходе создания программного продукта.

Выбор модели процесса является первым шагом в создании ПО. Правильный выбор модели очень важен, так как во многом определяет успех проекта. Выбор «тяжелых» процессов может утопить проект, а слишком легкое отношение к процессам – к потере контроля над ходом выполнения.

В соответствии с двумя типами процессов – основных и дополнительных – можно говорить о двух типах моделей процесса: модели процесса разработки (модели жизненного цикла) и модели организации работ по выполнению разработки.

К моделям I типа (модели жизненного цикла) относятся такие, в которых описывается порядок выполнения действий по созданию продукта. Наиболее известны следующие модели:

1. Каскадная (водопадная) модель – процесс разбивается на последовательность выполнения стадий. Каждая стадия начинается после полного завершения предыдущей, продукт создается завершением последней стадии и должен полностью соответствовать изначально установленным требованиям.

2. Спиральная (циклическая) модель – процесс также разбивается на стадии, но стадии выполняются циклическим повторением. На первом цикле создается прототип продукта, выполняющий часть требований. Дальнейшие циклы связаны с наращиванием прототипа до полного удовлетворения требований.

3. Компонентная модель предполагает сборку продукта из заранее написанных частей – компонентов. Основной упор делается на интеграцию и совместное тестирование компонентов.

4. Формальная модель основана на формальном описании требований с последующим их преобразованием (трансляцией) в исходный код. Применение формальной модели гарантирует соответствие кода описанным требованиям.

Следует отметить, что различия между этими моделями существуют только в теории. На практике спиральная модель может быть дополнена элементами каскадной и компонентной. Задача программного инженера – подобрать правильную их комбинацию, ориентируясь только на конечный результат.

К моделям II типа (моделям организации работ) относятся:

1. Модель потока работ (Workflow Model) – регламентирует последовательность действий, выполняемых людьми на различных этапах создания ПО. Для каждого действия указываются входы (исходные данные), выходы (результаты) и связи по входам и выходам.

2. Модель потоков данных (Data Flow Model) – представляет процесс в виде последовательного преобразования данных. Каждое преобразование может выполняться в одно или несколько действий.

3. Сетевая модель – представляет собой сетевой график, определяющий структуру плана работ по проекту. Дает возможность определить задачи, лежащие на критическом пути, позволяет оптимизировать структуру проекта.

4. Ролевая модель – показывает роли людей, участвующих в программном процессе, а также действия и результаты, за которые они отвечают.

2.5. ФАЗЫ И ВИДЫ ДЕЯТЕЛЬНОСТИ

Говоря о моделях процесса разработки ПО, необходимо различать фазы и виды деятельности.

Фаза (phase) программного процесса – это определенный этап программного процесса, имеющий начало, конец и выходной результат.

Фазы следуют друг за другом в линейном порядке, характеризуются предоставлением отчетности заказчику и выплатой денег за выполненную часть работы.

Редко какой заказчик согласится первый раз увидеть результаты только после завершения проекта. В то же время подрядчики предпочитают получать деньги постепенно по мере того, как выполняются отдельные части работы. Таким образом, появляются фазы, позволяющие создавать и предъявлять промежуточные результаты проекта.

Фазы полезны также безотносительно взаимодействия с заказчиком – с их помощью можно синхронизировать деятельность разных рабочих групп, а также отслеживать продвижение проекта.

Примерами фаз может служить согласование с заказчиком технического задания, реализация определенной функциональности ПО, этап разработки, оканчивающийся сдачей системы на тестирование или выпуском альфа-версии.

Вид деятельности (activity) программного процесса – это определенный тип работы, выполняемый в процессе разработки ПО.

Разные виды деятельности часто требуют разных профессиональных навыков и выполняются разными специалистами. Например, управление проектом выполняется менеджером проекта, кодирование – программистом, тестирование – тестировщиком. Есть виды деятельности, которые могут выполняться одними и теми же специалистами – например, кодирование и проектирование (особенно в небольшом проекте) часто выполняют одни и те же люди.

В рамках одной фазы может выполняться несколько различных видов деятельности. Кроме того, один вид деятельности может выполняться на разных фазах. Например, тестирование: на фазе анализа и проектирования можно писать тесты и налаживать тестовое окружение, при разработке и перед сдачей производить, собственно, само тестирование.

На настоящий момент для сложного ПО используются многомерные модели процесса, в которых отделение фаз от видов деятельности существенно облегчает управление разработкой ПО. К таким моделям относится, например, модель жизненного цикла решения MSF.

В каждом методе разработки ПО виды деятельности могут носить разные названия. В RUP они называются рабочими процессами (Work Flow), в CMM – ключевыми областями процесса (Key Process Area). В соответствии со стандартами ISO их называют процессами ЖЦ ПО, которые фактически являются подпроцессами процесса разработки ПО.

2.6. КАСКАДНАЯ МОДЕЛЬ ПРОЦЕССА РАЗРАБОТКИ ПО

Данная модель предложена в 1970 г. Винстоном Ройсом. Впервые в процессе разработки ПО были выделены различные шаги разработки и поколеблены примитивные представления о разработке ПО в виде анализа системы и ее кодирования.

Изначально каждое приложение представляло собой единое целое. Для разработки такого типа программ и применялся каскадный способ. Его основной характеристикой является разбиение всей разработки на этапы, причем **переход с одного этапа на следующий происходит только после полного завершения работ на текущем** (рис. 2.2). Каждый этап завершается выпуском полного комплекта документации, достаточной для того, чтобы разработка могла быть продолжена другой командой разработчиков.

Достоинством этой модели стало ограничение возможности возвратов на произвольный шаг назад, например, от тестирования – к анализу, от разработки – к работе над требованиями и т.д. Отмечалось, что такие возвраты могут катастрофически увеличить стоимость проекта и сроки его выполнения. Например, если при тестировании обнаруживаются ошибки проектирования или анализа, то их исправление часто приводит к полной переделке системы. В норме этой моделью допустимы возвраты только на предыдущий шаг.

Еще одно достоинство модели – введение прототипирования. То есть предлагалось разрабатывать систему дважды, чтобы уменьшить риски разработки. Первая версия – прототип – позволяет увидеть основные риски и обосновано принять главные архитектурные решения. На создание прототипа отводилось до одной трети времени всей разработки. В дальнейшем в связи с развитием программной инженерии и осознанием итеративного характера процесса разработки ПО эта модель активно критиковалась.

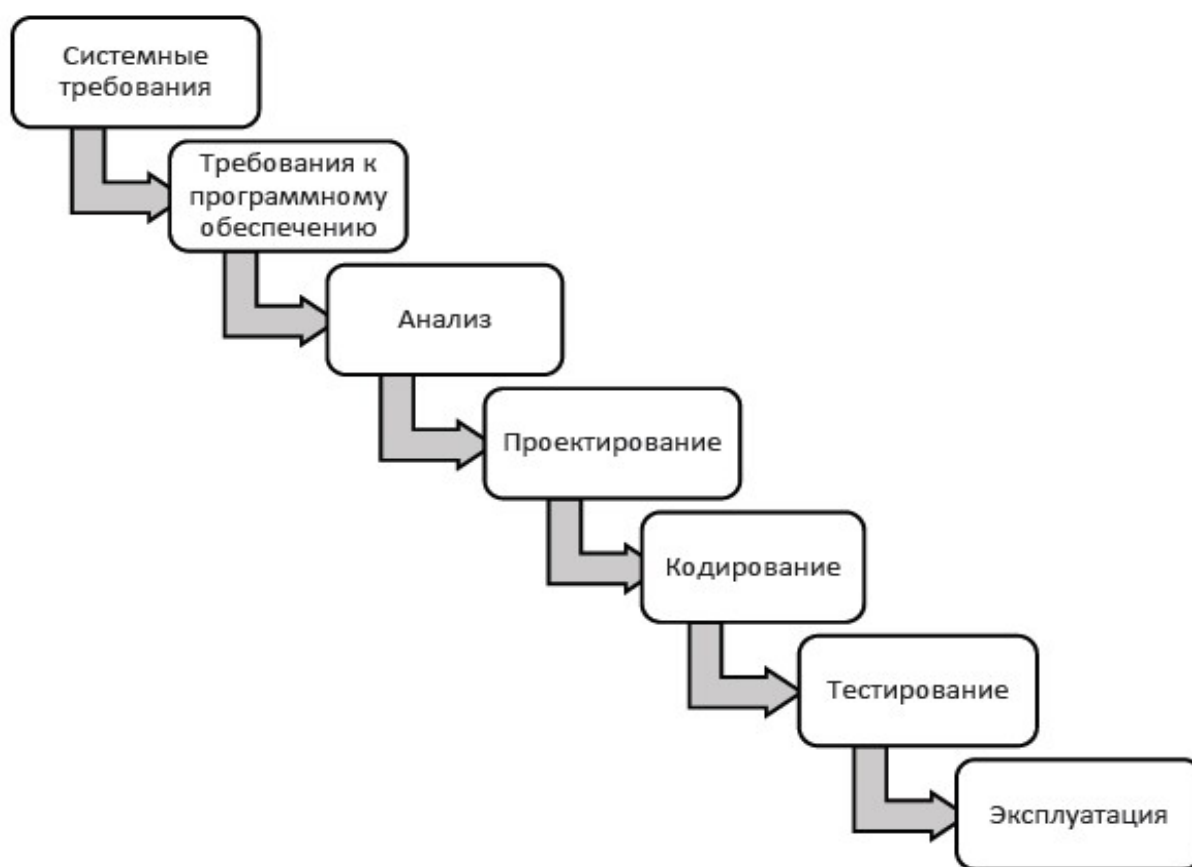


Рис. 2.2. Каскадная модель разработки ПО
(классический ЖЦ ИС по У. Ройсу)

Недостатки каскадной модели:

– отождествление фаз и видов деятельности, что влечет потерю гибкости разработки, в частности, трудности поддержки итеративного процесса разработки;

– требование полного окончания фазы, закрепление результатов в виде подробного документа: технического задания, проектной спецификации (опыт разработки ПО показывает, что невозможно полностью завершить разработку требований, дизайн системы и т.д. – все это подвержено изменениям; и причины тут не только в том, что подвижно окружение проекта, но и в том, что заранее не удастся точно определить и сформулировать многие решения, они проясняются и уточняются лишь впоследствии);

– интеграция всех результатов разработки происходит в конце, вследствие чего интеграционные проблемы дают о себе знать слишком поздно;

– пользователи и заказчик не могут ознакомиться с вариантами системы во время разработки и видят результат только в самом конце (тем самым, они не могут повлиять на процесс создания системы, и поэтому увеличиваются риски непонимания между разработчиками и заказчиком);

– модель неустойчива к сбоям в финансировании проекта или перераспределению денежных средств.

Однако данная модель продолжает использоваться на практике – для небольших проектов или при разработке типовых систем, где итеративность не так востребована. С ее помощью удобно отслеживать разработку и осуществлять поэтапный контроль за проектом.

2.7. СПИРАЛЬНАЯ МОДЕЛЬ ЖИЗНЕННОГО ЦИКЛА ПО

Спиральная модель ЖЦ ПО предложена Бэри Боемом в 1988 г. для преодоления недостатков каскадной модели, прежде всего, для лучшего управления рисками. Согласно этой модели, разработка продукта осуществляется по спирали, каждый виток которой – отдельная фаза разработки, в рамках которой может осуществляться много различных видов деятельности, т.е. модель двумерна. Главные отличия от каскадной модели: переход осуществляется в соответствии с планом, даже если не вся запланированная работа завершена.

Спиральная модель (рис. 2.3) делает упор на начальные этапы ЖЦ: анализ и проектирование. На этих этапах реализуемость технических решений проверяется путем создания прототипов. Каждый виток спирали соответствует созданию фрагмента или версии ПО, на нем уточняются цели и характеристики проекта, определяется его качество и планируются работы следующего витка спирали. Таким образом углубляются и последовательно конкретизируются детали проекта и в результате выбирается обоснованный вариант, который доводится до реализации.

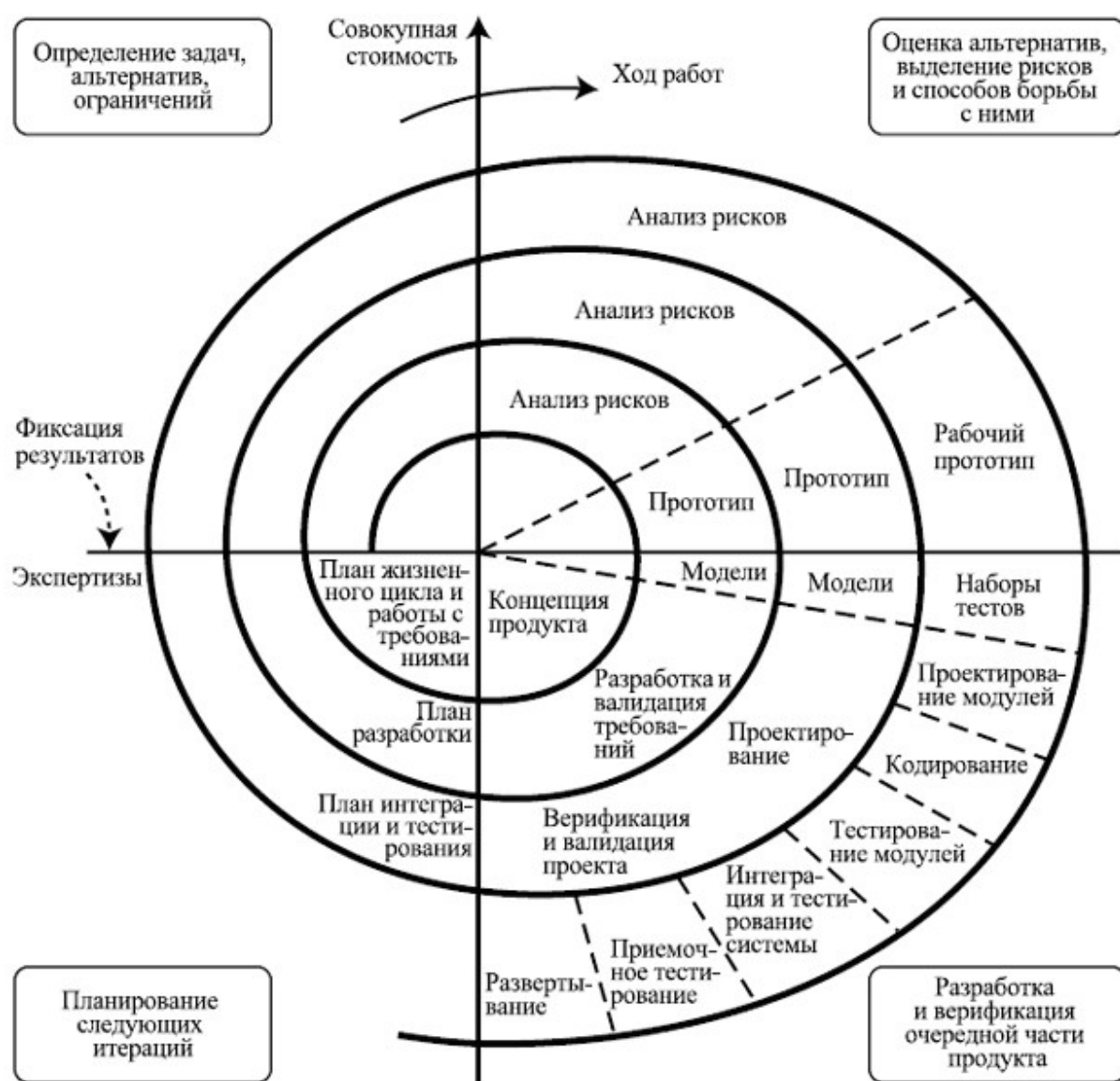


Рис. 2.3. Спиральная модель ЖЦ ПО

Как видно на рис. 2.3, спиральная модель состоит из четырех главных повторяющихся стадий. В ходе процесса разработки проект несколько раз проходит через все эти фазы. Каждая такая итерация называется «спиралью». Важным отличием спиральной модели ЖЦ является то, что количество требуемых итераций заранее неизвестно. Перед началом каждой итерации выполняется анализ полученных результатов и принимается решение о продолжении или прекращении разработки системы. Если цель достигнута и разработанная система полностью удовлетворяет потребностям заказчика, то разработка прекращается. Если же возникает необходимость доработки информационной системы, то процесс разработки переходит на следующий виток спирали.

Четыре главные фазы спиральной модели.

1. Определение целей, альтернатив, ограничений, или фаза планирования. С этой стадии начинается работа над проектом. Команда разработчиков формулирует цели проекта, основные требования (такие как, например, Business Requirement Specifications, или BRS, System Requirement Specifications, или SRS), возможный дизайн и т.д. На последующих спиралях требования формируются согласно отзывам, полученным от заказчика. Именно поэтому постоянная коммуникация между заказчиком и командой важна.

2. Анализ, определение и разрешение рисков является одной из самых значимых стадий разработки. В данном контексте риски – это возможные события и состояния проекта, препятствующие достижению командой разработчиков поставленных целей. Существует довольно обширный диапазон возможных рисков – от тривиальных и легко преодолимых до крайне серьезных. Главная задача для команды разработчиков – выявление всех возможных рисков и присвоение им определенного уровня приоритета на основе их значимости. Следующий шаг – разработка возможных стратегий преодоления этих рисков. В итоге этих действий возможны изменения в последующих стадиях разработки. В качестве результата работы

на этом этапе создается прототип. Подробнее о риск-менеджменте программного процесса излагается в главе 7.

3. Фаза разработки. На этом этапе происходит разработка и последующее тестирование продукта. Во время первой итерации, когда общие требования еще не четко сформулированы, разрабатывается так называемая концепция будущего продукта (Proof Of Concept), которая необходима для получения отзыва заказчика. На последующих витках спирали рабочие версии продукта, или билды (builds), отправляются заказчику. Это позволяет получить более детальный отзыв и четче сформулировать требования.

4. Планирование следующей фазы. Вся полученная информация используется для планирования дальнейших этапов разработки.

Спиральную модель часто называют метамоделью, поскольку в ней используются два подхода: каскадная модель и модель прототипирования. Но важно понимать, что спиральная модель не является простой последовательностью этапов разработки, следующих каскадной модели. На самом деле спиральная модель довольно гибкая. Стоит помнить, что схема, приведенная ранее, содержит определенные упрощения. Может показаться, что все стадии следуют одной спиральной последовательности. Но реальный жизненный цикл ПО более гибкий, чем это изображено на схеме. Существует даже возможность вернуться к предыдущим фазам в случае необходимости пересмотра принятых решений.

Так как в спиральной модели нет предопределенного и обязательного набора витков, каждый виток может стать последним при разработке системы. Последовательность витков может быть такой:

- 1) разработка прототипа с реализацией первичной функциональности ПО;
- 2) разработка первой версии программного продукта с ограниченной функциональностью;
- 3) разработка второй версии программного продукта в полной или более полной функциональности и т.д.

Витки могут иметь и иные значения. Каждый виток имеет определенную структуру (секторы), соответствующую основным процессам разработки ПО. Последний сектор предполагает принятие решения о целесообразности захода на новый виток.

Отдельная спираль может соответствовать разработке некоторого программного компонента или внесению очередных изменений в продукт. Таким образом, у модели может появиться третье измерение.

Разработка итерациями отражает объективно существующий спиральный цикл создания системы. Неполное завершение работ на каждом этапе позволяет переходить на следующий этап, не дожидаясь полного завершения работы на текущем этапе. Недостающую работу можно будет выполнить на следующей итерации. Главная же задача – как можно быстрее показать пользователям системы работоспособный продукт, тем самым активизируя процесс уточнения и дополнения требований.

Достоинства спиральной модели.

1. Мониторинг рисков является одной из главных особенностей, делающих данную модель привлекательной в том случае, если вам предстоит управление большим, сложным и дорогостоящим проектом. Более того, проект будет более прозрачным, поскольку спиральная модель изначально была спроектирована таким образом, чтобы каждая итерация тщательно анализировалась.

2. Заказчик может увидеть работающую версию продукта уже на ранних стадиях жизненного цикла ПО.

3. Изменения могут быть внесены на поздних стадиях разработки.

4. Проект может быть разделен на несколько частей, и те из них, которые, согласно анализу, окажутся более рискованными, могут быть реализованы на ранних стадиях. Такой подход может снизить трудности, связанные с управлением проектом.

5. Строгий контроль над документацией как результат постоянного анализа рисков.

Недостатки спиральной модели.

1. Мониторинг рисков требует дополнительных ресурсов, а значит, эта модель может оказаться весьма затратной. Каждая итерация требует отдельной экспертизы, что делает управление проектом сложнее. Именно поэтому спиральная модель плохо подходит для небольших проектов.

2. Большое количество промежуточных стадий разработки. Как следствие – большой объем документации.

3. На самых ранних стадиях дата завершения работы над проектом может быть неизвестна, что также усложняет контроль над процессом разработки.

Основная проблема спирального цикла – определение момента перехода на следующий этап. Для ее решения необходимо ввести временные ограничения на каждый из этапов жизненного цикла. Переход осуществляется в соответствии с планом, даже если не вся запланированная работа закончена. План составляется на основе статистических данных, полученных в предыдущих проектах, и личного опыта разработчиков.

Спиральную модель нецелесообразно применять:

- в проектах с небольшой степенью риска;
- в проектах с ограниченным бюджетом;
- для небольших проектов.

2.8. V-ОБРАЗНАЯ МОДЕЛЬ ЖИЗНЕННОГО ЦИКЛА ПО

V-образная модель (или V-модель) является модификацией каскадной модели. Её специфика – обеспечение тестирования системы на отдельных, главным образом, ранних стадиях проекта. Особенностью этой модели является акцент на процессы, связанные с верификацией документов разработки.

Верификация – это один из вспомогательных процессов жизненного цикла ПО, определяющий, отвечает ли текущее состояние разработки, достигнутое на данном этапе, требованиям данного этапа жизненного цикла.

Нисходящая ветвь модели описывает собственно разработку программной документации: требования к ПО, функции программных элементов, архитектуру – связи программных функций, программный код. Восходящая ветвь – этапы тестирования ПО (рис. 2.4).

В этой модели, подобно каскадной, каждый шаг начинается после завершения предыдущего. Модель имеет последовательную структуру цикла разработки. При проектировании V-образную модель выбирают чаще всего в ситуациях повышенных требований к качеству продукта.



Рис. 2.4. V-образная модель ЖЦ ПО

Модель основана на объединении фазы тестирования с каждой соответствующей стадией разработки. Разработка каждого шага напрямую связана с этапом тестирования. Каждый этап разработки напрямую связан с тестированием этого этапа.

На этапе анализа требований к системе разрабатывается план ввода в действие и обеспечения приемки продукта. Этап включает взаимодействие с заказчиком с целью выделить его

требования и ожидания от проекта. Иногда, в случае повышенной сложности проекта, выделяется предварительный этап сбора требований, как показано на рис. 2.4.

На этапе проектирования системы осуществляется разработка функциональных требований, проектирование системы и настройка оборудования и коммуникаций для разработки продукта, а также разрабатывается план квалификационных испытаний системы, этапы интеграционного тестирования.

На этапе проектирования архитектуры системы проектируются отдельные модули ПО и формируется план модульного тестирования. Выполняется системный дизайн, разбивка на модули, выполняющие различные функции, передача данных и связь между внутренними модулями и с другими системами.

На этапе разработки системы конструируется программный код, после чего запускается собственно процесс тестирования.

Этапы тестирования в V-модели.

1. Модульное тестирование. Этапы модульного тестирования разрабатываются на этапе проектирования модуля. Модульное тестирование выполняется для устранения ошибок на уровне кода или модуля.

2. Интеграционное тестирование выполняется после завершения модульного тестирования. При этом модули интегрируются и система тестируется. Интеграционное тестирование выполняется на этапе проектирования архитектуры. Этот тест проверяет связь модулей между собой.

3. После завершения интеграции проверяются функциональные и нефункциональные требования разработанного приложения в рамках системного тестирования.

4. Приемо-сдаточное (приемочное) тестирование. Выполняется в пользовательской среде, проверяет, что поставленная система соответствует требованиям пользователя и готова к использованию в реальном производственном процессе (в «боевом» режиме).

Подробные сведения о видах тестирования можно найти в главе 5.

Преимущества V-образной модели жизненного цикла. V-образная модель является развитием каскадной модели. Поэтому она обладает всеми её достоинствами. Кроме того, при подходящем использовании V-образная модель обеспечивает следующие преимущества:

- планирование на ранних стадиях разработки системы ее тестирования;
- обеспечение проверки, аттестации и верификации всех промежуточных результатов разработки;
- упрощение (по сравнению с каскадной моделью) отслеживания хода процесса разработки, возможность более реального использования графика проекта;
- простота в использовании.

Недостатки V-образной модели жизненного цикла. При использовании V-образной модели для несоответствующего ей проекта выявляются следующие ее недостатки:

- сложность поддержки параллельных событий (структура V-модели подразумевает последовательное выполнение этапов разработки с параллельной проверкой и верификацией на каждом из них, однако параллельные события требуют синхронизации между различными командами и этапами. Изменения в требованиях или дизайне могут повлиять на несколько параллельно выполняемых задач, что увеличивает сложность управления проектом);
- непредусмотренность итераций между фазами (V-модель изначально предполагает строгое последовательное выполнение этапов разработки: от анализа требований до проектирования, реализации, тестирования и поддержки);
- невозможность внесения динамических изменений в требования на разных этапах жизненного цикла;
- поздние сроки тестирования требований в жизненном цикле, что оказывает существенное влияние на график выполнения проекта при необходимости выполнить их изменения;

- отсутствие в модели действий, направленных на анализ рисков.

Общераспространенная модификация V-образной модели, направленная на преодоление ее недостатков, включает в себя внесение итерационных циклов для разрешения изменения в требованиях за рамками этапа анализа.

Область применения V-образной модели. Использование V-образной модели наиболее эффективно в следующих случаях:

- при разработке проектов, для которых требования максимально четко определены и доступны заранее;
- при разработке проектов, для которых определены и понятны методы реализации решения и технология, а разработчики имеют опыт в работе с данной технологией;
- при разработке систем, в которых требуется высокая надежность.

2.9. АРХИТЕКТУРА ПО. МНОЖЕСТВЕННОСТЬ ТОЧЕК ЗРЕНИЯ

Архитектура ПО – это сквозная концепция или комплекс таких концепций, предназначенных для преодоления энтропии и хаоса, стремящихся проглотить разработку в виду сложности, нематериальности, согласовываемости и изменчивости ПО.

Архитектура ПО – это внутренняя структура программного продукта (компоненты и их связи), основы пользовательского интерфейса продукта, а также средства разработки и управления проектом.

Часто под архитектурой понимают внутреннее устройство ПО, выраженное, например, в виде UML-диаграмм, в частности, диаграммы компонентов.

При разработке архитектуры ПО важным оказывается совмещение множества точек зрения. ПО может быть настолько сложным, что его архитектуру не построить как единую модель, связи между компонентами сложны и плохо выразимы в явном виде. Важнее оказывается построение множества моделей,

созданных с разных точек зрения, прежде всего, из-за разных видов деятельности процесса разработки ПО (табл. 2.2).

Кроме того, в ЖЦ ПО вовлечено большое количество очень разных специалистов (программисты, инженеры, тестеры, технические писатели, менеджеры, заказчик, пользователи, продавцы-маркетологи и т.д.), для которых нужна разная информация о программной системе.

Таблица 2.2

Точки зрения на ПО в разных видах деятельности
программного процесса

Вид деятельности	Точка зрения на ПО
Составление функциональных требований к ПО	Важно, какая именно функциональность должна быть реализована, но при этом опускаются принципы и детали реализации
Проектирование	На первое место выходят принципы реализации ПО
Тестирование	Детали реализации снова неважны – на ПО смотрят как на черный ящик, реализующий (неважно, каким способом) некоторый набор пользовательской функциональности
Развертывание	Набор компонентов: файлов, документов, хранилищ данных и т. д.

Множественность точек зрения происходит также от того, что нет единых стандартов и норм разработки ПО для любой предметной области. Часто приходится изобретать новую точку зрения прямо по ситуации – чтобы именно этот эксперт понял суть продукта, чтобы именно эти особенности программы были отражены. Часто здесь – как в лотерее: создается несколько описаний системы с разных точек зрения, какое-то оказывается удачным и его все используют в дальнейшем.

Точка зрения (viewpoint) – это специальный термин программной инженерии, выражает определенный взгляд на систему, который осуществляется для выполнения какой-то определенной задачи кем-либо из участников проекта.

Точку зрения нужно ясно осознавать при создании визуальных моделей, например, модели вариантов использования UML. Важно понимать, что она может быть в каждом конкретном случае своя. Важнейшими характеристиками точки зрения моделирования является цель (зачем создается модель) и целевая аудитория (т.е. для кого она предназначена).

Методика. Для ориентации на целевую аудиторию выбирают одного представителя такой аудитории и создают модели, понятные именно ему. При этом важно не обсуждать чрезмерно с ним модели, поскольку это может создать дополнительный контекст, которого другие пользователи моделей будут лишены. Полезно представлять, воображать себе этого человека при работе над моделями – его реакции, вопросы, недоумения и пр., и, исходя из этого, корректировать, исправлять созданное. И, конечно, полезно проверить свои предположения, показав ему, что получилось.

Кроме того, важно, чтобы точка зрения была «живая», а не выдумывалась аналитиком или копировалась из книг и тренингов по UML. Незаметно для себя аналитик может придумать собственный проект, своих пользователей системы, заказчика и т.д. То есть аналитик навязывает самому себе определенное восприятие существующих людей, задач, сильно искажая реальное положение дел. И именно в контексте этой воображаемой ситуации он создает свои модели.

Следует помнить, что люди, разные реальные ситуации имеют большой диапазон вариативности. Соответственно, аналитик должен обладать гибкостью мышления, большим диапазоном техник, а также чуткостью и искренним стремлением к тому, чтобы сделать каждый проект, где он участвует, более гармоничным, адекватным.

3. УПРАВЛЕНИЕ ТРЕБОВАНИЯМИ

3.1. ЗАДАЧА УПРАВЛЕНИЯ ТРЕБОВАНИЯМИ К ПО

Несмотря на постоянный и значительный прогресс в индустрии программного обеспечения, многие компании-разработчики все еще испытывают трудности с выявлением, сбором, записью и управлением требованиями к программному обеспечению. Из-за неполной формулировки требований, изменения требований и недостаточной информации от пользователей компании-разработчики не всегда могут завершить работу в срок и предоставить заказчику требуемые функции. Управление требованием – один из важнейших этапов системного проектирования и краеугольный камень успеха любого проекта. Спрос позволяет получить наиболее интуитивно понятное и последовательное представление продукта, сокращая цикл разработки и время выхода на рынок.

Управление требованиями – это процесс выявления, записи, анализа, отслеживания, определения приоритета требований, достижения согласия по требованиям, а затем управления изменениями и уведомления заинтересованных сторон.

Задача управления требованиями к ПО возникает по следующим причинам:

- значительное разнообразие программных систем, которые создает одна компания, одна команда;
- большой дефицит программистов и команд, хорошо разбирающихся в специфике некоторых предметных областей;
- большие трудности при формулировании адекватных требований к ПО для принципиально новых предметных областей;
- трудности в понимании между заказчиком и программистами;
- существенное влияние индивидуальности заказчика – персональной или его компании, которое сильно связано со

спецификой бизнеса и используемого в этой области оборудования;

– изменчивость ПО – зачастую требования имеют тенденцию меняться в ходе разработки.

В большинстве случаев далеко не очевидно, что ту систему, которую хочет заказчик, вообще можно сделать. Также нет гарантий, что программа, построенная так, как поняли и реализовали разработчики, окажется удобным, востребованным на рынке инструментом.

Ошибки и разночтения, которые возникают при выявлении требований к системе, оказываются одними из самых дорогих.

Требование – исходное понимание задачи разработчиками, которому должна соответствовать система или программный продукт и которое является основой всей разработки.

В соответствии со стандартом ISO/IEC 29148, требования представляют собой четкие, проверяемые и измеримые утверждения, которые идентифицируют рабочие характеристики, функциональные параметры и конструктивные ограничения продукта или процесса.

Управление требованиями включает общение между командой проекта и заинтересованными сторонами для корректировки требований на протяжении всего проекта. Важно непрерывное общение между всеми участниками проекта, чтобы ни одна потребность не перевешивала другие потребности. Например, при разработке ПО для внутреннего использования у бизнеса может быть такой высокий спрос, что он может игнорировать потребности пользователя или предполагать, что созданные варианты использования также будут покрывать потребности пользователя.

На взаимопонимание заказчика и разработчиков оказывает влияние большой разрыв между программистами и другими людьми в двух аспектах.

1. Компетентность разработчиков в данной предметной области. Для успешного решения задачи создания ПО в той или

иной предметной области необходимо в нее вникнуть, поработать в ней достаточное время и пр.

2. Специфичность программирования как сферы деятельности. Для большинства пользователей и заказчиков крайне не просто сформулировать точное знание, которое необходимо программистам.

Причины изменчивости ПО:

- изменение ситуации в предметной области, для которой предназначается система;
- перманентная изменчивость требований к системе из-за быстро меняющихся перспектив продажи еще неготовой системы;
- возникновение в ходе разработки проблем и трудностей, из-за которых итоговая функциональность меняется (видоизменяется, урезается, расширяется);
- внезапное изменение заказчиком своего видения системы.

Изменчивость требований в процессе разработки сказывается на продукте. Часто по ходу работы над проектом разработчикам поступает огромный поток дополнительных требований. Все их реализовать в полном объеме не удастся, в итоге система оказывается набором демофункциональности. Это требует особого подхода к задаче управления требованиями к ПО.

3.2. ВИДЫ ТРЕБОВАНИЙ К ПО

Схема многообразия требований к ПО представлена на рис. 3.1.

Бизнес-требования определяют назначение ПО, излагаются в документе о видении (vision) и границах проекта (scope). Бизнес-требования описывают, почему организации нужна такая система, т.е. какие цели преследует организация. Основное содержание бизнес-требований – бизнес-цели организации или клиента, заказывающих систему. Это верхний уровень абстракции требований к системе. Они не относятся напрямую к

реализации проекта, а в первую очередь отражают цели бизнеса, абстрагированные от реализации системы и ограниченные определенным бизнес-процессом.

Бизнес-процесс – совокупность взаимосвязанных мероприятий или задач, направленных на создание определенного продукта или услуги для потребителей. Бизнес-процессы лежат в основе любой производственной деятельности. Их также называют управленческими функциями, каждая из которых представляет собой процесс.



Рис. 3.1. Многоуровневая структура требований к ПО

В конечном итоге бизнес-требования формируют документ концепции и границ.

Примеры бизнес-требований:

1. Создать промо-сайт, привлекающий внимание определенной аудитории к конкретной продукции компании (идея).

2. Сократить время обработки заказа на 50 % (цель) – система должна предоставить интерфейс, упрощающий создание заказа (концепция).

3. Увеличить клиентскую конверсию до 35 % (цель) – в системе должны быть представлены механизмы побуждения клиента к заказу (концепция).

Пользовательские требования (user requirements) определяют набор пользовательских задач, которые должна решать программа, а также способы (сценарии) их решения в системе. Пользовательские требования могут выражаться в виде фраз утверждений, способов применения (use case), пользовательских историй (user story), сценариев взаимодействия (scenario). Область пользовательских требований также включает описания атрибутов или характеристик продукта, которые важны для пользователей. Пользовательские требования часто именуются фичами.

Фича (функциональность) – функционально обобщенная часть системы, решающая отдельную задачу пользователей или интерпретирующая бизнес-процессы (и их артефакты), которые будут реализованы в рамках системы.

Исходя из описанных определений, пользовательские требования содержат:

- цели и задачи пользователей;
- сценарии использования – способ решения задачи пользователей;
- описание самих пользователей системы: пользовательские роли и уровни доступа.

В конечном итоге пользовательские требования формируют «Документ пользовательских требований». Пользовательские требования могут быть представлены в виде:

- текстового описания;
- вариантов использования, сценариев использования (Use Case);
- пользовательских историй (User Story);
- диаграммы вариантов использования.

Как правило, пользовательские требования описываются по следующему шаблону: *Пользователь должен иметь возможность + {тезис}*.

Примеры пользовательских требований.

1. Пользователь должен иметь возможность добавить объект в избранное (функциональность).

2. Система должна представлять пользователю диалоговые окна для ввода исчерпывающей информации о заказе и последующей фиксации этой информации в БД.

Источники пользовательских требований (где искать?):

- анализ и декомпозиция бизнес-требований;
- описание бизнес-процессов;
- артефакты бизнес-процессов: документы входные / выходные, стандарты, регламенты, обрабатываемые данные, иная информация, используемая и / или производимая в бизнес-процессе;
- отраслевые стандарты;
- реализованные проекты, проекты конкурентов.

Функциональные требования представляют собой детальное описание поведения и сервисов программной системы, ее функционала. Они определяют, какие функции система должна выполнять.

Функциональные требования определяют, каким должно быть поведение продукта в тех или иных условиях. Это самые низкоуровневые требования, являются результатом декомпозиции верхнеуровневых требований и описывают атомарные функции, которые должны быть реализованы в системе. Они определяют, что разработчики должны создать, чтобы пользователи смогли выполнить свои задачи (пользовательские требования) в рамках бизнес-требований.

Функциональные требования описывают следующие характеристики:

- функция или объект функционирования;
- входные данные и источники;
- выходные данные и пункт их назначения;

- средства для выполнения функции;
- описание предварительных и заключительных условий (предусловий), если функция специфицирована.

Примеры функциональных требований.

1. Система должна по электронной почте отправлять пользователю подтверждение о заказе или заказ может быть создан, отредактирован, удален и перемещен с участка на участок.

2. Операция продажи должна сопровождаться формированием и выводом на печать фискального чека и расходной накладной.

Больше примеров корректного формулирования функциональных требований содержится в параграфе 3.7.

Источники функциональных требований (где искать?):

- анализ пользовательских требований;
- расширенные описания пользовательских задач (таски);
- прототипы интерфейса (мокапы);
- отраслевые и корпоративные стандарты на производство продукции;
- внешние системы и документация к ним (интеграции).

Функциональные требования являются основными, документируются в спецификации требований к ПО (software requirements specification, SRS). Так же как и для пользовательских требований, для них строятся диаграммы вариантов использования. Требуемое поведение системы специфицируется одним или несколькими вариантами использования, которые определяются в соответствии с потребностями акторов. Пример модели требований в нотации UML показан на рис. 3.2.

Нефункциональные требования не являются описанием функций системы. Этот вид требований описывает качественные характеристики программной системы. Наиболее очевидны из них следующие:

1. Сопровождаемость (maintainability) – означает, что программа должна быть написана с расчетом на дальнейшее развитие. Это критическое свойство системы, так как изменения

ПО неизбежны вследствие изменения бизнеса. Сопровождение программы выполняют, как правило, не те люди, которые ее разрабатывали. Включает такие элементы:

- наличие и понятность проектной документации;
- соответствие проектной документации исходному коду;
- понятность исходного кода;
- простота изменений исходного кода;
- простота добавления новых функций.

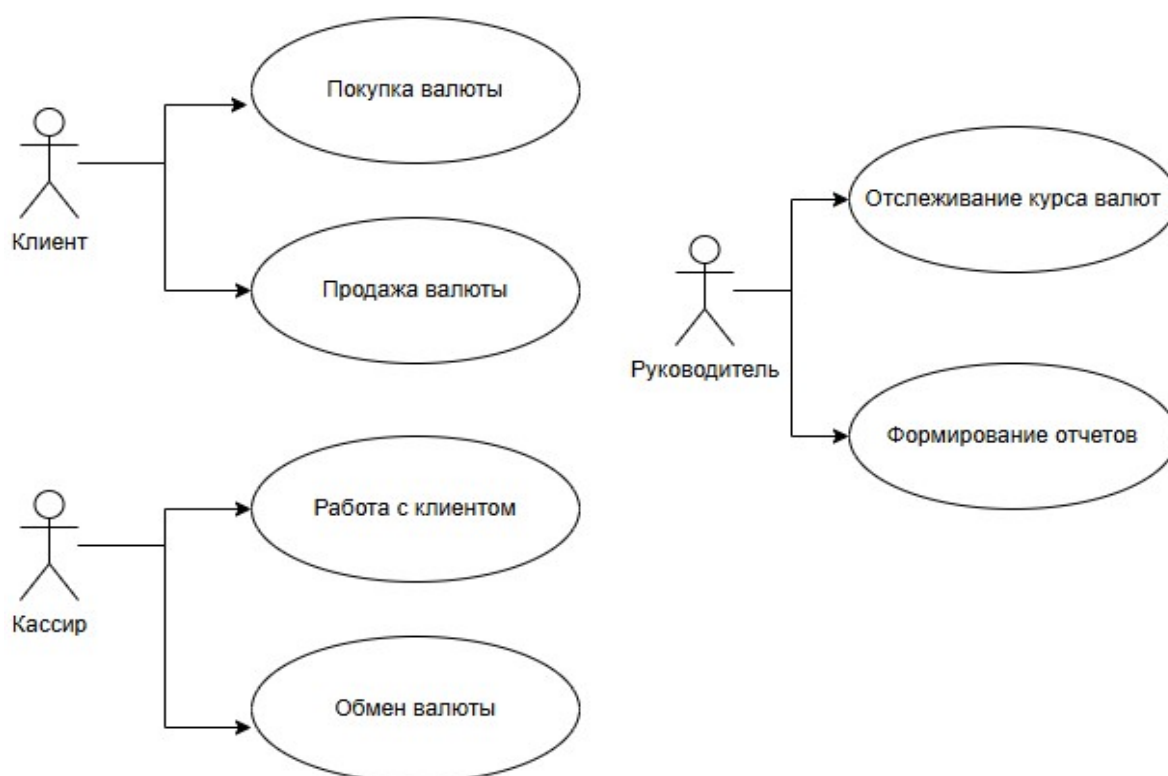


Рис. 3.2. Модель требований UML для ПС выполнения операции обмена валюты

2. Надежность (dependability). Надежность ПО – комплексное свойство, включает такие элементы, как:

- отказоустойчивость – возможность восстановления программы и данных в случае сбоев в работе;
- безопасность – сбои в работе программы не должны приводить к опасным последствиям (авариям);

– защищенность от случайных или преднамеренных внешних воздействий (защита от «дурака», вирусов, спама).

3. Эффективность (efficiency). ПО не должно впустую тратить системные ресурсы, такие как память, процессорное время, каналы связи.

Эффективность ПО – степень его приспособленности к достижению цели, проявляется только при функционировании ПО и зависит от его свойств, способа применения и от воздействий внешней среды. В общем случае характеризует качество его функционирования, оцениваемое как соответствие требуемого и достигаемого результата, которые могут различаться.

Эффективность ПО оценивается следующими показателями:

- время выполнения кода;
- загруженность процессора;
- объем требуемой памяти;
- время отклика и т.п.

Данные показатели определяют свойства ПО, в том числе и по отношению к внешней среде.

Внешняя среда – множество объектов предметной области, которые оказывают влияние на ПО и сами находятся под его воздействием.

4. Удобство использования (usability). ПО должно быть легким в использовании: интерфейс пользователя и документация. Причем пользовательский интерфейс должен быть не интуитивно, а **профессионально понятным** пользователю.

5. Особенности поставки – наличие инсталлятора, документации.

6. Определенный уровень качества. Например, для новой Java-машины это будет означать, что она удовлетворяет набору тестов, поддерживаемому компанией Sun.

7. Требования к средствам и процессу разработки системы.

8. Требования к переносимости.

9. Требования соответствию стандартам и т.д.

Требования этого вида часто относятся ко всей системе в целом. На практике особенно начинающие специалисты часто забывают о некоторых важных нефункциональных требованиях. Более полный перечень нефункциональных требований представлен в прил. А.

Реализация нефункциональных требований часто ведет к бóльшим затратам, чем функциональных. Так, сопровождаемость требует значительных усилий по поддержанию соответствия проекта исходному коду и применения специальных методов создания модифицируемых программ; надежность – дополнительных средств восстановления системы при сбоях; эффективность – поиска специальных архитектурных решений и оптимизации кода; удобство – проектирования не «интуитивно» понятного, а профессионально понятного интерфейса пользователя.

3.3. ОСНОВНЫЕ ТРУДНОСТИ ПРИ ФОРМИРОВАНИИ ТРЕБОВАНИЙ К ПО

Все трудности в работе в той или иной степени связаны с главной проблемой программной инженерии: поиском универсального метода и процесса, пригодного для создания программного продукта любого типа в любых условиях. Здесь главная проблема – меняющиеся условия. В этой связи разработчики ПО сталкиваются со следующими трудностями.

1. Наследование ранее созданного ПО (legacy systems). Существует достаточно много систем, созданных много лет назад, морально устаревших, но продолжающих работать. Проблема наследования таких систем состоит в их сопровождении – поддержке и развитии.

2. Разнородность программных систем. ПО должно работать в распределенных сетях, на разнородном оборудовании, в разных средах, под управлением различных операционных систем и разных категорий пользователей.

3. Сокращение времени на разработку. Запросы рынка требования к программным системам меняются очень быстро.

Суть проблемы состоит в том, чтобы сократить время разработки ПО без снижения его качества.

Эти трудности часто оказываются связанными между собой. Задача разработки сетевого варианта старой локальной базы данных в ограниченные сроки достаточно типична.

3.4. СВОЙСТВА ТРЕБОВАНИЙ К ПО

Сформулируем ряд важных свойств требований.

1. Ясность, недвусмысленность – однозначность понимания требований заказчиком и разработчиками. Часто этого трудно достичь, поскольку конечная формализация требований, выполненная с точки зрения потребностей дальнейшей разработки, трудна для восприятия заказчиком или специалистом предметной области, которые должны проинспектировать правильность формализации.

2. Полнота и непротиворечивость – требует полного определения в одном месте, вся необходимая информация присутствует. Также требования не противоречат другим требованиям и полностью соответствуют документу.

3. Необходимый уровень детализации. Требования должны обладать ясно осознаваемым уровнем детализации, стилем описания, способом формализации. Возможные варианты:

- описание свойств предметной области, для которой предназначается ПО;
- техническое задание, которое прилагается к контракту;
- проектная спецификация, которая должна быть уточнена в дальнейшем, при детальном проектировании и т.д.

Важно также ясно видеть и понимать тех, для кого данное описание требований предназначено, иначе не избежать недопонимания и последующих за этим трудностей. Ведь в разработке ПО задействовано много разных специалистов – инженеров, программистов, тестировщиков, представителей заказчика, возможно, будущих пользователей – и все они имеют разное образование, профессиональные навыки и специализацию, часто говорят на разных языках. Здесь также важно, чтобы

требования были максимально абстрактны и независимы от реализации.

4. Прослеживаемость – важно видеть то или иное требование в различных моделях, документах, наконец, в коде системы. Например, часто возникают вопросы типа «Кто знает, почему мы решили, что такой-то модуль должен работать следующим образом?». Прослеживаемость функциональных требований достигается путем их дробления на отдельные, элементарные требования, присвоения им идентификаторов и создания трассировочной модели, которая в идеале должна протягиваться до программного кода. Например, полезно знать, где нужно изменить код, если данное требование изменилось.

На практике полная формальная прослеживаемость труднодостижима, поскольку логика и структура реализации системы могут сильно не совпадать с таковыми для модели требований. В итоге одно требование оказывается сильно распределено по тексту программы, а тот или иной участок кода может влиять на много требований. Но стремиться к прослеживаемости необходимо, разумно совмещая формальные и неформальные подходы.

5. Тестируемость и проверяемость – необходимо, чтобы существовали способы оттестировать и проверить данное требование. Причем важны оба аспекта, поскольку часто проверить заказчик может, а вот тестировать данное требование очень трудно или невозможно ввиду ограниченности доступа (например, по соображениям безопасности) к окружению системы для команды разработчика. Необходимые процедуры проверки требований:

- выполнение тестов;
- проведение инспекций кода;
- проведение формальной верификации части требований;
- определение рационального порога качества (чем выше качество, тем оно дороже стоит);
- определение критериев полноты проверок, чтобы выполняющие их и руководители проекта четко осознавали, что именно проверено, а что еще нет; и пр.

6. Модифицируемость – определяет процедуры внесения изменений в требования.

7. Единичность – требование описывает одну и только одну функциональность.

8. Актуальность – требование не стало устаревшим с течением времени.

9. Завершённость – требование полностью определено в одном месте, и вся необходимая информация присутствует.

10. Атомарность – требование не может быть разбито на ряд более детальных требований без потери завершённости.

3.5. ВАРИАНТЫ ФОРМАЛИЗАЦИИ ТРЕБОВАНИЙ

Требования как таковые – это некоторая абстракция. В реальной практике они всегда существуют в виде некоторого представления – документа, модели, формальной спецификации, списка и т.д. Требования важны, потому что дают понимание разработчикам нужд заказчика и будущих пользователей ПО. Но так как в программном процессе много различных аспектов, видов деятельности и фаз разработки, то это понимание может принимать разные формы. Каждое представление требований выполняет определенную задачу, например:

- служит мостом, фиксацией соглашения между разными группами специалистов;

- используется для оперативного управления проектом (отслеживается, в какой фазе реализации находится то или иное требование, кто за него отвечает и пр.);

- используется для верификации и модельно-ориентированного тестирования.

Во всех случаях мы имеем дело с требованиями, но формализованы они будут по-разному. Формализация требований в проекте может быть очень разной – это зависит от его величины, принятого процесса разработки, используемых инструментальных средств, а также тех задач, которые решают формализованные требования.

Более того, может существовать параллельно несколько формализаций, решающих различные задачи. Рассмотрим варианты.

1. Неформальная постановка требований в переписке по электронной почте. Хорошо работает в небольших проектах, при вовлеченности заказчика в разработку (например, команда выполняет субподряд). Хорошо также при таком стиле, когда есть взаимопонимание между заказчиком и командой (лишние формальности не требуются). Однако электронные письма в такой ситуации часто оказываются важными документами. Необходимо уметь вести деловую переписку, подводить итоги, хранить важные письма и пользоваться ими при разногласиях. Важно также вовремя понять, когда такой способ перестает работать и необходимы более формальные подходы.

2. Требования в виде документа – описание предметной области и ее свойств, техническое задание как приложение к контракту, функциональная спецификация для разработчиков и т.д.

3. Требования в виде графа с зависимостями в одном из средств поддержки требований (IBM Rational RequisitePro, DOORS, Borland CaliberRM и др.). Такое представление удобно:

- при частом изменении требований;
- при отслеживании выполнения требований;
- при организации привязки к требованиям задач, людей, тестов, кода.

Важно также, чтобы была возможность легко создавать такие графы из текстовых документов и, наоборот, создавать презентационные документы по таким графам.

4. Формальная модель требований для верификации, модельно-ориентированного тестирования и т.д.

Итак, каждый способ представления требований должен отвечать на следующие вопросы:

- Кто потребитель, пользователь этого представления?
- Зачем, с какой целью это представление используется?
- Как это представление используется?

3.6. ОШИБКИ ПРИ ДОКУМЕНТИРОВАНИИ ТРЕБОВАНИЙ

К сожалению, в процессе составления требований также могут возникать проблемы или же ошибки. Это связано со многими моментами самого процесса составления, так как он очень объемен и зависит напрямую от пожеланий заказчика, будь то человек или компания. Диапазон причин огромен, и в данном параграфе представлен их систематизированный анализ.

Перечислим ряд ошибок, встречающихся при составлении технических заданий и иных документов с требованиями. Первые три проблемы возникают, когда заказчик не указывает четких требований.

1. Описание возможных решений вместо требований. В данном случае заказчик предлагает множество возможных решений. При этом используются формулировки в настоящем времени, что является нарушением формального описания требований. В таком случае заказчику задается вопрос «Зачем заказчику нужен исполнитель?». Этот момент также создает некую абстракцию для исполнителя ввиду множественной вариативности желаний со стороны заказчика.

Данная проблема разрешается путем разграничения ролей в процессе общения с заказчиком и уточнением требований самого заказчика к готовому продукту.

Неправильно	Правильно
В систему вводятся данные о поставщиках комплектующих. При этом производится контроль корректности данных (отсутствие дублирования записей, корректность банковских реквизитов и т.п.). Данные сохраняются в БД для долгосрочного хранения	Система должна обеспечивать корректный ввод данных о поставщиках комплектующих: наименование, ИНН, юридический и физический адрес, телефоны, банковские реквизиты. Требования к корректности ввода: – отсутствие дублирования записей; – корректность ИНН; – корректность реквизитов банка. В результате ввода данные должны сохраняться в БД

При включении зажигания компьютер опрашивает состояние датчика присутствия водителя на сидении автомобиля. Если водитель зафиксирован, то фиксируется изображение лица водителя	Фиксация изображения лица водителя должна выполняться при поступлении сигналов с блока зажигания и датчика присутствия водителя на сидении автомобиля
---	---

2. Обобщенные требования, отсутствие конкретики по существу их реализации. В данном случае заказчик так и не смог определиться с тем, что он хочет получить в качестве продукта, будь то программное обеспечение или приложение. Таким образом, разработка делается невозможной или максимально абстрактной. Формулировки таких требований, как правило, очень краткие и включают общие фразы и обобщающие эпитеты.

Неправильно	Правильно
При покупке товара клиенту выдается соответствующий документ	Операция продажи должна сопровождаться формированием и выводом на печать фискального чека и расходной накладной. Форматы выходных документов представлены в приложении
Необходимо обеспечить ввод в систему исходных данных	Система должна предусматривать ввод и сохранение в БД входных данных: – данные о пациентах (ФИО, дата рождения, паспортные данные, адрес проживания, адрес регистрации, место работы или учебы, контактный телефон); – данные о врачах (ФИО, дата рождения, паспортные данные, специальность, квалификация, адрес, телефон, предыдущая трудовая деятельность); – данные о записи на приём (ФИО пациента, ФИО врача, дата, время). При записи на прием должны быть зафиксированы дата и время внесения записи, а также присвоен номер талона

3. Нечеткие требования способствуют недосказанности, имеют оттенок советов, обсуждений, рекомендаций: «Возможно, что имеет смысл реализовать также...» и т.д. Они не дают однозначного понимания и не допускают однозначную проверку.

Проблему отсутствия чётких требований можно разрешить путем наличия в группе разработчиков человека, имеющего контакт с заказчиком. Постоянный контакт с заказчиком позволит сформировать как сами требования, так и лучше понять моменты разработки, например, какую технологию стоит применить в процессе разработки и на каком языке программирования это будет лучше сделать.

Неправильно	Правильно
Входные данные должны быть организованы в виде вводимого в базу данных текста или файла, соответствующего определенному шаблону	Входные данные о вакансиях должны быть представлены в формате CSV. Последовательность значений должна определяться порядком расположения полей таблицы вакансий в базе данных ИС
Изображение лица водителя должно быть зафиксировано четко для качественного его распознавания	Разрешение фиксируемого изображения должно быть не менее 300 dpi

4. Игнорирование аудитории, для которой предназначено представление требований. Данная проблема связана со сферой деятельности составителя требований. Например, если спецификацию составляет инженер заказчика, то часто встречается переизбыток информации об оборудовании, с которым должна работать программная система, отсутствует глоссарий терминов и определений основных понятий, используются многочисленные синонимы и т.д. Или допущен слишком большой уклон в сторону программирования, что делает данную спецификацию непонятной всем непрограммистам.

Данная проблема разрешается путем разграничения ролей в процессе общения с заказчиком и уточнением требований самого заказчика к готовому продукту.

Неправильно	Правильно
Мастер для подтверждения приема заказа на выполнение должен посмотреть объект для ремонта, определить, какие материалы ему нужны и в каком количестве. После осмотра объекта статус становится «в работе», при этом клиенту приходит сообщение об этом	Подтверждение приема заказа должно осуществляться мастером на основе результатов обследования объекта. При этом в систему дополнительно должны быть внесены данные о необходимых материалах. После подтверждения приема заказа нажатием на кнопку «Подтвердить» статус заказа становится «В работе» и формируется сообщение клиенту (требования к формированию сообщения клиенту см. в п. 3.1)

5. Пропуск важных аспектов, связанных с нефункциональными требованиями, в частности:

- требуемых состава БД и специального ПО;
- требуемого взаимодействия подсистем программного средства, в том числе посредством телекоммуникаций;
- информации об окружении системы;
- требования к защите информации;
- сроков готовности других систем, с которыми должна взаимодействовать данная.

Это случается, например, когда данная программная система является частью более крупного проекта. Типичны проблемы при создании программно-аппаратных систем, когда аппаратура не успевает вовремя и ПО невозможно тестировать, а в сроках и требованиях это не предусмотрено.

Данная ошибка может привести к полной или частичной неработоспособности программного обеспечения, в случае если не были рассмотрены моменты взаимодействия с ПО. Подобная проблема разрешима путем уточнения данных моментов у

заказчика, с отсылкой на адаптивность ПО под сферу использования.

3.7. ЦИКЛ РАБОТЫ С ТРЕБОВАНИЯМИ

В программной инженерии определяются следующие виды деятельности при работе с требованиями:

- выделение требований (requirements elicitation) – выявление всех возможных источников требований и ограничений на работу системы;
- извлечение требований из этих источников;
- анализ требований (requirements analysis), целью которого является обнаружение и устранение противоречий и неоднозначностей в требованиях, их уточнение и систематизация;
- описание требований (requirements specification), в результате чего требования должны быть оформлены в виде структурированного набора документов и моделей, который может систематически анализироваться, оцениваться с разных позиций и в итоге должен быть утвержден как официальная формулировка требований к системе;
- валидация требований (requirements validation), которая решает задачу оценки понятности сформулированных требований и их характеристик, необходимых, чтобы разрабатывать ПО на их основе, в первую очередь, непротиворечивости и полноты, а также соответствия корпоративным стандартам на техническую документацию.

Задача формирования требований к ПО – одна из трудно формализуемых, наиболее дорогих и сложных для исправления в случае ошибки. Современные инструментальные средства и программные продукты позволяют достаточно быстро создавать ПО по готовым требованиям. Но зачастую такие системы не удовлетворяют заказчиков, требуют многочисленных доработок, что приводит к резкому удорожанию фактической стоимости ПО. Основной причиной такого положения служит неправильное,

неточное или неполное определение требований к ПО на начальном этапе жизненного цикла.

Блок-схема цикла управления требованиями показана на рис. 3.3. Этап разработки требований состоит из следующих шагов:

- анализ предметной области, в рамках которой будет эксплуатироваться система;
- сбор требований – это процесс взаимодействия с лицами, формирующими требования;
- классификация требований – здесь бесформенный набор требований преобразуется в логически связанные группы требований;
- разрешение противоречий – требования лиц, занятых в процессе формирования требований, могут быть противоречивыми; на данном шаге определяются и разрешаются противоречия такого рода;
- назначение приоритетов – определяются наиболее важные требования, реализовать которые необходимо в первой версии программы;
- проверка требований на полноту, последовательность и непротиворечивость.

Ошибки, допущенные на стадии сбора требований, составляют от 40 до 60 % всех дефектов проекта (Davis, 1993; Leffingwell, 1997). Frederick Brooks выразительно определил критическую роль требований при разработке ПО в классическом эссе (1987) «No Silver Bullet: Essence and Accidents of Software Engineering»: «Строжайшее и единственное правило построения систем ПО – решить точно, что же строить. Никакая другая часть концептуальной работы не является такой трудной, как выяснение деталей технических требований, в том числе и взаимодействие с людьми, с механизмами и с иными системами. Никакая другая часть работы так не портит результат, если она выполнена плохо. Ошибки никакого другого этапа работы не исправляются так трудно».



Рис. 3.3. Цикл управления требованиями к ПО

Разработчики ПО при формировании требований могут столкнуться со следующими проблемами, приводящими к пересмотру списка требований и изменению в ИС.

1. Недостаточное вовлечение пользователей на всех этапах разработки требований. Для решения данной проблемы необходимо:

- определить различные классы пользователей ИС;
- выделить основные источники получения информации о потребностях клиентов;
- выбрать представителей каждого класса пользователей и прочих групп заинтересованных лиц и поработать с ними;
- согласовать с пользователями ответственных за принятие решений по проекту.

В качестве дополнительных классов пользователей можно рассматривать сторонние приложения и аппаратные компоненты, с которыми взаимодействует система. Например, ИС «Деканат», использующая информацию об индивидуальном образовательном маршруте студентов, представляет собой пользовательский класс ИС «Индивидуальный образовательный

маршрут». Требования к ИС «Индивидуальный образовательный маршрут» необходимо уточнить у разработчиков ИС «Деканат».

2. Увеличение требований пользователей. Так как требования тщательно прорабатываются и их объем со временем увеличивается, проект часто выходит за установленные рамки как по срокам, так и по бюджету. Изменение списка требований часто приводит к добавлению программного кода, что может стать причиной нарушения принципов дизайна и разрыва логических связей внутри ИС. Чтобы минимизировать возможность потери качества ПО, необходимо проанализировать, как возможные изменения отразятся на архитектуре и дизайне, а затем реализовать их непосредственно в программном коде.

3. Двусмысленность требований. Пользователи ИС могут интерпретировать одно и то же положение по-разному, у нескольких читателей требований возникает разное представление о продукте. Кроме того, двусмысленность зачастую является следствием неточности и плохой детализации требований, в результате чего разработчикам приходится заполнять возникающие пробелы собственными силами. Один из способов обнаружить двусмысленности – пригласить различных представителей классов пользователей для официальной экспертизы требований.

4. Добавление в ИС функций, ненужных заказчику. Применение диаграммы вариантов использования для извлечения требований поможет сосредоточиться на выборе тех функций и элементов графического интерфейса, которые помогут пользователям выполнять их задачи.

3.8. ЭТИЧЕСКИЕ ТРЕБОВАНИЯ К ПРОГРАММИСТАМ

Программисты, как и специалисты других профессий, должны быть честными и порядочными людьми. Но вместе с тем программисты не могут руководствоваться только моральными нормами или юридическими ограничениями.

Важнейшее качество программного инженера – его профессиональная компетентность. Программный специалист

должен обладать необходимыми компетенциями в своей области. Он не должен завышать свой истинный уровень компетентности и не должен сознательно браться за работу, которая этому уровню не соответствует.

Следует отметить также профессиональные обязательства перед заказчиком или потенциальным пользователем своего интеллектуального продукта.

1. Конфиденциальность – программные специалисты должны уважать конфиденциальность в отношении своих работодателей или заказчиков независимо от того, подписывалось ли ими соответствующее соглашение.

2. Защита интеллектуальной собственности – специалист должен соблюдать законодательство и принципы защиты интеллектуальной собственности при использовании чужой интеллектуальной собственности. Кроме того, он должен защищать интеллектуальную собственность работодателя и клиента. Важный момент: создаваемая им интеллектуальная собственность является собственностью работодателя или клиента.

3. Злоупотребление компьютером – программные специалисты не должны злоупотреблять компьютерными ресурсами работодателя или заказчика; под злоупотреблением мы понимаем широкий спектр – от компьютерных игр на рабочем месте до распространения вирусов и т.п.

В 1980-е гг. был введен термин «компьютерный профессионал», обозначающий человека, зарабатывающего на жизнь работой на компьютере. Здесь имелись в виду не только программисты, системные аналитики, системотехники, продавцы компьютерного оборудования, но и пользователи программных и аппаратных средств. Для регулирования взаимоотношений между компьютерными профессионалами и Ассоциацией вычислительной техники (Association for Computing Machinery – ACM) был разработан Кодекс этики IEEE-CS/ACM. Данный кодекс рекомендован и совместно одобрен ACM и IEEE-CS (Institute of Electrical and Electronic Engineers – Институт инженеров по электротехнике и электронике; British Computer

Society – Британское компьютерное общество) в качестве стандарта обучения и работы в области программной инженерии. Указанные сообщества совместно разработали и опубликовали IEEE-CS/ACM Software Engineering Code of Ethics and Professional Practices – Кодекс этики и профессиональной практики программной инженерии.

Члены этих организаций принимают обязательство следовать этому кодексу в момент вступления в организацию. Кодекс содержит восемь принципов, связанных с поведением и решениями, принимаемыми профессиональными программистами, включая практиков, преподавателей, менеджеров и руководителей высшего звена. Кодекс распространяется также на студентов и помощников программистов, изучающих данную профессию.

Кодекс имеет краткую и полную версии. Краткая версия кодекса определяет базовые принципы разработки ПО на высоком уровне абстракции. Полная версия показывает, как эти стремления отражаются на деятельности профессиональных программистов. Без базовых принципов детали кодекса станут казуистическими и нудными. Без деталей стремления останутся возвышенными, но пустыми. Вместе же они образуют целостный и гармоничный кодекс.

Программные инженеры должны добиваться повышения престижности профессии «инженер-программист». Они должны руководствоваться следующими восемью принципами.

1. Общество – программные инженеры должны действовать соответственно общественным интересам.

2. Клиент и работодатель – программные инженеры должны действовать в интересах клиентов и работодателя, соответственно общественным интересам.

3. Продукт – программные инженеры должны добиваться, чтобы произведенные ими продукты и их модификации соответствовали высочайшим профессиональным стандартам.

4. Суждение – программные инженеры должны быть честны и независимы в своих профессиональных суждениях.

5. Менеджмент – менеджеры и лидеры программных инженеров должны руководствоваться этическим подходом к руководству разработкой и сопровождением ПО, а также продвигать и развивать этот подход.

6. Профессия – программные инженеры должны периодически повышать свою квалификацию, улучшать репутацию своей профессии соответственно с интересами общества.

7. Коллеги – программные инженеры должны быть честными по отношению к своим коллегам и всячески их поддерживать.

8. Личная ответственность – программные инженеры должны непрерывно учиться навыкам своей профессии и способствовать продвижению этического подхода к своей деятельности.

Это краткая версия кодекса, которая довольно абстрактно резюмирует основные положения. В полной версии кодекса (см. прил. В) приведены детальные примеры того, как эти положения меняют образ действий программных инженеров как профессионалов. Без основных положений детали становятся формальными и скучными; без деталей основные положения звучат высокопарно; вместе же основные положения и детали образуют цельный кодекс.

В России кодекс был принят в 1996 г. и известен как Национальный кодекс деятельности в области информатики и телекоммуникаций. Он является средством самодисциплины, а также предназначен для использования судами в качестве справочного документа в рамках соответствующего законодательства. Кодекс распространяется на все виды деятельности юридических и физических лиц в области информатики и телекоммуникаций.

4. КОНФИГУРАЦИОННОЕ УПРАВЛЕНИЕ И ВЕРСИОННЫЙ КОНТРОЛЬ

Современная разработка программного обеспечения требует эффективного управления изменениями в коде, а также совместной работы множества разработчиков над одним проектом. Для этого используются средства версионного контроля ПО. В данной главе будет рассмотрено определение и основные принципы версионного контроля, средства и методы, их функциональность, интеграция с другими инструментами, преимущества и недостатки, примеры применения, сравнение различных систем контроля версий, а также ключевые факторы их выбора.

4.1. ПОНЯТИЕ КОНФИГУРАЦИОННОГО УПРАВЛЕНИЯ

При разработке ПО разработчики создают и используют существующие файлы проекта в большом количестве. Создать новый файл легко. Многие технологии программирования поддерживают стиль, когда, например, для каждого класса создается свой отдельный файл. Для крупных проектов это количество может составлять десятки тысяч. В связи с этим возникает задача обеспечить качественный учет и контроль файлов проекта.

При этом важно понимать, что файл – это виртуальная информационная единица. А главное отличие файла от материальных единиц учета в том, что у файла может быть **версия**, и не одна. Причем породить эти версии легко – достаточно скопировать данный файл в другое место на диске. В то время как материальные предметы существуют на складе сами по себе, и для них нет понятия версии. А версия файла – это очень непростой объект и имеют значение такие аспекты, важные с точки зрения управления программным процессом.

– Чем одна версия отличается от другой? Несколькими строчками текста или полностью обновленным содержанием?

– Какая из двух и более версий главнее, лучше?

– Какая из них соответствует текущей версии продукта?

К этому добавляется еще и то, что многие рабочие продукты могут состоять из набора файлов, и каждый из них может иметь по несколько версий. В итоге в программном проекте начинают происходить странные события:

– тщательно оттестированная программа на показательных испытаниях не работает;

– функциональность, документированная обновленными функциональными требованиями, добавленная в продукт и отосланная заказчику в виде новой версии продукта, таинственным образом исчезла из него;

– на компьютере разработчика программа работает, а у заказчика – нет.

Причина может оказаться в версиях файлов. Но проблема в том, что «версия всего продукта» – это абстрактное понятие. На деле есть версии отдельных файлов. Если один или несколько файлов в поставке продукта имеют не ту версию, программа не будет корректно работать.

Таким образом, в программных проектах необходима специальная деятельность по поддержанию файловых активов проекта в порядке. Эта деятельность называется *конфигурационным управлением*.

Конфигурационное управление (англ. software configuration management, SCM) – комплекс методов программной инженерии, направленных на систематический учёт изменений, вносимых разработчиками в программный продукт в процессе его разработки и сопровождения, сохранение целостности системы после изменений, предотвращение нежелательных и непредсказуемых эффектов, формализацию процесса внесения изменений.

Выделим две основные задачи в конфигурационном управлении.

1. Управление версиями – отвечает за контроль версий файлов и выполняется в проекте на основе специальных программных пакетов – средств версионного контроля.

Существует большое количество таких средств – Microsoft Visual SourceSafe, IBM ClearCase, svn, subversion и др. Более подробно данная задача рассмотрена в главе 4.

2. Управление сборками – это автоматизированный процесс трансформации исходных текстов ПО в пакет исполняемых модулей, учитывающий многочисленные настройки проекта, настройки компиляции и интегрируемый с процессом автоматического тестирования. Эта процедура является мощным средством интеграции проекта, основой итеративной разработки и рассматривается в параграфе 5.15.

4.2. ОБЪЕКТЫ КОНФИГУРАЦИОННОГО УПРАВЛЕНИЯ

Объектами КУ являются подвергающиеся изменениям в процессе разработки ПО продукты, состоящие из наборов файлов. Такие продукты принято называть *единицами конфигурационного управления* (ЕКУ) (configuration management items). К ним относятся:

- исходные тексты ПО;
- библиотеки классов;
- пакеты тестов;
- инсталляционные пакеты ПО;
- тестовые отчеты;
- проектная документация;
- пользовательская документация.

Каждая ЕКУ характеризуется следующими атрибутами.

1. Структура – упорядоченный набор файлов, связанных и взаимодействующих между собой. Например, пользовательская документация в html должна включать индекс-файл и набор html-файлов, а также набор вынесенных картинок (gif- или jpeg-файлы). Эта структура должна быть хорошо определена и отслеживаться при конфигурационном управлении – что все файлы не потеряны и присутствуют, имеют одинаковую версию, корректные ссылки друг на друга и т.д. Сами файлы представляют собой *элементы конфигурационного управления* (ЭКУ) (рис.4.1).

2. Ответственное лицо и, возможно, группа тех, кто их разрабатывает, а также более широкая и менее ответственная группа тех, кто пользуется этой информацией. Например, определенным программным компонентом могут в проекте пользоваться многие разработчики, но отвечать за ее разработку, исправление ошибок и пр. должен кто-то один.

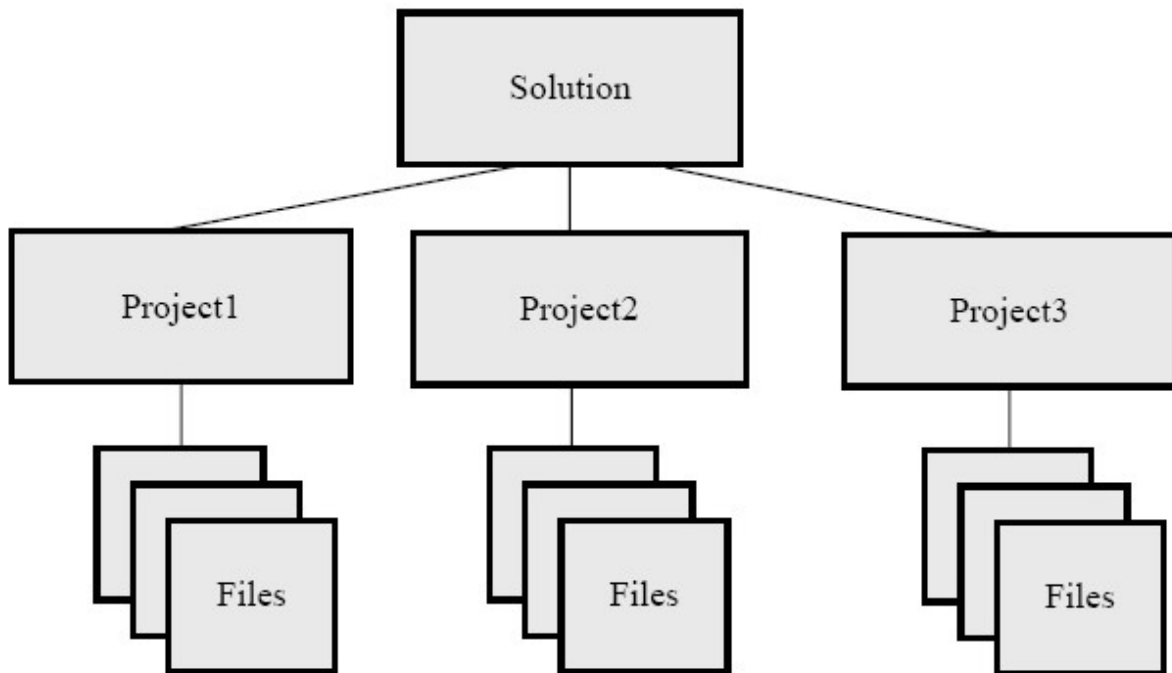


Рис. 4.1. Иерархическая структура конфигурации ПО

3. Практика конфигурационного управления:

- правила размещения новых версий ЭКУ в средство управления версиями (Кто? Как? Куда? Зачем?);
- правила именования и комментирования ЭКУ в этой версии;
- дальнейшие манипуляции с элементом;
- правила изменения тестов и тестовых пакетов при изменении кода (более высокоуровневые правила) и др.

4. Автоматическая процедура контроля целостности элемента – например, сборка для исходных текстов программ. Есть не у всех элементов, может не быть у документации, датасетов, тестовых пакетов.

Элементы конфигурационного управления (файлы) могут образовывать иерархию (рис. 4.1).

4.3. УПРАВЛЕНИЕ ВЕРСИЯМИ СОСТАВНЫХ КОНФИГУРАЦИОННЫХ ОБЪЕКТОВ

Поскольку программисты имеют дело с большим количеством файлов, а проекты в большинстве случаев являются командными, многие файлы в один момент могут быть необходимы нескольким специалистам. Важно, чтобы все эти файлы постоянно составляли единую компилируемую версию продукта. Также необходимо, чтобы была налажена оперативная работа с файлами исходного кода и с другими типами файлов. В этой ситуации файлы оказываются самыми младшими (по иерархии включения) элементами конфигурационного управления (рис. 4.1), могут иметь различные версии и уровни доступа.

Одновременно может существовать несколько версий системы как разный набор исходных текстов. При этом возможны два варианта:

- 1) одного проекта для одного заказчика;
- 2) одного проекта для разных заказчиков.

И в том, и в другом случае в средстве управления версиями образуются разные *ветви* проекта. Документы в разных ветвях имеют одинаковую историю до точки ветвления и разные – после неё.

Ветвь проекта – направление разработки ПО, независимое от других. Ветвь представляет собой копию части хранилища, в которую можно вносить изменения, не влияющие на другие ветви.

Рассмотрим подробнее вариант 1. Каждая ветвь содержит полный образ исходного кода и других артефактов, находящихся в системе контроля версий. Каждая ветвь может развиваться независимо, а может в определенных точках интегрироваться с

другими ветвями. В процессе интеграции изменения, произведенные в одной из ветвей, полуавтоматически переносятся в другую. В качестве примера можно рассмотреть следующую структуру разделения проекта на ветви:

- V1.0 – ветвь, соответствующая выпущенному релизу. Внесение изменений в такие ветви запрещены, и они хранят образ кода системы на момент выпуска релиза;

- Fix V1.0.1 – ветвь, соответствующая выпущенному пакету исправлений к определенной версии. Подобные ветви ответвляются от исходной версии, а не от основной ветви и замораживаются сразу после выхода пакета исправлений;

- Upcoming (V1.1) – ветвь, соответствующая релизу, готовящемуся к выпуску и находящемуся в стадии стабилизации (функциональное тестирование всей системы после окончания разработки). Для таких ветвей действуют более строгие правила, и работа в них ведется более формально;

- Mainline – ветвь, соответствующая основному направлению развития проекта. По мере созревания именно от этой ветви отходят ветви готовящихся релизов;

- WCF Experiment – ветвь, созданная для проверки некоторого технического решения, перехода на новую технологию или внесения большого пакета изменений, потенциально нарушающих работоспособность кода на длительное время. Такие ветви, как правило, делаются доступными только для определенного круга разработчиков и удаляются по завершению работ после интеграции с основной веткой.

4.4. ПОНЯТИЕ BASELINE

Baseline – это базовая, последняя целостная версия некоторого продукта разработки, например, документации, программного кода и т.д. Подразумевается, что разработка идет не сплошным потоком, а с фиксацией промежуточных результатов в виде текущей официальной версии. Принятие

такой версии сопровождается дополнительными действиями по оформлению, сглаживанию, тестированию, включению только законченных фрагментов. Этот результат можно посмотреть, отдать тестировщикам, передать заказчику и т.д. Baseline служит хорошим средством синхронизации групповой работы.

Baseline может быть совсем простой – веткой в системе управления версиями, где разработчики хранят текущую версию своих исходных кодов. Единственным требованием в этом случае является общая компилируемость проекта. Но поддержка baseline может быть сложной формальной процедурой, как показано на рис. 4.2.

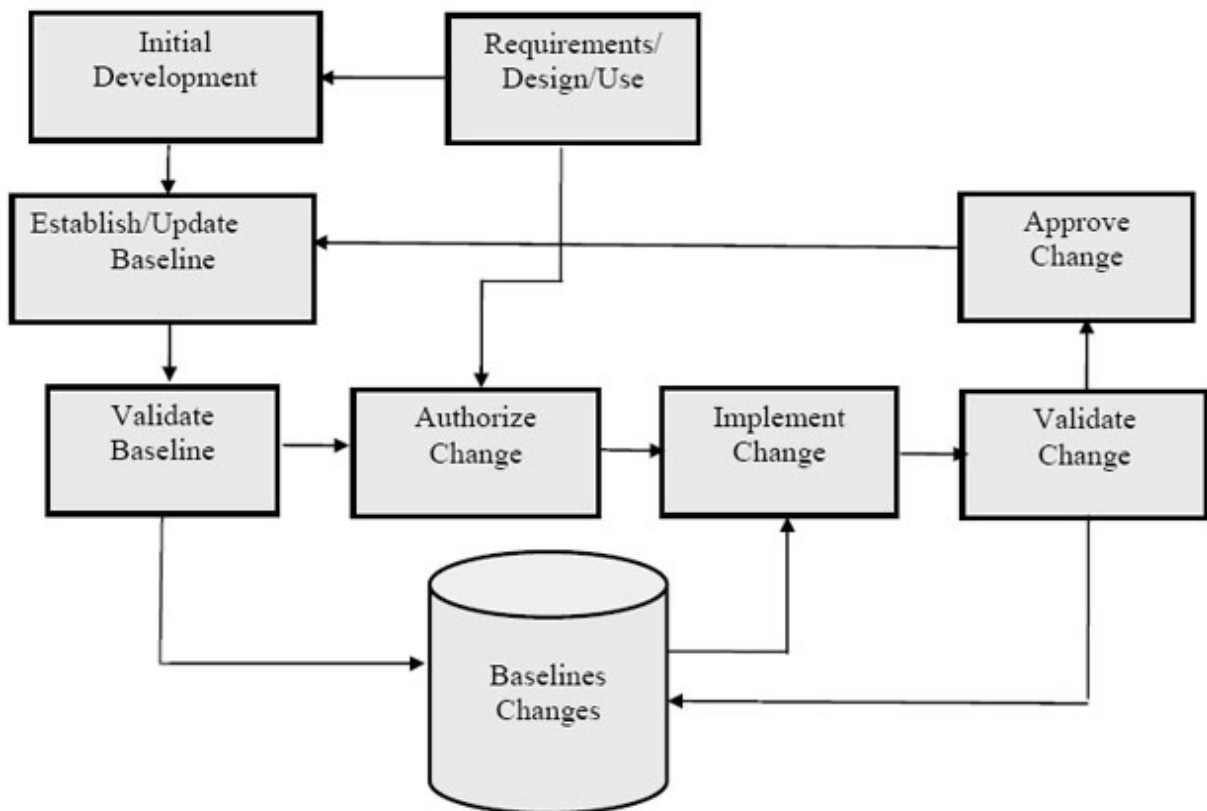


Рис. 4.2. Блок-схема построения baseline-версии продукта

Важно, что Baseline не должна устанавливаться слишком рано. Сначала нужно написать некоторое количество кода, реализовав какую-то функциональность для обеспечения корректной интеграции. Кроме того, вначале много внимания уделяется разработке основных архитектурных решений, и

целостная версия оказывается не востребованной. Однако, начиная с определенного членами команды момента, она просто необходима. Наконец, существуют проекты, где автоматическая сборка не нужна вовсе – это простые проекты, разрабатываемые небольшим количеством участников, где нет большого количества исходных текстов программ, проектов, сложных параметров компиляции.

4.5. ВОЗМОЖНОСТИ СРЕДСТВ ВЕРСИОННОГО КОНТРОЛЯ ПО

Для эффективного управления версиями используются системы контроля версий (англ. Version Control System, VCS). Основная функциональность VCS заключается в управлении изменениями в коде. В рамках этой функциональности средства версионного контроля предоставляют следующие возможности.

1. Создание веток – создание веток позволяет разработчикам работать параллельно над различными версиями кода. Каждая ветка имеет свою историю изменений и может быть объединена с другими ветками.

2. Слияние изменений – позволяет объединять изменения, сделанные в разных ветках, в одну общую версию.

3. Откат на предыдущие версии – возможность возврата к предыдущим версиям кода.

4. Управление изменениями в коде – средства версионного контроля ПО позволяют отслеживать историю изменений в коде. Это помогает разработчикам понимать, кто и когда вносил изменения в код, а также отслеживать причины ошибок и неправильной работы программы.

Системы контроля версий ПО могут быть интегрированы с другими инструментами разработки ПО, такими как системы сборки, тестирования и развертывания. В главе 5 рассматривается технология Azure DevOps Server, в которую интегрированы две VCS: Team Foundation Server и Git. Это позволяет гибко автоматизировать процесс разработки ПО.

Интеграционные средства системы контроля версий бывают трех типов.

1. Интеграция с системами сборки – средства VCS могут быть интегрированы с системами сборки, такими как Maven или Gradle. Это позволяет автоматически создавать сборки приложения на основе последней версии кода из репозитория.

2. Интеграция с системами тестирования – средства версионного контроля ПО могут быть интегрированы с системами автоматического тестирования, такими как JUnit или Selenium. Это позволяет автоматически запускать тесты приложения при каждом изменении кода.

3. Интеграция с системами развертывания – системы контроля версий ПО могут быть интегрированы с системами автоматического развертывания, такими как Ansible или Docker. Это позволяет автоматически разворачивать новые версии приложения на сервере при каждом изменении кода.

4.6. РАСШИРЕНИЯ И ИНСТРУМЕНТЫ ДЛЯ ВЕРСИОННОГО КОНТРОЛЯ ПО

Средства версионного контроля ПО могут быть расширены и дополнены различными инструментами и расширениями. Рассмотрим некоторые такие решения.

Git LFS (Large File Storage) – это расширение для Git, которое позволяет хранить большие файлы (например, изображения или видео) в отдельном хранилище вместо репозитория. Это позволяет уменьшить размер репозитория и ускорить процессы слияния и синхронизации.

GitFlow и GitHub Flow – это модели ветвления для Git, которые предлагают определенный подход к работе с ветками и слиянием изменений. GitFlow предлагает строгую модель ветвления, которая разделяет разработку на основную ветку (master) и различные ветки для разработки новых функций (feature), исправления ошибок (hotfix), выпуска новых версий (release) и т.д. GitHub Flow предлагает более простую модель ветвления, в которой существует только одна основная ветка (master), а каждое новое изменение создает временную ветку, которая затем сливается с основной веткой.

5. ТЕХНОЛОГИИ ОБЕСПЕЧЕНИЯ КАЧЕСТВА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Качество программного продукта определяется по нескольким критериям:

- 1) соответствовать стандартам (международным и корпоративным);
- 2) отвечать функциональным и нефункциональным требованиям, в соответствии с которыми он создавался;
- 3) иметь ценность для бизнеса и отвечать ожиданиям пользователей.

В жизненном цикле управления приложениями качество должно отслеживаться на всех его этапах. Оно начинает формироваться с определения необходимых требований. При этом важно указывать желаемую функциональность и способы проверки её достижения.

Кроме того, программный продукт должен удовлетворять определенному уровню потребностей пользователя. Качественный программный продукт должен обладать высоким потребительским качеством, независимо от области применения: внутреннее использование разработчиком, бизнес, наука и образование, медицина, коммерческие продажи, социальная сфера, развлечения, веб и др.

5.1. СТАНДАРТИЗАЦИЯ КАЧЕСТВА

Понятие о качестве ПО тесно связано со стандартами в области производства ПО, а также процессом стандартизации. Рассмотрим эти понятия более подробно.

Предпосылками возникновения задачи стандартизации товаров и услуг являются:

- глобализация и активное развитие мирового рынка;
- развитие глобальных сервисов, в частности, телекоммуникационных, банковских и др.;

– разнородные стандарты в разных странах для одних и тех же технологий, обусловившие технические барьеры в промышленности, торговле и бизнесе.

Как следствие, стали создаваться национальные и международные комитеты по стандартизации. Рассмотрим самые известные из них.

1865 г. – образован комитет, который ныне называется ITU (International Telecommunication Union). Сейчас штаб-квартира в Женеве (Швейцария), а ITU является частью ООН. Его основная задача – стандартизация телекоммуникационных протоколов и интерфейсов с целью поддержания и развития глобальной мировой телекоммуникационной сети. Самыми известными стандартами ITU считаются:

– ISDN (цифровая телефонная связь, объединяющая телефонные сервисы и передачу данных);

– ADSL (широко известная модемная технология, позволяющая использовать телефонную линию для выхода в Интернет, не блокируя при этом обычного телефонного сервиса);

– OSI (модель открытого 7-уровневого сетевого протокола, на которой базируются все современные стандартные сетевые интерфейсы и протоколы; также является стандартом ISO);

– языки визуального проектирования телекоммуникационных систем, SDL (Simple DirectMedia Layer – свободная кроссплатформенная мультимедийная библиотека) и UML (Unified Modeling Language – унифицированный язык моделирования).

Многие стандарты ITU переводятся на русский язык и превращаются в российские стандарты в виде ГОСТов.

1946 г. – создана организация ISO (International Organization for Standardization). Цель – содействие развитию стандартизации, а также смежных видов деятельности в мире для обеспечения международного обмена товарами и услугами, содействие и развитие сотрудничества в интеллектуальной, научно-технической и экономической областях. К настоящему времени создано около 17 000 стандартов в разных областях промышленности – продовольственные и иные товары,

различное оборудование, банковские сервисы и т.д. Некоторые стандарты ISO, актуальные для программной инженерии:

- серия стандартов ISO 9000 – стандартизация качества товаров и услуг; определение качества, определение системы поддержки качества на всех жизненных фазах изделия, товара, услуги (проектирование, разработка, коммерциализация, установка и обслуживание), описание процедур по улучшению деятельности компании, промышленного производства;

- ISO/IEC 90003:2004 – адаптация стандартов ISO 9000 к производству ПО в русле обеспечения качества в жизненном цикле ПО;

- ISO 9126:2001 – определение качественного ПО и различных атрибутов, описывающих это качество.

Многие стандарты ISO переводятся на русский язык и трансформируются в российские стандарты в виде ГОСТов. Имеется много стандартов в области информационных технологий (ГОСТ Р ИСО/МЭК 12207-99 «Информационная технология. Процессы жизненного цикла программных средств», ГОСТ Р ИСО/МЭК 15504-1-2009 «Информационные технологии. Оценка процессов»), а также в области программной инженерии (например, ГОСТ 19.701-90 (ИСО 5807-85) «Схемы алгоритмов, программ, данных и систем. Госстандарт СССР»). На соответствие стандартам ISO существует сертификация. В частности, компании сертифицируются на соответствие стандартам ISO 9000, т.е. на качественный процесс разработки ПО.

1988 г. – образование организации ETSI (European Telecommunications Standards Institute), штаб-квартира в г. София Антиполис (Франция). Независимая, некоммерческая, организация по стандартизации в телекоммуникационной промышленности (изготовители оборудования и операторы сети) в Европе. Самые известные стандарты – GSM, система профессиональной мобильной радиосвязи.

Рассмотрим ряд комитетов, непосредственно связанных с разработкой ПО.

1984 г. – создание SEI (Software Engineering Institute) на базе университета Карнеги Меллон в г. Питсбурге (США). Инициатор и главный спонсор – Министерство обороны США. Основная задача – стандартизация в области программной инженерии, выработка критериев для сертификации надежных и зрелых компаний (что в первую очередь интересует Минобороны США для выполнения его заказов). Самые известные продукты – стандарт CMM (Capability Maturity Model – модель зрелости возможностей создания ПО: эволюционная модель развития способности компании разрабатывать программное обеспечение), CMMI (Integration – обобщающий стандарт), разработки в области семейства программных продуктов (product lines). Эти продукты шагнули далеко за пределы военных разработок США, их использование и развитие стало международной деятельностью. Некоторые продукты SEI стандартизованы также ISO. На соответствие CMM/CMMI проводится сертификация.

1963 г. – создание IEEE (Institute of Electrical and Electronics Engineers). Ведет историю с конца XIX в. в контексте промышленной стандартизации в США. Сейчас IEEE – международная некоммерческая ассоциация специалистов в области техники, мировой лидер в области разработки стандартов по радиоэлектронике и электротехнике. Штаб-квартира в США, существуют многочисленные подразделения в разных странах, включая Россию. IEEE издаёт третью часть мировой технической литературы, касающейся применения радиоэлектроники, компьютеров, систем управления, электротехники, в том числе 102 реферируемых научных журнала и 36 отраслевых журналов для специалистов, проводит в год более 300 крупных конференций, принимала участие в разработке около 900 действующих стандартов.

1989 г. – группа американских IT-компаний (в том числе Hewlett Packard, Sun Microsystems, Canon) организовала OMG (Object Management Group). Сейчас включает около 800 тыс. компаний-членов. Основное направление – разработка и продвижение объектно-ориентированных технологий и стандартов, в том числе для создания платформонезависимых

программных приложений уровня предприятий. Известные стандарты – CORBA, UML, MDA.

Все эти комитеты и организации включают программную инженерию в сферу своей деятельности, сотрудничают, выпускают совместные стандарты, используют наработки друг друга и т.д.

В рассмотренных стандартах с точки зрения тестирования ПО имеет значение стандартизация качества (как контекст тестирования) – сначала самой продукции, а затем и процессов ее разработки (качественного результата не создать без качественного процесса).

Качество продукта или сервиса, предназначенного потребителю, определяется в стандарте ISO 9000:2005 как степень соответствия его характеристик требованиям – обязательным или подразумеваемым.

Исчерпывающий перечень стандартов программной инженерии представлен в прил. Г.

5.2. МЕТОДЫ ОБЕСПЕЧЕНИЯ КАЧЕСТВА ПО

При разработке ПО на практике используются пять способов контроля качества продукта.

1. Наладка качественного процесса. Для комплексного улучшения процессов компании разработчиками ПО используются стандарты CMM/CMMI, а также стандарты серии ISO 9000 (с последующей официальной сертификацией). Также применяются локальные стратегии, менее дорогостоящие и более направленные на решение отдельных проблем (подход organization pull).

2. Формальные методы (математизированные подходы к разработке ПО) – использование математических формализмов для доказательства корректности, спецификации, проверки формального соответствия, автоматической генерации и т.д.:

- доказательство правильности работы программ;
- проверка на моделях определенных свойств (model cheking);

- статический анализ кода по дереву разбора программы (например, проверка корректности кода по определенным критериям – аккуратная работа с памятью, поиск мертвого кода и пр.);

- модельно-ориентированное тестирование (model-based testing): автоматическая генерация тестов и тестового окружения по формальным спецификациям требований к системе и т.д.

На практике применяются ограниченно из-за необходимости серьезной математической подготовки пользователей, сложности в освоении, большой работы по развертыванию. Формальные методы эффективны в случае повышенных требований к надежности продукта, а также в руках высококвалифицированных специалистов.

3. Исследование и анализ динамических свойств ПО. ПО позволяет определить поведение программы во время работы, проанализировать использование системой различных ресурсов, таких как объем используемой системной памяти, ее быстродействие. Этот метод определяет качество организации параллельных процессов, взаимодействующих в программном продукте. Практикуются следующие методы:

- профилирование – исследование использования системой памяти, ее быстродействия и других характеристик путем запуска и непосредственных наблюдений в виде графиков, отчетов и пр.; используется при распараллеливании программ, при поиске «узких» мест;

- моделирование и анализ производительности (performance modeling and analysis) – моделируется нагрузочное окружение системы (число одновременных пользователей системы, сетевой трафик и пр.) и наблюдается поведение системы.

4. Обеспечение качества кода. Сюда относится целый комплекс различных мероприятий и методов. Рассмотрим самые известные из них:

- разработка стандартов оформления кода в проекте и контроль за соблюдением этих стандартов – включает правила на создание идентификаторов переменных, методов и имен классов,

на оформление комментариев, правила использования стандартных библиотек и т.д.;

- регулярный рефакторинг для предотвращения образования из кода «винегрета». При разработке кода имеет место тенденция ухудшения его структуры (избыточность, неиспользуемые или слабо используемые фрагменты, запутанная и труднопонижаемая структура) при внесении в него новой функциональности, исправления ошибок и пр. Рефакторинг – это регулярная деятельность по переписыванию кода с целью улучшения его структуры;

- инспекция кода, например, техника peer code review – заключается в том, что код каждого участника проекта читается и обсуждается на специальных встречах (code review meetings), делается это регулярно. Практика показывает, что в целом код улучшается;

- вычитка кода – используется при разработке критических систем реального времени. Ею занимаются также разработчики, но их роль в данном проекте – вычитка, а не разработка.

Обеспечение качества кода не только позволяет реализовать приемлемые продукты, но и эффективно модернизировать их и расширять функционал.

5. Тестирование. Самый распространенный способ контроля качества ПО, представленный фактически в каждом программном проекте.

5.3. КРИТЕРИИ КАЧЕСТВА ПРОГРАММНОГО ПРОДУКТА

Качество программного продукта определяется как совокупность его характеристик, позволяющих достичь поставленные для него цели без дополнительных усилий.

Инженерное определение качества – мера соответствия продукта заявленным требованиям к нему.

Качество ПО определяется несколькими критериями. Прежде всего, качественный программный продукт должен удовлетворять функциональным требованиям, т.е. обладать

способностью выполнять набор функций, соответствующих его спецификациям. Критериями качества также являются выполнение нефункциональных требований, основные из которых следующие (полный перечень нефункциональных требований представлен в прил. Б):

- надежность ПО определяет способность без сбоев выполнять заданные функции в заданных условиях и в течение заданного отрезка времени. Надежность вообще есть самое комплексное качество. В отношении ПО оно проверяется такими факторами, как отсутствие ошибок, устойчивость к ошибкам (отказоустойчивость), перезапускаемость, безопасность и др. Более детализированно можно выделить ряд факторов надежности программных продуктов;

- производительность характеризуется временем выполнения заданных транзакций или длительных операций;

- защищенность определяет степень безопасности системы от повреждений, утраты, несанкционированного доступа и преступной деятельности;

- удобство сопровождения определяет легкость, с которой обслуживается продукт в плане простоты исправления дефектов, внесения корректив для соответствия новым требованиям, управления измененной средой;

- эффективность как критерий качества ПО характеризуется двумя составляющими – ресурсной и временной экономичностью. Первый параметр определяет способность программного продукта выполнять функциональные требования при определенных ресурсных ограничениях, например, по используемой памяти. Временная экономичность определяется способностью продукта выполнять собственный функционал за определенный отрезок времени. Сопровождаемость программного продукта характеризуется удобством для анализа, изменяемостью, стабильностью, тестируемостью. Кроме того, к критериям качества ПО относятся: точность, определяемая величиной погрешности формируемых программным продуктом результатов с точки зрения области применения; защищенность, характеризующая способность ПО противостоять

преднамеренным или деструктивным действиям со стороны пользователя.

При разработке программного продукта критерии качества должны отслеживаться на всех этапах его жизненного цикла. Уровень качества определяется уже на этапе формирования требований к продукту. Следует указывать набор необходимых функций и способы проверки реализации желаемого функционала.

Управление жизненным циклом ПО, контроль качества продукта на всех его этапах разработки помогает целенаправленно проектировать качественный программный продукт, снижать ресурсные потери на повторное проектирование и переделку.

В ходе любой деятельности человека наблюдаются случайные ошибки, которые могут возникнуть в процессе создания или использования систем. Эти ошибки могут проявляться в виде дефектов, погрешностей или случайных искажений объектов или процессов. Если исходные данные программы фиксированы, то выполнение осуществляется по определенным алгоритмам, в соответствии с которыми формируются точно определенные результаты. Разнообразие вариантов использования программ с различными входными данными может быть воспринято наблюдателем как случайные события, поэтому возникновение ошибок рассматривается как случайное явление с различными причинами и последствиями. Для анализа критерия надежности программного продукта можно использовать модель взаимодействия основных компонентов (рис. 5.1).

Объекты уязвимости (первый блок на рис. 5.1), работоспособность которых влияет на надежность программного продукта: вычислительный процесс обработки данных, процесс подготовки данных, набор управляющих команд; информация, хранящаяся в базе данных; объектный код программы; информация, предоставляемая пользователю программы.



Рис. 5.1. Анализ надежности ПО

На эти объекты влияют различные внешние и внутренние факторы (второй блок на рис. 5.1), приводящие к дестабилизации программного продукта. К внутренним факторам относятся

системные ошибки, возникающие на уровне формулирования требований к программному продукту, описания характеристик алгоритма и параметров внешней среды, а также алгоритмические ошибки, возникающие при реализации ПО, выборе основных структур данных, условий взаимодействия модулей ПО, в том числе разработки структуры базы данных. К внутренним факторам также относятся ошибки программирования и описания данных, ошибки и неточности в документации отдельных компонентов и в целом программного продукта, недостаточная эффективность выбранных методов и средств обеспечения защиты кода и данных от сбоев.

Внешние факторы, определяющие надежность программного продукта, включают ошибки обслуживающего персонала при эксплуатации продукта, искажение внешней информации, поступающей от пользователя или других внешних источников, сбои в аппаратной части программной системы, а также изменение её конфигурации, выходящее за рамки допустимого в эксплуатационной документации.

5.4. ПОНЯТИЕ ТЕСТИРОВАНИЯ ПО

При разработке программного продукта неизбежно на разных этапах могут возникать ошибки. Своевременно обнаружить и исправить программные ошибки позволяет тестирование ПО.

Тестирование ПО – это процесс анализа ПО с целью определить, что функционал программы соответствует заявленным требованиям.

Это проверка сопоставления реальных и ожидаемых результатов поведения программы, проводимая на конечном наборе тестов, выбранном определённым образом.

Более строго тестирование ПО можно определить как проверку соответствия между реальным поведением программы и ее ожидаемым поведением в специально заданных, искусственных условиях. Разберем это определение по частям.

Ожидаемое поведение программы. Исходной информацией для тестирования являются требования к ней или к ее отдельной части, т.е. знание о том, как система должна себя вести.

Самый распространенный способ тестирования – метод **черного ящика**. При этом реализация системы недоступна тестирующим и тестируется только ее интерфейс. Часто это закрепляется и организацией коллектива – тестирующие являются отдельными сотрудниками и в некоторых компаниях они даже принципиально не общаются с разработчиками, чтобы минимально знать реализационные детали и максимально полно выступить в роли проверяющей инстанции.

Существует тестирование методом **белого ящика**, когда исходный код доступен тестирующим и используется в качестве источника информации о системе. Его схема представлена на рис. 5.2.

На основе требований к системе создается реализация и тестовая модель системы. Тестирование есть сопоставление двух этих представлений с целью выявить их несоответствия.

Составление этих представлений должно выполняться независимо друг от друга. Иначе, если тестирующие существенно используют информацию о реализации системы при составлении тестов, то они могут невольно внести в тесты ошибки реализации. Важно понимать, что при тестировании найденное несоответствие – это еще не ошибка, поскольку сами тестирующие могли неправильно понять требования, в тестах и средствах тестирования могли быть ошибки.

Данный подход закрепляется также и в организации коллективов компаний: тестирующие, как правило, отделены от разработчиков. Однако и те и другие должны исходить из общего видения системы – ее требований.

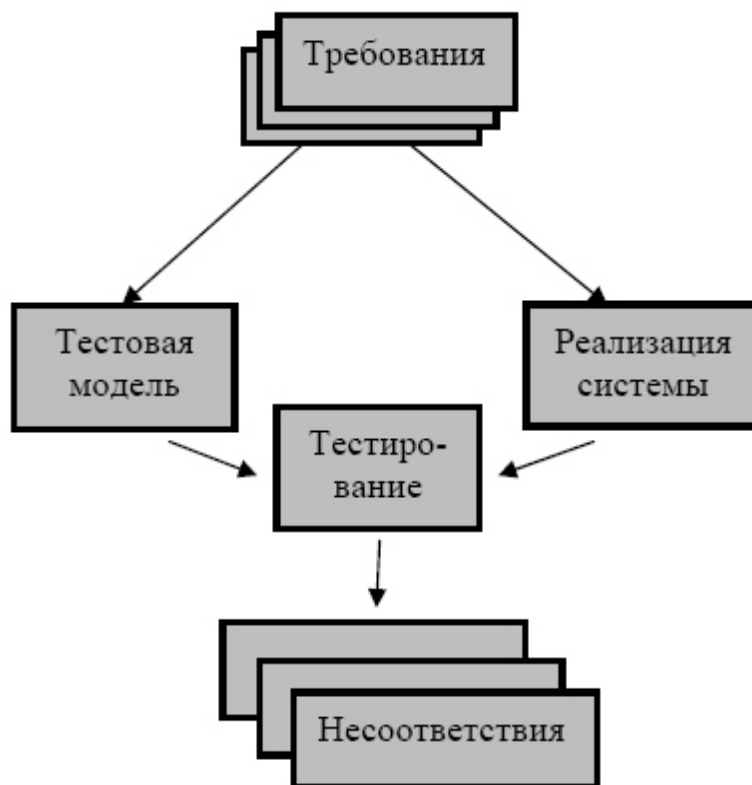


Рис. 5.2. Тестирование методом белого ящика

Специально заданные, искусственные условия – те условия, которые максимально соответствуют реальным условиям эксплуатации и в которых осуществляется тестирование.

При этом ключевым аспектом здесь является наличие **тестов** – воспроизводимых шагов манипуляции с системой, приводящих к ее некорректной работе. Концепция теста очень важна, так как необходимо не просто обнаружить некорректное поведение системы, а создать и зафиксировать алгоритм воспроизведения ошибки – чтобы повторить его для разработчика или чтобы разработчик сам смог воспроизвести ошибку. Если ошибка не воспроизводится, то нет возможности ее исправить.

Предпосылки к тестированию определяются на этапе постановки задачи жизненного цикла ПО. Техническое задание (ТЗ) определяет вид и функционал программы. При тестировании же необходимо не только выявить соответствие заявленному виду и набору функций, но и выяснить, правильно ли работает

программа. Поиск дефектов программного продукта позволяет проанализировать корректность работы программы и ее соответствие заданным критериям, а следовательно, определить, достигнут ли необходимый уровень качества объекта тестирования. Тестирование проводится на всех этапах разработки продукта и позволяет своевременно избежать дефектов на следующих стадиях. Тестирование не исправляет дефекты, но демонстрирует текущее состояние программного продукта, а значит, предоставляет заинтересованным людям информацию, на основе которой можно принимать решение о выпуске продукта или приостановке разработки. Эффективное тестирование снижает уровень риска возникновения ненадлежащего качества программного продукта.

5.5. ЦЕЛИ И ПРИНЦИПЫ ТЕСТИРОВАНИЯ ПО

Выделим основные цели тестирования:

- проверка соответствия требованиям;
- обнаружение проблем на более ранних этапах разработки и предотвращение повышения стоимости продукта;
- обнаружение вариантов использования, которые не были предусмотрены при разработке;
- повышение лояльности к компании и продукту, так как любой обнаруженный дефект негативно влияет на доверие пользователей.

Приведем основные принципы тестирования ПО.

Принцип 1. Тестирование демонстрирует наличие дефектов. Тестирование только снижает вероятность наличия дефектов, которые находятся в программном обеспечении, но не гарантирует их отсутствия.

Принцип 2. Исчерпывающее тестирование невозможно. Полное тестирование с использованием всех входных комбинаций данных, результатов и предусловий физически невыполнимо.

Принцип 3. Раннее тестирование. Следует начинать тестирование на ранних стадиях жизненного цикла разработки ПО, чтобы найти дефекты как можно раньше.

Принцип 4. Скопление дефектов. Большая часть дефектов находится в ограниченном количестве модулей.

Принцип 5. Парадокс пестицида. Если повторять одни и те же тестовые сценарии, то в какой-то момент этот набор тестов перестанет выявлять новые дефекты.

Принцип 6. Тестирование зависит от контекста. Тестирование проводится по-разному в зависимости от контекста. Например, программное обеспечение, в котором критически важна безопасность, тестируется иначе, чем новостной портал.

Принцип 7. Заблуждение об отсутствии ошибок. Отсутствие найденных дефектов при тестировании не всегда означает готовность продукта к релизу. Система должна быть удобна пользователю в использовании и удовлетворять его ожиданиям и потребностям.

5.6. КЛАССИФИКАЦИЯ ВИДОВ ТЕСТИРОВАНИЯ

В силу многоаспектности понятия качества ПО процесс его тестирования также многообразен и классифицируется по множеству признаков.

5.6.1. Классификация по концептуальному признаку

Концептуально тестирование разделяется на QA- и QC-тестирование (рис. 5.3).

QA-тестирование (Quality Assurance) предполагает процесс обеспечения качества продукта.

Это тестирование начинается на первом этапе одновременно с формированием технического задания. QA-тестирование выполняется на всех этапах разработки, начиная от создания требований к продукту до выхода продукта на рынок.

Специалисты QA создают и реализуют различные тактики для повышения качества на всех стадиях производства:

- проверяют ТЗ, чтобы убедиться, что в нем достаточно информации для работы и нет противоречий;
- разрабатывают метрики и критерии для оценки качества ПО;
- составляют план тестирования, определяют, какие функции приложения будут тестироваться и какие тестовые сценарии будут использоваться;
- контролируют функциональное и интеграционное тестирование, чтобы убедиться, что приложение выполняет свои функции и стабильно работает на любом оборудовании и ПО.



Рис. 5.3. Соотношение между QA- и QC-тестированием

QC-тестирование (Quality Control) – это контроль качества уже созданного продукта, процесс, отвечающий за анализ результатов тестирования, поиск ошибок и их устранение.

QC-тестирование выполняют на завершающей стадии разработки. К основным задачам QC-тестирования относят:

- проверку соответствия продукта ТЗ и заявленным требованиям;

- поиск и исправление ошибок, мешающих корректной работе приложения;
- проверку производительности (скорость работы, время загрузки и т.д.);
- проверку совместимости приложения с разными устройствами и операционными системами;
- тестирование пользовательского интерфейса с целью понимания, что приложение удобно для пользователей;
- тестирование безопасности, гарантирующее защиту приложения от взлома и утечки персональных данных.

5.6.2. По степени автоматизации

Тесты могут быть ручными и автоматическими.

Ручной тест – это последовательность действий тестировщика, которую он (или разработчик) может воспроизвести, и ошибка произойдет.

Как правило, в средствах контроля ошибок такие последовательности действий содержатся в описании ошибки.

Автоматический тест – это некоторая программа, которая воздействует на систему и проверяет то или иное ее свойство.

Автоматический тест часто требует применения так называемых инструментов тестирования – специальных программных продуктов, в задачу которых входит:

- запуск теста на системе;
- «прогон» целого пакета тестов;
- анализ полученных результатов и их обработка.

Важной задачей инструментов тестирования является обеспечение доступа теста к системе через ее интерфейс.

Преимущества автоматических тестов:

- можно легко воспроизводить без участия человека;
- создавать наборы тестов и прогонять их часто, например, в режиме регрессионного тестирования;

- генерировать по более высокоуровневым спецификациям, например, по формально описанным требованиям к системе.

Недостатки автоматического тестирования:

- необходимость использования инструментов тестирования, которые также являются неотъемлемой частью специально заданных, искусственных условий;

- затруднение доступа инструментов тестирования к тестируемой программе, например, в силу политических обстоятельств, когда сторонними разработчиками делается подсистема некоторой стратегической системы, и доступ к этой объемлющей системе у разработчиков сильно ограничен, или в силу аппаратных ограничений;

- для сложных программ возникают трудности с полнотой тестирования;

- высокая стоимость и трудоемкость как настройки и развертки готовых (сторонних), так и собственных тестовых инструментов;

- проблема системных ресурсов для автоматического тестирования, особенно при автоматической генерации тестов: часто есть возможность автоматически сгенерировать очень большое количество тестов, так что если их еще выполнять регулярно, в режиме непрерывной интеграции, то не хватит имеющихся системных ресурсов.

5.6.3. Классификация по виду запуска кода на исполнение

Статическое тестирование – процесс тестирования, который проводится для верификации практически любого артефакта разработки: программного кода компонент, требований, системных спецификаций, функциональных спецификаций, документов проектирования и архитектуры программных систем и их компонентов.

Динамическое тестирование – тестирование проводится на работающей системе, не может быть осуществлено без запуска программного кода приложения.

5.6.4. Классификация по уровню доступа к коду и архитектуре

Тестирование белого ящика – метод тестирования ПО, который предполагает полный доступ к коду проекта. Этот метод основан на анализе внутренней структуры компонентов программного продукта.

Тестирование серого ящика – метод тестирования ПО, который предполагает частичный доступ к коду проекта (комбинация White Box и Black Box методов).

Тестирование чёрного ящика – метод тестирования ПО, который не предполагает доступа (полного или частичного) к системе. Основывается на работе исключительно с внешним интерфейсом тестируемой системы. В этом случае не предполагается знание о внутренней структуре компонентов продукта.

5.6.5. Классификация по архитектурному уровню тестирования

Тестирование на разных уровнях производится на протяжении всего жизненного цикла разработки и сопровождения программного обеспечения. Уровень тестирования определяется объектом тестирования: конкретный модуль, набор модулей или продукт в целом. Различают следующие уровни тестирования:

- компонентное или модульное тестирование;
- интеграционное тестирование;
- системное тестирование;
- приемочное тестирование.

При модульном тестировании рассматривается отдельная единица программы.

Это может быть класс, функция или модуль, который тестируется в отрыве от остальной системы. Самый распространенный случай применения – тестирование модуля

самим разработчиком, проверка того, что отдельные модули, классы, методы выполняют действительно то, что от них ожидается. Различные среды разработки широко поддерживают средства модульного тестирования. Созданные разработчиком модульные тесты часто включаются в пакет регрессионных тестов и таким образом могут запускаться многократно.

Интеграционное тестирование определяет функционирование единиц программы, объединенных в единое целое, их совместную работу.

При этом два и более компонента тестируются на совместимость. Это очень важный вид тестирования, поскольку разные компоненты могут создаваться разными людьми, в разное время, на разных технологиях. Этот вид тестирования должен применяться самими программистами, чтобы, по крайней мере, удостовериться, что все компоненты программы совместимы в первом приближении. Далее тонкости интеграции могут исследовать тестировщики.

Такие «ошибки на стыках» непросто обнаруживать и устранять. Во время разработки компоненты все вместе не готовы, интеграция откладывается, а в конце обнаруживаются трудные ошибки (в том смысле, что их устранение требует существенной работы: реструктуризации, рефакторинга). Здесь выходом является ранняя интеграция системы и в дальнейшем использование практики постоянной интеграции.

При системном тестировании учитываются все уровни интеграции ПО.

Рассматривается функциональное тестирование, тестирование интерфейса, безопасности, локализации, надежности и доступности, производительности и эффективности, тестирование резервных копий и их восстановления, тестирование переносимости и др. Тестировщики, менеджеры и разработчики акцентируют внимание на том, как ПО выглядит и работает в целом, удобно ли оно, удовлетворяет ли ожиданиям заказчика. При этом могут открываться различные дефекты, такие как неудобство в

использовании тех или иных функций, забытые или плохо понятые требования.

Приемочное тестирование выполняется на заключительном этапе разработки ПО при приемке системы заказчиком и при удачном выполнении приводит к принятию программы заказчиком.

5.6.6. Классификация по принципу работы с программным продуктом

Выделяют позитивное и негативное тестирование. При позитивном тестировании используются только корректные данные для проверки. При некорректном тестировании анализируются некорректные операции.

5.6.7. Классификация по функциональному признаку

Дымовое тестирование – тестирование, выполняемое на вновь установленном программном продукте, подтверждающее, что программа может быть установлена и функционирует.

Тестирование критического пути проверяет корректность последовательности действий, выполняемых пользователем.

Расширенное тестирование предполагает исследование всего набора функциональных требований.

5.6.8. Классификация по степени зависимости от исполнителей

Альфа-тестирование представляет собой тестирование ранней версией программного продукта. Чаще всего выполняется внутри компании-разработчика.

Бета-тестирование предполагает тестирование программного продукта, выпущенного для ограниченного количества пользователей. Необходимо получить отзывы клиентов о продукте и внести соответствующие изменения.

5.6.9. Классификация по целям тестирования

Функциональное тестирование направлено на проверку корректности работы функциональности приложения.

Нефункциональное тестирование предполагает тестирование атрибутов компонента или системы, не относящихся к функциональности.

Нефункциональное тестирование включает ряд подтипов.

1. *Тестирование производительности* позволяет выяснить уровень стабильности и потребления ресурсов в условиях различных сценариев использования и нагрузок.

2. *Нагрузочное тестирование* определяет или собирает показатели производительности и времени отклика программного продукта в ответ на внешний запрос с целью установления соответствия требованиям, предъявляемым к продукту. Например, это может быть проверка баз данных на корректную обработку большого (предельного) объема записей, исследование поведения серверного ПО при большом количестве клиентских соединений, эксперименты с предельным трафиком для сетевых и телекоммуникационных систем, одновременное открытие большого числа файлов, проектов и т.д.

3. *Тестирование масштабируемости* предполагает измерение производительности программного продукта при изменении количества пользовательских запросов в сторону увеличения или уменьшения.

4. *Объёмное тестирование* проводится для тестирования программного продукта с определенным объемом данных.

5. *Стрессовое тестирование* направлено на проверку реакции программного продукта при лавинно нарастающей нагрузке, на устойчивость к непредвиденным ситуациям. Этот вид тестирования нужен далеко не для каждой системы, так как подразумевает высокую планку качества.

6. *Инсталляционное тестирование* проверяет корректность успешной установки и настройки, обновления или удаления продукта.

7. *Тестирование интерфейса* выполняет тестирование требований к пользовательскому интерфейсу.

8. *Тестирование удобства* использования направлено на определение степени удобства использования, понятности и привлекательности для пользователей разрабатываемого продукта в контексте заданных условий.

9. *Тестирование локализации* проверяет адаптацию программного продукта для определенной аудитории в соответствии с ее культурными особенностями.

10. *Тестирование безопасности* используется для проверки безопасности программного продукта, а также для анализа рисков, связанных с обеспечением целостного подхода к защите приложения, атак хакеров, вирусов, несанкционированного доступа к конфиденциальным данным.

11. *Тестирование надёжности* определяет работоспособность программного продукта при длительном тестировании с ожидаемым уровнем нагрузки.

12. *Регрессионное тестирование* предполагает тестирование уже проверенной ранее функциональности после внесения изменений в код приложения для уверенности в том, что эти изменения не внесли ошибки в областях, которые не подверглись изменениям, не ухудшили уже существующей функциональности. Для этого создаются пакеты регрессионных тестов, которые запускаются с определенной периодичностью – например, в пакетном режиме, связанные с процедурой постоянной интеграции.

13. *Повторное / подтверждающее тестирование* выполняет тестовые сценарии, выявившие ошибки во время последнего запуска, для подтверждения успешности исправления этих ошибок.

5.6.10. Классификация по уровню контроля хода тестирования

По этому признаку выделяют сценарное и исследовательское тестирование.

Сценарное тестирование – системный подход к тестированию ПО по предварительно написанным и задокументированным тест-кейсам (сценариям) – последовательностей действий пользователя в приложении.

Такой подход может помочь тестировщику решить некоторые проблемы с тестированием UI или логики приложения. Предполагает разделение задачи на этапы (подготовка, выполнение, завершение и т.д.) и последовательное выполнение этих этапов. К особенностям этого подхода относятся:

- четкое понимание того, какие функции покрыты тестами;
- уверенность, что все задокументированные тест-кейсы будут пройдены в срок;
- быстрое и легкое подключение нового специалиста на проект благодаря наличию подробных сценариев.

Исследовательское тестирование – параллельное проектирование и выполнение тестов, разработка и выполнения тестов в одно и то же время. Является противоположностью сценарного подхода (с его predetermined процедурами тестирования, неважно ручными или автоматизированными).

Исследовательские тесты, в отличие от сценарных тестов, не определены заранее и не выполняются в точном соответствии с планом. Даже в том случае, если тщательно определены все тестовые сценарии, то работа с большим количеством отдельных проверок (например, как быстро печатать на клавиатуре или какие виды поведения программы признать ошибочными) остаются на усмотрение тестировщика. Кроме того, даже в свободной форме поисковой сессии тест будет включать ограничения, состоящие в том, какую часть продукта тестировать или какую стратегию использовать. Хороший исследовательский тестировщик будет записывать идеи тестов и использовать их в последующих циклах испытаний. Такие заметки иногда очень похожи на сценарии тестирования, даже если они таковыми не являются.

Если каждый следующий тест выбирается по результатам предыдущего теста, это означает использование исследовательского тестирования. Основное отличие исследовательского тестирования от сценарного состоит в том, что тесты конструируются в зависимости от предыдущих шагов тестирования. Исследовательское тестирование выбирается в том случае, если предполагается, что непосредственно в процессе тестирования будет появляться дополнительная информация. Кроме того, предпочтение сценарному подходу отдается, когда существует необходимость тщательной проработки наборов тестов, требуются усилия по работе с этими наборами для обеспечения надежности и эффективности тестирования. Результаты исследовательского тестирования не обязательно радикально отличаются от тех, которые будут получены с помощью сценарного тестирования, и оба подхода к тестированию полностью совместимы. Такие компании, как Nortel и Microsoft обычно используют оба подхода в одном проекте. Тем не менее есть много важных различий между двумя подходами.

Проблемными направлениями в управлении эффективным исследовательским циклом тестирования являются стратегии тестирования и отчетность. Сценарный подход к тестированию – попытка механизировать процесс тестирования. Подобный способ тестирования полезен, но тестировщики, использующие исследовательский подход, придерживаются мнения, что запись тестовых сценариев и следование им «отупляет» тестировщика, мешая ему быстро находить ключевые проблемы. Чем более интеллектуальным становится тестирование, тем больше шансов протестировать приложение правильно и успеть сделать это вовремя. В этом и заключается преимущество исследовательского тестирования: богатство процесса ограничивается только шириной и глубиной нашей фантазии, а также пониманием природы тестируемого приложения.

Сценарное тестирование занимает свою нишу. Нередко в тестировании эффективность и воспроизводимость настолько важны, что необходимо разрабатывать и автоматизировать

сценарии. Исследовательское тестирование особенно полезно в сложных ситуациях тестирования, когда мало что известно о продукте, или как часть подготовки набора сценариев тестов. Основное правило заключается в следующем: исследовательское тестирование используется в тех случаях, когда выполнение следующего теста неочевидно, или когда следует выйти за рамки очевидного.

5.7. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ, ОБЛАСТЬ ПРИМЕНЕНИЯ МОДУЛЬНЫХ ТЕСТОВ

Unit-тесты, или модульные тесты позволяют программно протестировать нестандартные задачи. Они пошагово выполняют отладку приложения. Unit-тесты не являются эффективным инструментом при сжатых сроках проекта, небольшом бюджете, простых функциональных требованиях.

Для определения целесообразности использования unit-тестов можно воспользоваться следующей моделью в координатной системе X и Y . Здесь X – алгоритмическая сложность, а Y – количество зависимостей. Тестируемый код можно разделить на четыре класса (рис. 5.4).

1. Простой код (без каких-либо зависимостей).
2. Сложный код (содержащий много зависимостей).
3. Сложный код (без каких-либо зависимостей).
4. Не очень сложный код (но с зависимостями).

Первая зона – это ситуации, когда все просто и в тестировании нет необходимости.

Вторая зона – случай, когда код состоит из большого числа функций, перекрестно вызывающих друг друга. Здесь желательно провести рефакторинг. Тесты разрабатывать до этого процесса не имеет смысла.

Третья зона – случай сложных алгоритмов, бизнес-логики и т.п. Тестируется код, имеющий важное значение, поэтому его следует покрыть тестами.

Четвертая зона – код объединяет различные компоненты системы. Не менее важный случай, требующий тестирования.



Рис. 5.4. Целесообразность использования unit-тестов

Третья и четвертая зона особенно важны при тестировании ПО, влияющего на жизни людей, экономическую безопасность, государственную безопасность и т.п.

Тестирование делает код стабильным и предсказуемым. Поэтому код, покрытый тестами, гораздо проще масштабировать и поддерживать, так как появляется большая доля уверенности, что в случае добавления нового функционала нигде ничего не сломается. Более того, такой код легче поддается рефакторингу.

5.8. НАГРУЗОЧНОЕ И СТРЕСС-ТЕСТИРОВАНИЕ

К тестированию нефункциональных требований относят нагрузочное тестирование, которое позволяет проанализировать производительность программного продукта, время отклика на внешний запрос. К основным целям нагрузочного тестирования относят оценку производительности и работоспособности программного продукта на этапах разработки, передачи заказчику, выпуска новых версий, а также оптимизацию производительности, выраженную в настройках кода и сервера.

Например, для веб-приложения учебных курсов при нагрузочном тестировании можно рассмотреть следующие показатели:

- N преподавателей, N слушателей, N курсов;
- каждый из N слушателей посещает каждый из N курсов с вероятностью 0,5 (в среднем $N^2/2$);
- каждый из N слушателей по каждому из посещаемых курсов получает по 10 оценок (в среднем);
- каждый из N курсов добавляется в расписание на каждый день недели;
- каждому из N курсов добавляется 10 проведенных занятий (итого $10*N$).

Выполним нагрузочное тестирование для значений N = 10, 50, 100, 500, 1000.

Произведены замеры времени загрузки следующих страниц: список всех преподавателей; список всех слушателей; список всех курсов; страница конкретного преподавателя; страница конкретного слушателя; страница конкретного курса; личное расписание преподавателя; личное расписание слушателя; табель успеваемости слушателя; общее расписание; отчет.

По результатам нагрузочного тестирования заметно меняется лишь время загрузки страниц личных расписаний, табелей успеваемости и отчета. В табл. 5.1 и на рис. 5.5 и 5.6 приведены результаты по данным страницам.

Таблица 5.1

Результаты нагрузочного тестирования

Страница	N = 10	N = 50	N = 100	N = 500	N = 1000
listener_schedule	0,025	0,045	0,07	0,022	0,043
grade_table	0,006	0,04	0,023	0,106	0,2235
report	0,005	0,015	0,041	0,997	12,637

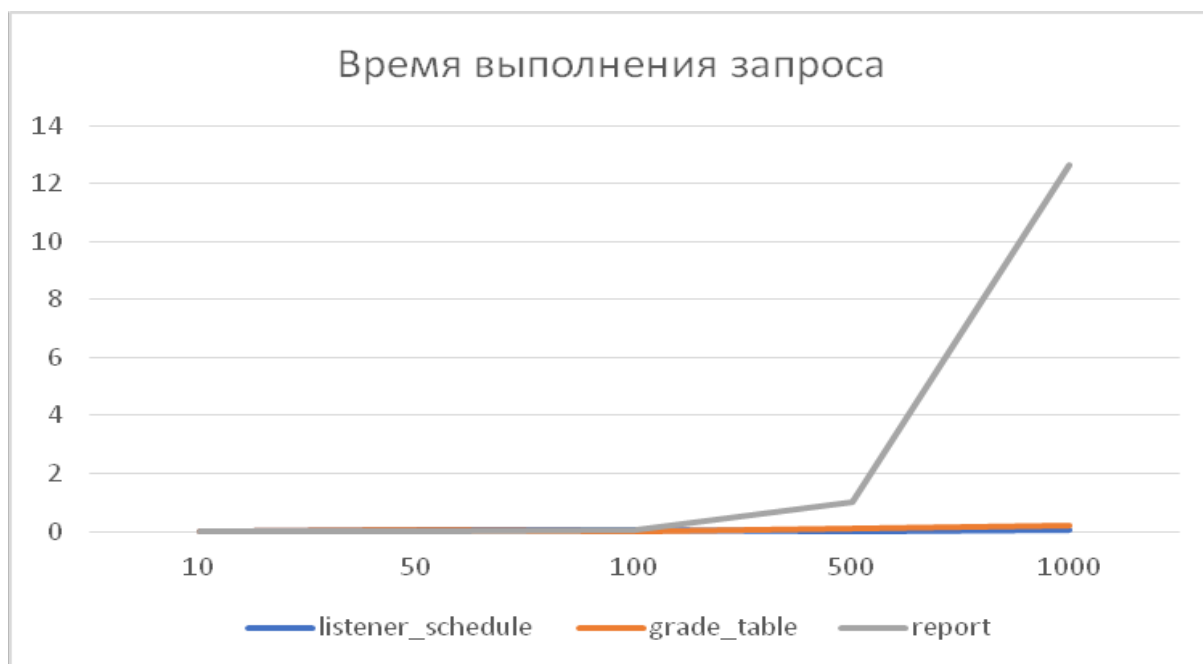


Рис. 5.5. Зависимость времени выполнения запроса от N

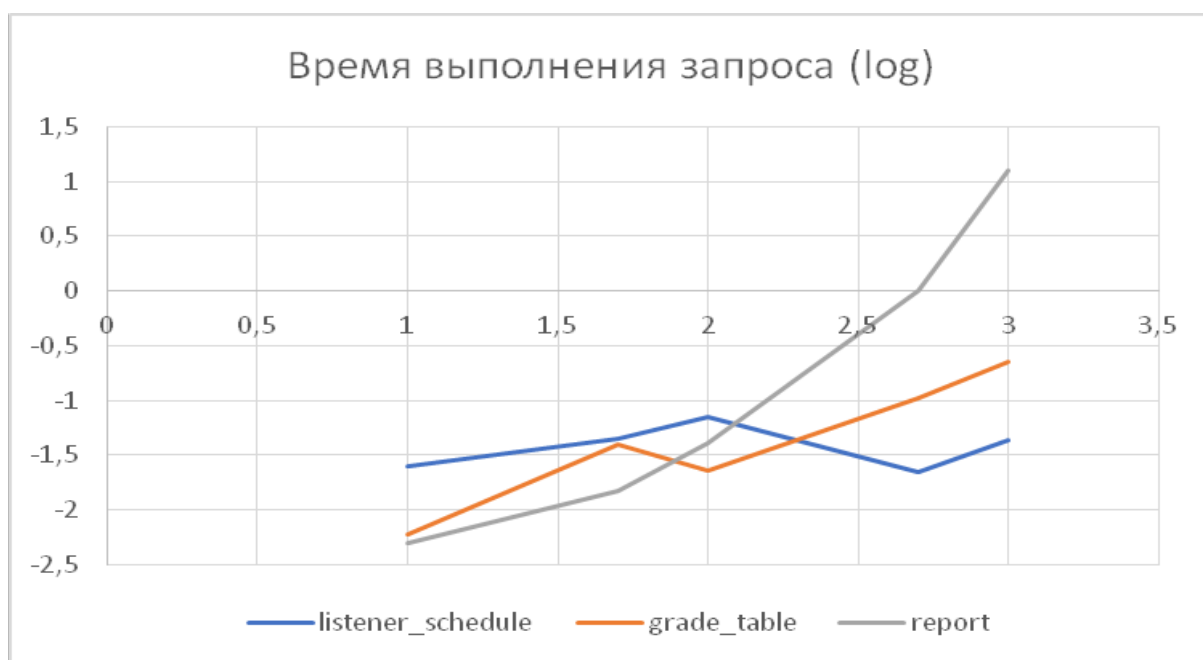


Рис. 5.6. Зависимость времени выполнения запроса от N (логарифмический график)

Стресс-тестирование также относится к нефункциональному тестированию, которое предназначено для анализа поведения программного продукта в условиях высокой нагрузки. Обычно тестируют количество пользователей или запросов,

обращающихся одновременно к серверу. Стресс-тестирование должно определить поведение продукта в экстремальных условиях; вычислить допустимое количество пользователей / запросов, одновременно обращающихся к серверу; выявить направления обеспечения безопасности системы и пользовательских данных.

Для веб-приложений стресс-тестирование обеспечивается следующими шагами: определить сценарии, которые максимально нагрузят приложение; выяснить поведение системы в различные дни, при низкой скорости подключения, на компьютерах с разной конфигурацией, на различных браузерах; оценить поведение различных компонент приложения в точках отказа.

Для мобильных приложений желательно проверить: поведение приложения при получении или отображении больших объемов данных; скроллинг; одновременное отправление на сервер большого числа однотипных действий (например, добавление / удаление в «Избранное», добавление / удаление товара в «Корзину»); нагрузку большим объемом данных при слабой мобильной сети.

Поясним разницу между нагрузочным и стрессовым тестированием в табл. 5.2.

Таблица 5.2

Характеристики нагрузочного и стресс-тестирования

Нагрузочное тестирование	Стресс-тестирование
Проверка поведения системы при стандартной нагрузке	Определение предела выносливости системы
Оценка поведения системы при ожидаемой нагрузке	Определение поведения системы при превышении предела нагрузки
Обработка ошибок практически не проверяется	Верификация обработки ошибок
Проверка безопасности и утечки данных не выполняется	Проверка угроз безопасности, утечки данных
Проверка надежности системы	Проверка устойчивости системы
Допустимо большое количество пользователей / запросов	Превышение максимального числа пользователей / запросов

5.9. ТЕХНИКИ ТЕСТ-ДИЗАЙНА

Один из этапов контроля качества программного продукта – тестирование, которое выполняется с помощью техник тест-дизайна. На этом этапе проектируются и создаются тест-кейсы, которые позволяют определить степень соответствия заранее определенным критериям качества и целям тестирования. Цель тест-дизайна заключается в конструировании наборов тестовых случаев, обеспечивающих оптимальное тестовое покрытие.

Техники тест-дизайна на практике дают следующие преимущества: позволяют исключить непродуктивные тест-кейсы и сократить общее количество кейсов, а следовательно, сократить время тестирования; покрыть тестами максимальное число функциональных блоков и не пропустить серьезных ситуаций в использовании программного продукта.

При тестировании исходного кода продукта, тестировании белого ящика важно рассматривать покрытие операторов, условий, путей, функций, значений параметров. При тестировании черного ящика, при работе с требованиями к программному продукту используют следующие техники.

Классы эквивалентности. Этот метод позволяет конструировать тесты не для каждого отдельного значения параметра, а объединять близкие значения в классы, предполагая, что для всех представителей класса результат тестирования должен быть одинаков.

Граничные значения. Предполагается, что на границах классов эквивалентности может возникать большое число ошибок, поэтому следует тестировать значения на границе классов.

Пример 1. Тестирование сайта расчета стоимости билета по дате рождения человека.

Правила.

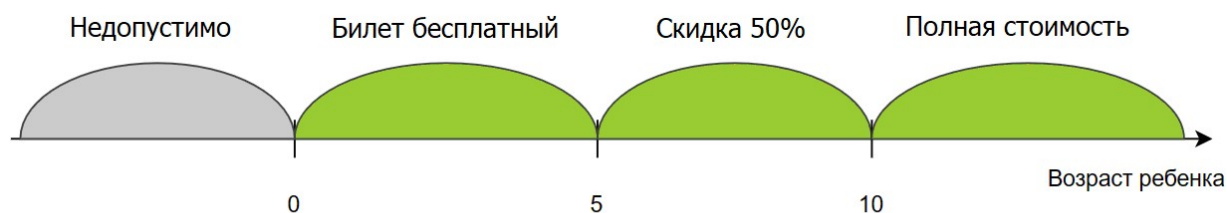
1. Возраст человека считается на момент поездки.
2. Если ребенку еще не исполнилось 5 лет, он едет бесплатно.

3. Если ребенку от 5 до 10 лет, он получает скидку на билет в размере 50 %.

4. Если человеку уже исполнилось 10 лет, то никакой скидки нет.

На рис 5.7 приведены классы эквивалентности и граничные значения.

Тогда тест-кейсы для проверки граничных значений могут быть следующими (табл. 5.3).



Пусть дата поездки 20.03.2024

Граничные значения, которые нужно проверить:

Граница у 0 {18.03.2024, 20.03.2019, 21.03.2019}

Граница у 5 {19.03.2019, 20.03.2019, 21.03.2019}

Граница у 10 {19.03.2024, 20.03.2024, 21.03.2024}

PS: в форме вводится не возраст человека, а дата его рождения.

Рис. 5.7. Классы эквивалентности и граничные значения

Попарное тестирование. Техника позволяет минимизировать число тестов. Вначале рассматриваются все значения всех параметров и в процессе количество вариантов сокращается.

Пример 2. В объектно-ориентированной системе объект класса "А" может передать сообщение, содержащее параметр "Р", объекту класса "Х". Классы "В", "С" и "D" унаследованы от класса "А", поэтому тоже могут послать сообщение. Классы "Q", "R", "S" и "Т" унаследованы от "Р", поэтому могут быть переданы как параметры. Классы "Y" и "Z" унаследованы от класса "Х" и могут получать данные.

Графически схема представлена на рис. 5.8.

Для каждого условия определен набор значений.

Отправитель: A, B, C, D.

Параметр: P, S, R, Q, T.

Получатель: X, Y, Z.

Тогда в результате применения техники попарного тестирования получим табл. 5.4.

Таблица 5.3

Тест-кейсы для проверки возраста при расчете скидки (считаем, что поездка 20.03.2024)

Описание тест-кейса	Входные данные	Ожидаемый результат
Ввод корректной даты рождения (от 0 до 5 лет)	"data" = "27.08.2021"	Ответ: ребенок едет бесплатно
Ввод корректной даты рождения (от 5 до 10 лет)	"data" = "27.08.2018"	Ответ: скидка в 50 % на билет
Ввод корректной даты рождения (более 10 лет)	"data" = "27.08.2004"	Ответ: билет по полной цене
Ввод некорректной даты рождения (родится после поездки)	"data" = "27.08.2099"	Ответ: ребенок родится после поездки?
Проверка граничных значений		
Граница возле 0	"data" = "19.03.2024", "data" = "20.03.2024"	Ответ: ребенок едет бесплатно
	data = "21.03.2024"	Ответ: ребенок родится после поездки?
Граница возле 5 лет	data = "19.03.2019"	Ответ: ребенок едет бесплатно
	data = "20.03.2019"	Ответ: скидка 50 % на билет
	data = "21.03.2019"	
Граница возле 10 лет	data = "19.03.2014"	Ответ: скидка 50 % на билет
	data = "20.03.2014"	Ответ: полная стоимость
	data = "21.03.2014"	

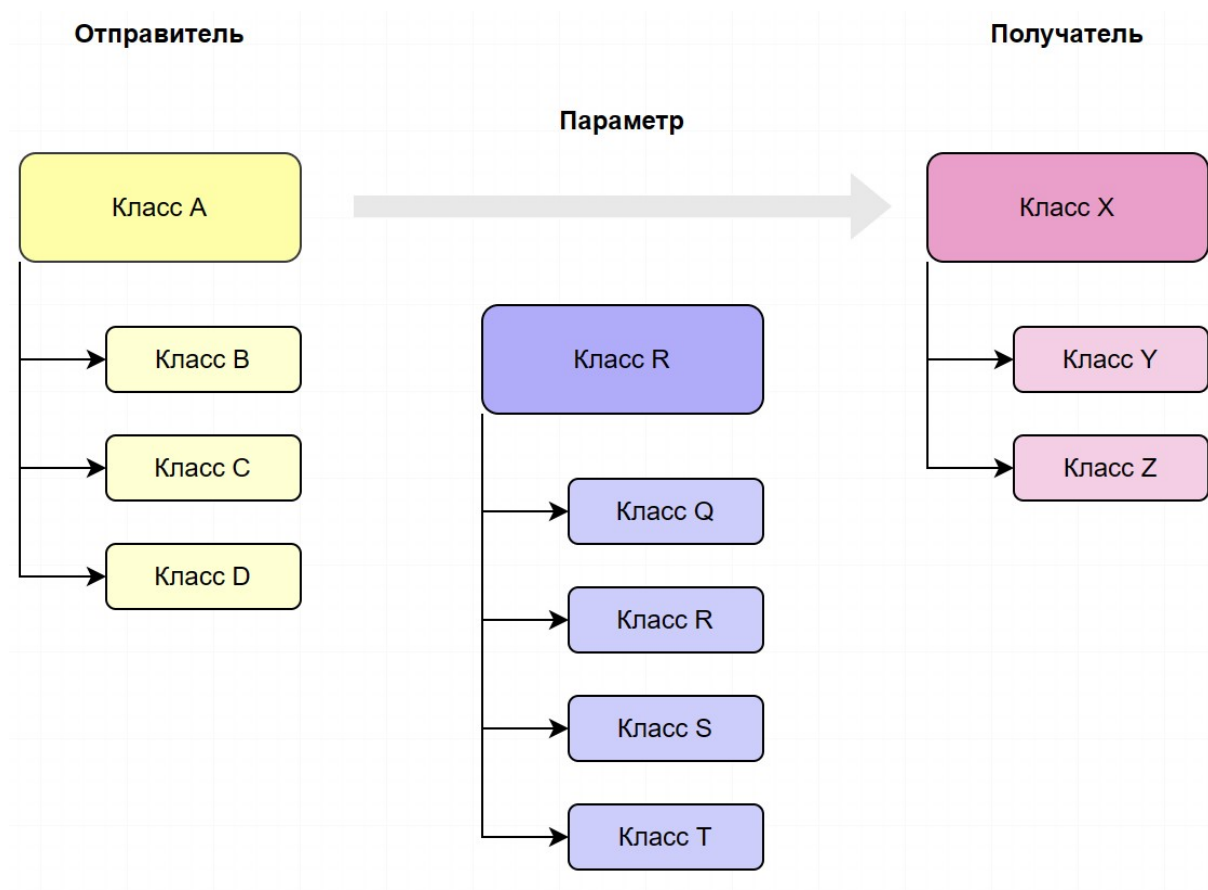


Рис. 5.8. Графическое представление

Таблица 5.4

Попарные тесты

№	Отправитель	Параметр	Получатель
<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>
1	A	R	Z
2	A	S	X
3	B	Q	Z
4	B	R	X
5	B	S	Y
6	B	T	X
7	B	P	Y
8	C	R	Y
9	C	S	X
10	C	T	Y
11	C	P	Z
12	C	Q	X

Окончание табл. 5.4

1	2	3	4
13	D	S	Y
14	D	T	Z
15	D	P	X
16	D	Q	Y
17	D	R	X
18	A	T	X
19	A	P	Y
20	A	Q	X
21	A	R	Y
22	A	S	Z

Таблица принятия решений. Техника применяется для систем со сложной логикой, например, для тестирования двухфакторной аутентификации. Выделяется список возможных условий, комбинации из выполнения или не выполнения этих условий, список возможных действий.

Пример 3. Для добавления нового студента в базу данных необходимо ввести имя, адрес и телефон, а затем нажать Enter. Студент будет добавлен в базу данных, система вернет новый student_id (табл. 5.5).

Диаграмма состояний и переходов. Визуализирует состояния программы в разные периоды времени и на разных этапах использования. Применяется для фиксирования требований и описания дизайна приложения.

Доменное тестирование. Техника основана на разбиении диапазона возможных значений переменной на поддиапазоны, с последующим выбором одного или нескольких значений из каждого домена для тестирования.

Сценарий использования. Описывает сценарий взаимодействия двух и более участников (как правило – пользователя и системы). Тесты содержат перечень шагов пользователя, которые он выполняет для совершения

определенного действия, а также информацию о том, как сайт или приложение реагирует на действия пользователя.

Таблица 5.5

Добавление информации о студенте

Условия	Правила							
	1	2	3	4	5	6	7	8
Имя	Пусто	Пусто	Пусто	Пусто	Есть	Есть	Есть	Есть
Адрес	Пусто	Пусто	Есть	Есть	Пусто	Пусто	Есть	Есть
Телефон	Пусто	Есть	Пусто	Есть	Пусто	Есть	Пусто	Есть
Действия								
Студент успешно добавлен								+
Сообщение: нужно заполнить все поля	+	+	+	+	+	+	+	

5.10. АВТОМАТИЗИРОВАННОЕ ТЕСТИРОВАНИЕ

Для многочисленной прогонки тестов удобнее их автоматизировать.

Автоматический тест – это программный модуль, который проверяет программный продукт на некоторое свойство.

Такие тесты можно генерировать ёпо высокоуровневым спецификациям. Например, при тестировании компилятора можно опираться на формальное описание конструкций языка.

Для создания автоматических тестов необходимы специальные средства – инструменты тестирования. Они позволяют запустить тест на программном продукте, выполнить несколько тестов, осуществить анализ результатов тестирования. Однако автоматизация тестов требует достаточно много ресурсов системы, генерация большого количества тестов, их регулярное выполнение, интеграция результатов достаточно трудоемки.

Поэтому количество автоматических тестов часто определяется бюджетом проекта. Необходимо установить приоритеты тестируемых требований как для работоспособности всего продукта, так и для заказчика.

В существующих системах программирования имеются встроенные инструменты автоматического тестирования. Так для разработчиков в MS Visual Studio присутствует возможность конструировать модульные и нагрузочные тесты, тесты пользовательского интерфейса. В системе представлены соответствующие шаблоны.

Приведем пример подобного модульного теста для страницы отчета веб-приложения:

```
def unit_test():
    try:
        if os.path.exists(database_path):
            os.remove(database_path)
        create_db()
        with app.app_context():
            get_db().executemany(r"INSERT INTO
Listeners (listener_id, first_name, last_name,
middle_name) VALUES (?, ?, ?, ?)",
                                [(1, "A", "A", "A"), (2, "B", "B",
"B"), (3, "C", "C", "C"), (4, "D", "D", "D")])
            get_db().executemany(r"INSERT INTO
Teachers (teacher_id, first_name, last_name,
middle_name) VALUES (?, ?, ?, ?)",
                                [(1, "T", "T", "T"), (2, "T2", "T2",
"T2")])
            get_db().executemany(r"INSERT INTO
Courses (course_id, course_name, teacher_id) VALUES
(?, ?, ?)",
                                [(1, "C1", 1), (2, "C2", 1), (3,
"C3", 2), (4, "C4", 2), (5, "C5", 2)])
            get_db().executemany(r"INSERT INTO
AttendedCourses (course_id, listener_id) VALUES (?,
?)",
                                [(1, 1), (2, 1), (2, 2), (3, 2), (3,
3), (3, 4), (4, 1), (4, 2), (4, 4)])
```

```

        get_db().executemany(r"INSERT INTO
ClassRecord (course_id, class_datetime) VALUES (?,
date('now', ?))",
        [(1, '-2 days'), (1, '-3 days'), (2,
'-5 days'), (3, '-50 days'), (4, '-10 days')])
        client = app.test_client()
        response = client.get('/report')
        text = response.data.decode('utf8')
        assert '<p>Количество проведенных
занятий: 4</p>' in text, "Incorrect count 1 in
report"
        assert '<p>Количество уникальных
слушателей: 3</p>' in text, "Incorrect count 2 in
report"
        assert '<p>Среднее количество слушателей
на курс: 2.0</p>' in text, "Incorrect count 3 in
report"
        print ('Unit test (report): successful!')
    finally:
        os.remove(database_path)

```

Результаты модульного тестирования:

```
Unit test (report): successful!
```

В среде MS Visual Studio существует инструмент Microsoft Test Manager (MTM), позволяющий управлять всем жизненным циклом процесса тестирования программного продукта. С помощью этого инструмента возможна подготовка плана тестирования, добавления в него наборов тестов различных конфигураций. Результаты тестирования сохраняются в базе данных, что предполагает их дальнейший анализ и документацию. При внесении модификаций в программный продукт необходимо выполнять регрессионное тестирование. В этом случае MTM создает план регрессионного тестирования, определяя, какие тесты выполняются повторно.

Автоматизация тестирования ПО предоставляет ряд преимуществ в сравнении с ручным тестированием.

Повышение эффективности и скорости тестирования: компьютерные средства автоматизации тестирования (КСАТ) позволяют автоматизировать выполнение повторяющихся и монотонных тестовых сценариев, что существенно ускоряет процесс тестирования и позволяет освободить ресурсы тестировщиков для выполнения более творческих задач.

Увеличение покрытия тестирования: КСАТ способны автоматизировать выполнение большого количества тестовых сценариев, что позволяет достичь более высокого уровня покрытия тестирования. Автоматизированные тесты могут выполняться на различных платформах, конфигурациях и наборах данных, что помогает выявить больше потенциальных проблем.

Снижение затрат: внедрение КСАТ требует начальных инвестиций, в долгосрочной перспективе они способны снизить затраты на тестирование. Автоматизация позволяет сократить количество ошибок, улучшить качество ПО и снизить затраты на повторное тестирование.

5.11. КОМПЬЮТЕРНЫЕ СРЕДСТВА АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ ПО

В современном информационном обществе разработка программного обеспечения является сложной и многопроцессной. При создании ПО необходимо удостовериться в его корректности, функциональности и надежности перед выпуском на рынок. Один из наиболее распространенных подходов для проверки ПО – это использование компьютерных средств автоматизации тестирования. Рассмотрим основные аспекты компьютерных средств автоматизации тестирования ПО.

Компьютерные средства автоматизации тестирования ПО представляют собой инструменты и программные средства, которые позволяют автоматизировать процесс тестирования программного обеспечения. КСАТ обладают возможностью создания, выполнения и анализа тестовых сценариев и тестовых данных, а также генерации отчетов о результатах тестирования.

5.11.1. Основные функциональные возможности КСАТ

КСАТ обладают различными функциональными возможностями, которые обеспечивают эффективное и качественное тестирование ПО.

Создание тестовых сценариев: КСАТ позволяют создавать тестовые сценарии, которые описывают последовательность шагов для проверки функциональности ПО. Это может быть сделано с помощью специальных языков программирования, графических интерфейсов или рекордеров действий.

Генерация тестовых данных: КСАТ предоставляют возможность автоматической генерации тестовых данных для проверки различных вариантов использования ПО. Это позволяет проверить ПО на разнообразные входные данные и обнаружить потенциальные ошибки.

Выполнение тестовых сценариев: КСАТ автоматизируют выполнение тестовых сценариев на целевых системах. Они могут взаимодействовать с интерфейсами ПО, эмулировать действия пользователя и проверять соответствие ожидаемых результатов фактическим.

Анализ результатов тестирования: КСАТ предоставляют возможность анализировать результаты тестирования, выявлять ошибки и сравнивать фактические результаты с ожидаемыми. Это помогает выявить проблемные области в ПО и предоставляет информацию для их исправления.

5.11.2. Примеры КСАТ

Существует множество различных КСАТ, которые предоставляют различный набор функций и возможностей. Рассмотрим наиболее популярные КСАТ.

Selenium – один из самых широко используемых КСАТ для веб-приложений. Он позволяет записывать и воспроизводить действия пользователя на веб-страницах, а также выполнять проверки и анализировать результаты.

Appium – представляет собой КСАТ для автоматизации тестирования мобильных приложений. Он поддерживает различные платформы, включая iOS и Android, и позволяет тестировать функциональность и интерфейс мобильных приложений.

JUnit – это фреймворк для тестирования Java-приложений. Он предоставляет набор инструментов для создания и выполнения тестовых сценариев, а также проверки результатов.

Cucumber – инструмент для автоматизированного функционального тестирования, основанный на языке Gherkin. Он позволяет создавать тестовые сценарии в формате естественного языка и автоматически выполнять их.

LoadRunner предоставляет возможности для нагрузочного тестирования и анализа производительности ПО. Он позволяет симулировать высокие нагрузки на систему и оценить ее производительность и стабильность.

5.11.3. Методы выбора КСАТ

При выборе компьютерных средств автоматизации тестирования ПО необходимо учитывать ряд факторов и методов.

1. Анализ требований: начальный этап выбора КСАТ – это анализ требований к ПО и тестированию. Важно определить, какие функциональности и возможности должны быть поддержаны КСАТ для эффективного тестирования ПО.

2. Бюджет: одним из факторов выбора КСАТ является бюджет, выделенный на автоматизацию тестирования. Некоторые инструменты могут быть платными, в то время как другие предлагают открытый и бесплатный доступ.

3. Тип ПО и платформы: необходимо учитывать тип разрабатываемого ПО и целевые платформы. Некоторые КСАТ специализируются на веб-приложениях, в то время как другие предназначены для мобильных приложений или настольного ПО.

4. Интеграция: важно учитывать возможность интеграции выбранного КСАТ с другими инструментами и средствами

разработки, такими как системы управления версиями или системы управления проектами.

5. Обучение и поддержка: необходимо учитывать наличие документации, обучающих материалов и поддержки со стороны разработчиков выбранного КСАТ. Это поможет обеспечить более эффективное использование инструмента.

5.11.4. Рекомендации по внедрению КСАТ

Внедрение компьютерных средств автоматизации тестирования ПО требует определенного подхода. Рассмотрим рекомендации для успешного внедрения КСАТ.

1. Планирование: разработать детальный план внедрения КСАТ, включающий этапы, ресурсы, сроки и ожидаемые результаты; учесть потребности команды тестирования и согласовать план с другими участниками процесса разработки ПО.

2. Обучение и поддержка: предоставить обучение тестирующим и другим заинтересованным лицам по использованию выбранного КСАТ; обеспечить надлежащую поддержку со стороны разработчиков инструмента.

3. Постепенное внедрение: рекомендуется внедрять КСАТ постепенно, начиная с выбранных тестовых сценариев или модулей ПО. Это поможет избежать слишком большой нагрузки на команду тестирования и обеспечит более плавный переход к автоматизации.

4. Регулярный анализ и оптимизация: внедрение КСАТ – это непрерывный процесс. Необходимо регулярно анализировать эффективность и результаты автоматизации, проводить оптимизацию с учетом обратной связи от команды тестирования и разработчиков.

5. Коллаборация и коммуникация: обеспечить эффективную коммуникацию и сотрудничество между командой тестирования, разработчиками и другими заинтересованными сторонами. Регулярно обмениваться информацией и опытом для улучшения процесса тестирования.

5.11.5. Преимущества и ограничения использования КСАТ

Компьютерные средства автоматизации тестирования ПО имеют свои преимущества и ограничения.

Преимущества использования КСАТ.

1. Увеличение эффективности и скорости тестирования: автоматизация тестирования позволяет выполнять тесты быстрее и более эффективно. Автоматическое выполнение повторяющихся тестов и быстрая проверка результатов сокращают время, необходимое для выполнения тестовых сценариев.

2. Улучшение точности и надежности: КСАТ позволяют устранить человеческие ошибки, связанные с ручным выполнением тестов. Автоматизированные тесты повторяются одинаковым образом при каждом запуске, обеспечивая надежный результат.

3. Повышение покрытия тестирования: с помощью КСАТ можно проводить тестирование на разных платформах, операционных системах и браузерах, расширяя покрытие тестирования. Это помогает выявить проблемы, которые могут возникнуть на различных конфигурациях.

4. Возможность раннего обнаружения ошибок: автоматизированные тесты могут быть запущены на ранних стадиях разработки, что позволяет обнаруживать и устранять ошибки на ранних этапах. Это помогает сократить время и затраты на исправление проблем в будущем.

5. Улучшение масштабируемости и поддерживаемости: КСАТ обеспечивают возможность повторного использования тестовых сценариев и компонентов, что упрощает поддержку и масштабирование тестового набора при изменениях в ПО.

Ограничения использования КСАТ.

1. Ограничения для некоторых типов тестирования: некоторые виды тестирования, такие как тестирование пользовательского интерфейса и тестирование пользовательского опыта, сложнее автоматизировать. В таких случаях ручное тестирование может оставаться необходимым.

2. Необходимость поддержки и обновления: КСАТ требуют постоянной поддержки и обновления со стороны разработчиков. Новые версии ПО и изменения в тестируемом приложении могут потребовать обновления тестовых скриптов и инструментов.

3. Затраты на внедрение и обучение: внедрение КСАТ может потребовать значительных затрат на приобретение лицензий, обновление инфраструктуры и обучение команды тестирования.

4. Сложность настройки и поддержки: некоторые КСАТ могут быть сложными в настройке и поддержке, требовать специализированных знаний и опыта. Это может потребовать дополнительных ресурсов и усилий со стороны команды тестирования.

5. Неполнота и ограничения инструментов: каждый инструмент КСАТ имеет свои ограничения и возможности. Некоторые функциональности или платформы могут быть недоступны или ограничены в выбранном инструменте, что может потребовать поиска альтернативных решений.

Итак, компьютерные средства автоматизации тестирования ПО предоставляют значительные преимущества в эффективности, надежности и покрытии тестирования. Однако их использование также имеет ограничения, которые необходимо учитывать при выборе и внедрении соответствующего инструмента.

5.11.6. Перспективы развития КСАТ

В сфере компьютерных средств автоматизации тестирования ПО наблюдается постоянное развитие и совершенствование. Технологии и инструменты обновляются и адаптируются к новым требованиям и вызовам, с которыми сталкиваются разработчики и тестировщики.

Выделим некоторые из направлений развития КСАТ в будущем.

1. Использование искусственного интеллекта и машинного обучения: искусственный интеллект и машинное обучение могут

играть важную роль в автоматизации тестирования ПО. Алгоритмы машинного обучения могут использоваться для автоматического создания тестовых сценариев, оптимизации выбора тестовых данных и обнаружения необычного поведения программы.

2. Расширение возможностей тестирования мобильных приложений: с увеличением популярности мобильных приложений и разнообразия платформ возникает потребность в развитии специализированных инструментов для автоматизации тестирования мобильных приложений. Развитие таких инструментов поможет улучшить качество и покрытие тестирования мобильных приложений.

3. Интеграция с DevOps и CI/CD: в современных процессах разработки программного обеспечения широко используется подход DevOps и непрерывной интеграции, и доставки (CI/CD). Будущее КСАТ связано с интеграцией и поддержкой этих процессов, чтобы обеспечить автоматическое тестирование и высокую скорость доставки ПО.

4. Расширение возможностей тестирования Интернета вещей (IoT): с ростом числа устройств IoT возникает потребность в инструментах для тестирования сложных экосистем IoT. Будущие КСАТ должны предоставлять возможности для тестирования и валидации различных аспектов IoT, таких как взаимодействие между устройствами, безопасность и производительность.

5. Аналитика и отчетность: аналитика и отчетность являются важными аспектами автоматизации тестирования. В будущем можно ожидать развития инструментов, позволяющих анализировать результаты тестирования, создавать наглядные отчеты и предоставлять ценную информацию о качестве ПО.

В целом развитие компьютерных средств автоматизации тестирования ПО будет направлено на повышение эффективности, точности и покрытия тестирования, а также на поддержку современных процессов разработки и доставки ПО.

5.11.7. Рекомендации по выбору и использованию компьютерных средств автоматизации тестирования ПО

При выборе и использовании компьютерных средств автоматизации тестирования ПО следует учитывать ряд рекомендаций.

1. Анализ требований проекта: важно понять требования проекта к тестированию и определить, какие задачи требуется автоматизировать. Это поможет выбрать соответствующие инструменты и функциональность КСАТ.

2. Оценка возможностей и ограничений инструментов: перед выбором конкретного инструмента автоматизации тестирования необходимо оценить его возможности, поддержку нужных технологий и платформ, а также ограничения и требования к инфраструктуре.

3. Обучение и поддержка: при внедрении КСАТ в команду тестирования необходимо предоставить обучение по использованию инструментов и поддержку в процессе работы. Это поможет сократить время на адаптацию и эффективно использовать инструменты.

4. Постепенное внедрение и оптимизация: рекомендуется внедрять автоматизацию тестирования поэтапно, начиная с наиболее приоритетных задач. Постепенное внедрение позволит учесть особенности проекта и оптимизировать процесс автоматизации.

5. Мониторинг и оценка результатов: важно проводить мониторинг эффективности автоматизации тестирования и оценивать результаты. Это позволит выявить потенциальные проблемы и внести корректировки в процесс автоматизации.

Использование компьютерных средств автоматизации тестирования ПО требует не только выбора правильных инструментов, но и учета специфики проекта, команды и требований к ПО. Применение этих рекомендаций поможет увеличить эффективность и успешность автоматизации тестирования.

Компьютерные средства автоматизации тестирования ПО являются неотъемлемой частью современного процесса разработки и тестирования программного обеспечения. Они позволяют увеличить эффективность, точность и покрытие тестирования, а также сократить время и затраты на тестирование.

Выбор правильного КСАТ зависит от множества факторов, таких как требования к ПО, бюджет, тип ПО и платформы, интеграция и обучение. При внедрении КСАТ важно разработать детальный план, обеспечить обучение и поддержку команды тестирования, а также постепенно внедрять и оптимизировать автоматизацию.

Необходимо учитывать и преимущества, и ограничения использования КСАТ. Важно осознавать, что автоматизация не может заменить полностью ручное тестирование и требует постоянного внимания и обновления.

Использование компьютерных средств автоматизации тестирования ПО помогает сократить время и ресурсы, улучшить качество и достоверность результатов тестирования. Однако это должно быть внедрено и управляемо с учетом специфики проекта и его требований.

В будущем развитие компьютерных средств автоматизации тестирования ПО будет направлено на использование искусственного интеллекта и машинного обучения, расширение возможностей тестирования мобильных приложений, интеграцию с DevOps и CI/CD, расширение возможностей тестирования IoT и улучшение аналитики и отчетности.

Выбор и использование КСАТ требует анализа требований проекта, оценки возможностей и ограничений инструментов, обучения и поддержки, постепенного внедрения и оптимизации, а также мониторинга и оценки результатов.

5.12. РАБОТА С ОШИБКАМИ

Между программистами и тестировщиками необходим специальный интерфейс общения. Тестировщики должны

удостовериться, что ошибки, которые они обнаружили, задокументировали и отправили разработчикам, действительно исправлены. Кроме того, менеджерам нужна статистика по найденным и исправленным ошибкам – это хороший инструмент контроля проекта. Все это изображено на рис. 5.9.

Чтобы справиться с этим потоком информации и обеспечить необходимые в работе, удобные сервисы, существует специальный класс программных средств – средства контроля ошибок (bug tracking systems).

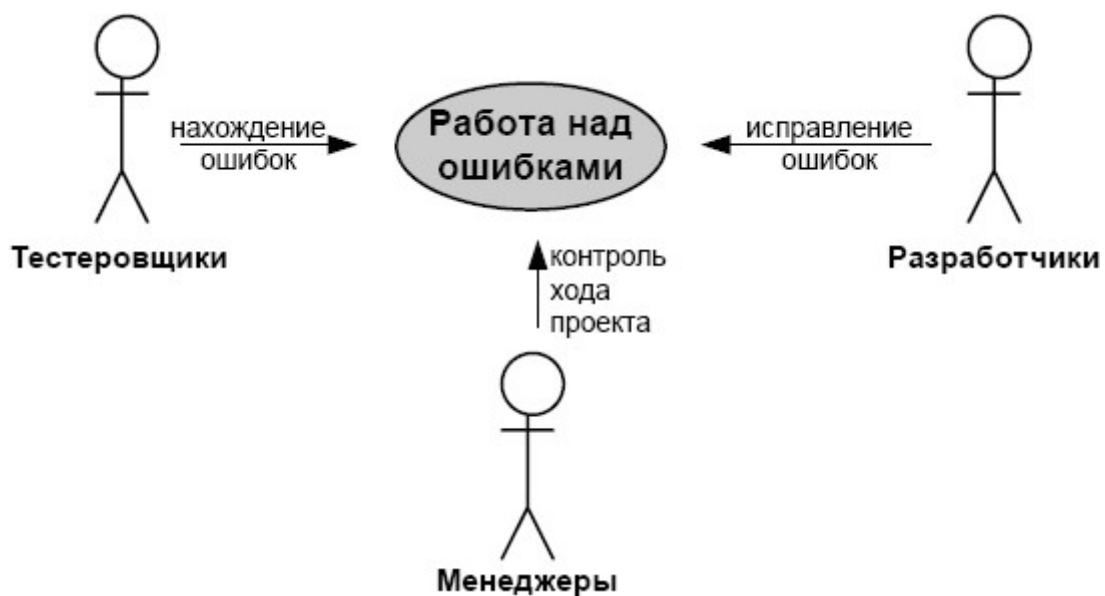


Рис. 5.9. Взаимодействие тестировщиков и разработчиков

Описание ошибки в системе контроля ошибок имеет следующие основные атрибуты:

- ответственного за ее проверку – тестировщика, который ее нашел и который проверяет, что исправления, сделанные разработчиком, действительно устраняют ошибку;
- ответственного за ее исправление – разработчика, которому ошибка отправляется на исправление;
- состояние, например, ошибка найдена, ошибка исправлена, ошибка закрыта, ошибка вновь проявилась и т.д.

Этот список существенно дополняется в различных программных средствах контроля ошибок.

Использование этих систем является общей практикой в разработке ПО наравне со средствами версионного контроля и многими иными инструментами. Они включают:

- базу данных для хранения ошибок;
- интерфейс к этой базе данных для внесения новых ошибок и задания их многочисленных атрибутов, для просмотра ошибок на основе различных фильтров – например, все найденные ошибки за последний месяц, все ошибки, за которые отвечает данный разработчик и т.д.;
- сетевой доступ, так как проекты все чаще оказываются распределенными;
- программный интерфейс для возможностей программной интеграции таких систем с другим ПО, поддерживающим разработку ПО (например, со средствами непрерывной интеграции – они могут автоматически вносить в базу данных найденные при автоматическом прогоне тестов ошибки).

5.13. Пять принципов чистых тестов

Качественный тест должен обладать рядом характеристик. К ним можно отнести читабельность теста, сопровождаемость, возможность поддержки в актуальном состоянии. Для разработки качественных тестов желательно придерживаться следующих принципов.

Принципы F.I.R.S.T.: Fast (Быстрота), Independent (Независимость), Repeatable (Повторяемость), Self-Validating (Самодостоверность), Timely (Своевременность).

Быстрота (Fast). Тесты должны выполняться быстро. Так как тесты необходимо проводить при каждом изменении кода, то их длительная работа не будет способствовать эффективности процесса разработки ПО.

Независимость (Independent). Результаты выполнения одного теста не должны быть входными данными для другого. Все тесты должны выполняться в произвольном порядке, поскольку в противном случае при сбое одного теста каскадно нарушится выполнение целой группы тестов.

Повторяемость (Repeatable). Тесты должны давать одинаковые результаты независимо от среды выполнения. Результаты не должны зависеть от того, выполняются ли они на конкретном локальном компьютере или сервере.

Самодостоверность (Self-Validating). Результатом выполнения теста должно быть логическое значение. Тест либо пройден, либо не пройден.

Своевременность (Timely). Тесты должны создаваться своевременно. Создавать тесты желательно перед кодом или как минимум параллельно с кодом.

5.14. РЕФАКТОРИНГ. ДИЗАЙН КОДА

Качество кода разрабатываемого программного продукта во многом зависит от того, насколько легко модифицировать код, вносить в него исправления и добавления. Созданное программное приложение может выполнять требуемые функции, но иметь проблемы с внесением изменений или пониманием созданного кода. В этом случае такое ПО нельзя назвать качественным, так как на этапе его сопровождения могут возникнуть проблемы с его модификацией при изменении пользовательских требований. Если при изменении исходного кода возникают проблемы, то такой программный продукт нельзя назвать качественным и на этапе сопровождения внесения модификаций в соответствии с требованиями заказчика могут оказаться нереализуемыми. Для улучшения качества кода применяют рефакторинг.

Рефакторинг – это процесс изменения программного кода таким образом, что его внешнее поведение не изменяется, а внутренняя структура улучшается. Это регулярная деятельность по переписыванию кода с целью предотвращения ухудшения его структуры (избыточность, неиспользуемые или слабо используемые фрагменты, запутанная и труднопонимаемая структура) при внесении в него новой функциональности, исправления ошибок и пр.

Следовательно, в процессе проектирования для создания высококачественного программного продукта необходимо обеспечить выполнение не только функциональных требований, но и нефункциональных, в частности, удобства сопровождения, что предполагает возможность и простоту внесения изменений в код, а также возможность легкого понимания созданного кода.

Таким образом, рефакторинг – это процесс улучшения кода, без изменения его функциональности, с целью облегчения понимания его работы и упрощения дальнейших модификаций. Сложный и запутанный код может существенно влиять на эффективность работы программного продукта. Рефакторинг позволяет сделать код читаемым, гибким для будущих изменений.

Перечислим основные причины проведения рефакторинга кода.

Накопленные за время разработки проблемы и недостатки в коде называются техническим долгом и могут существенно усложнить процесс создания программного продукта. Рефакторинг способен снизить технический долг. С увеличением объема исходного кода ухудшается его читаемость, поэтому на определенном этапе проведение рефакторинга поможет повысить читаемость кода, а следовательно, при расширении кода уменьшит время на чтение исходного кода и в дальнейшем будет способствовать написанию автоматических тестов. Хорошо структурированный код позволяет легко добавлять новые функции программного продукта и вносить изменения без риска нарушить функциональность готовых блоков.

Можно выделить ряд причин, по которым программному коду необходим рефакторинг.

При построении автоматизированных тестов наличие сложных зависимостей классов, вложенностей функций может стать существенным препятствием. Изменения требований к программному продукту приводят к изменениям в коде и нередко всей архитектуре программы. Если эти изменения вносятся несвоевременно или с ошибками, то возникает проблема технического долга. Существенной проблемой может быть

отсутствие договоренности в команде разработчиков, в результате которого придется выполнять рефакторинг.

Следует отметить, что для проведения рефакторинга необходимо иметь надежные тесты, которые обеспечивают контроль соблюдения функциональных требований при улучшении дизайна кода ПО. В частности, для контроля качества продукта производится модульное и регрессионное тестирование.

5.14.1. Дизайн программного продукта

Некачественный дизайн кода определяется по ряду признаков:

- жесткость – характеристика программы, выражающая затруднение внесения изменений в код;
- хрупкость – свойство программы повреждаться во многих местах при внесении единственного изменения;
- косность – характеризуется тем, что код содержит части, которые могли бы оказаться полезными в других системах, но усилия и риски, сопряженные с попыткой отделить эти части кода от оригинальной системы, слишком велики;
- ненужная сложность – характеризуется тем, что программа содержит элементы, которые не используются в текущий момент;
- ненужные повторения – состоят в повторяющихся фрагментах кода в программе;
- непрозрачность, которая характеризует трудность кода для понимания.

Для создания качественного дизайна кода целесообразно применять некоторые принципы и паттерны проектирования ПО.

1. Принцип единственной обязанности определяет, что у класса должна быть только одна причина для изменения. Из этого принципа следует, что классу целесообразно поручать только одну обязанность.

2. Принцип открытости / закрытости определяет, что программные сущности (классы, модули, функции) должны быть открыты для расширения, но закрыты для модификации.

3. Принцип подстановки определяет, что должна быть возможность вместо базового класса подставить любой его подтип.

4. Принцип инверсии зависимостей определяет два положения:

1) модули верхнего уровня не должны зависеть от модулей нижнего уровня. И те и другие должны зависеть от абстракций;

2) абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций.

5. Принцип разделения интерфейсов определяет, что клиенты должны знать только об абстрактных интерфейсах, обладающих свойством сцепленности.

Паттерны проектирования – это повторяемая архитектурная конструкция, представляющая собой решение проблемы проектирования в рамках некоторого часто возникающего контекста.

Паттерны предлагают универсальные, проверенные практикой решения. Среди них следует выделить те, которые целесообразно применять при гибкой разработке программного обеспечения. Это паттерны Команда, Стратегия, Фасад, Посредник, Одиночка, Фабрика, Компоновщик, Наблюдатель, Абстрактный сервер / адаптер / шлюз, Заместитель и Шлюз, Посетитель и Состояние.

Использование паттернов при разработке позволяет создавать легко модифицируемое и сопровождаемое ПО.

5.14.2. Использование принципов и паттернов проектирования ПО

Рассмотрим некоторые из них в качестве примера.

Порождающий паттерн «Фабрика» генерирует экземпляр для клиента без предоставления какой-либо логики экземпляра. Это функция или метод, возвращающий объекты разных прототипов или классов из вызова какого-то метода, который считается новым. Аналогия паттерна следующая: если нам

необходимы двери, мы не будем их создавать самостоятельно, а закажем на фабрике.

Приведем пример использования паттерна «Фабрика». Вначале создадим интерфейс объекта (дверь) и его реализацию.

```
interface Door
{
    public function getWidth(): float;
    public function getHeight(): float;
}

class WoodenDoor implements Door
{
    protected $width;
    protected $height;

    public function __construct(float $width, float $height)
    {
        $this->width = $width;
        $this->height = $height;
    }
    public function getWidth(): float
    {
        return $this->width;
    }

    public function getHeight(): float
    {
        return $this->height;
    }
}
```

Теперь разработаем фабрику дверей, которая создаёт и возвращает нам двери.

```
class DoorFactory
{
    public static function makeDoor($width, $height):
    Door
    {
```

```

        return new WoodenDoor($width, $height);
    }
}

```

Использование описанных команд:

```

$door = DoorFactory::makeDoor(100, 200);
echo 'Width: ' . $door->getWidth();
echo 'Height: ' . $door->getHeight();

```

Структурный паттерн «Фасад» представляет упрощенный интерфейс для сложного и более крупного фрагмента кода, например библиотеки классов. Создадим класс computer:

```

class Computer
{
    public function getElectricShock()
    {
        echo "Ouch!";
    }

    public function makeSound()
    {
        echo "Beep beep!";
    }

    public function showLoadingScreen()
    {
        echo "Loading..";
    }

    public function bam()
    {
        echo "Ready to be used!";
    }

    public function closeEverything()
    {
        echo "Bup bup bup buzzzz!";
    }

    public function sooth()

```

```

    {
        echo "Zzzzzz";
    }

    public function pullCurrent()
    {
        echo "Haaah!";
    }
}

```

Основной код:

```

class ComputerFacade
{
    protected $computer;

    public function __construct(Computer $computer)
    {
        $this->computer = $computer;
    }

    public function turnOn()
    {
        $this->computer->getElectricShock();
        $this->computer->makeSound();
        $this->computer->showLoadingScreen();
        $this->computer->bam();
    }

    public function turnOff()
    {
        $this->computer->closeEverything();
        $this->computer->pullCurrent();
        $this->computer->sooth();
    }
}

```

Использование:

```

$computer = new ComputerFacade(new Computer());
$computer->turnOn();
$computer->turnOff();

```

Поведенческий паттерн «Команда» позволяет инкапсулировать действия в объекты. Аналогия паттерна следующая: вы (Client) включаете (Command) телевизор (Receiver) с помощью пульта (Invoker).

Рассмотрим пример использования этого паттерна. Сначала определим получателя, содержащего реализации каждого действия, которое может быть выполнено.

```
// Receiver
class Bulb
{
    public function turnOn()
    {
        echo "Bulb has been lit";
    }

    public function turnOff()
    {
        echo "Darkness!";
    }
}
```

Теперь создадим интерфейс, который будет реализовывать каждая команда. Также сделаем набор команд.

```
interface Command
{
    public function execute();
    public function undo();
    public function redo();
}

// Command
class TurnOn implements Command
{
    protected $bulb;

    public function __construct(Bulb $bulb)
    {
        $this->bulb = $bulb;
    }
}
```



```

    public function execute()
    {
        $this->bulb->turnOn();
    }

    public function undo()
    {
        $this->bulb->turnOff();
    }

    public function redo()
    {
        $this->execute();
    }
}

class TurnOff implements Command
{
    protected $bulb;

    public function __construct(Bulb $bulb)
    {
        $this->bulb = $bulb;
    }

    public function execute()
    {
        $this->bulb->turnOff();
    }

    public function undo()
    {
        $this->bulb->turnOn();
    }

    public function redo()
    {
        $this->execute();
    }
}

```

Далее создадим вызывающего `Invoker`, с которым будет взаимодействовать клиент для обработки команд.

```
// Invoker
class RemoteControl
{
    public function submit(Command $command)
    {
        $command->execute();
    }
}
```

Использование команд.

```
$bulb = new Bulb();

$turnOn = new TurnOn($bulb);
$turnOff = new TurnOff($bulb);

$remote = new RemoteControl();
$remote->submit($turnOn);
$remote->submit($turnOff);
```

5.15. УПРАВЛЕНИЕ СБОРКАМИ

Процедура преобразования программного кода в машинный код осуществляется посредством компиляции и создания `exe`- и `dll`-файлов по исходникам проекта. Данная процедура может выполняться многократно в день каждым разработчиком на его ПК и с его собственной версией проекта. При этом могут отличаться параметры компиляции, а также набор подпроектов, собираемых разработчиком. К примеру, он может собирать не весь проект, а только какую-то его часть. Другая же часть либо им не используется вовсе, либо не пересобирается очень давно. При этом если не собирать регулярно итоговую версию проекта, то общая интеграция может выявить много разных проблем, таких как несоответствие друг другу различных частей проекта, наличие специфических ошибок, возникших из-за того, что

отдельные проекты разрабатывались без учета параметров компиляции.

В связи с этим процедуру сборки проекта автоматизируют, т.е. выполняют не из среды разработки, а из специального скрипта – build-скрипта. Этот скрипт используется тогда, когда разработчику требуется полная сборка всего проекта. А также он используется в процедуре непрерывной интеграции, т.е. регулярной сборке всего проекта (как правило, каждую ночь). Процедура непрерывной интеграции включает и регрессионное тестирование, и часто создание инсталляционных пакетов. Общая схема автоматизированной сборки представлена на рис. 5.10.



Рис. 5.10. Блок-схема автоматизированной сборки приложения

Тестировщики должны тестировать по возможности итоговую и целостную версию продукта, так что результаты регулярной сборки оказываются очень востребованы. Кроме того, наличие базовой, актуальной, целостной версии продукта позволяет организовать разработку в итеративно-инкрементальном стиле, т.е. на основе внесения изменений. Такой стиль разработки называется baseline-метод.

5.15.1. Сценарии сборки

Выбирать технологию управления сборками решения следует исходя из степени соответствия их компонентов конкретной ситуации. В командном программном процессе используют сценарии сборки.

Сценарий сборки – это унифицированный и отчасти формализованный процесс создания сборок ПО.

При возникновении сомнений следует использовать самый простой из возможных сценариев – плановую сборку. Более сложный сценарий применяют только при необходимости. Далее перечислены наиболее распространенные сценарии командных сборок.

1. Ежедневная сборка. Используется в большинстве команд для предоставления тестировщикам и другим потребителям согласованных двоичных файлов. Эти сборки более эффективны, чем событийные, поскольку событий может быть много, а ресурсы не бесконечны.

2. Ежедневная сборка и непрерывная интеграция. Используется для оперативного предоставления разработчикам информации о качестве кода. Выполняется при каждом возврате исходного кода. Подходит для небольших команд разработчиков, а крупные команды с частым возвратом исходного кода и продолжительной сборкой рискуют перегрузить сервер сборки.

3. Несколько лабораторий сборки, в каждой из которых используются ежедневные сборки и непрерывная интеграция. Применяются в очень больших командах для повышения качества и своевременности сборки, при этом для сокращения количества сбоев можно использовать возврат кода с контролем качества, отклоняя ненадежный код, прежде чем он попадет в дерево. Команды с подразделениями могут использовать несколько серверов сборки (по одному для каждого подразделения), чтобы обеспечить концентрацию каждой группы на выполнении определенной задачи.

4. Плановая сборка выполняется через определенные промежутки времени. Цель плановой сборки – создание согласованных надежных сборок, которые можно использовать для получения информации о качестве кода. Плановые сборки обычно выполняются ежедневно, но могут реже или чаще, в зависимости от потребностей.

Чаще всего потребителями плановых сборок являются:

– тестировщики;

- внутренние контролеры сборки;
- внешние контролеры.

Для создания плановой сборки требуется создание пакетного файла для выполнения сборки из командной строки и настройка выполнения пакетного файла по расписанию, например, с помощью планировщика Windows.

5. Непрерывная интеграция. При непрерывной интеграции сборка запускается каждый раз, когда происходит возврат кода после правки. Назначение такой сборки – максимально быстрое получение информации о качестве кода. Потребителями непрерывной интеграции обычно являются команды разработчиков.

В зависимости от размера команды, размера сборки и частоты возврата кода выполняется:

- сборка сразу после возврата кода;
- скользящая сборка после указанного числа возвратов кода или через определенный период времени.

6. Возврат кода с контролем качества. Возврат кода с контролем качества означает, что возвращаемый код должен удовлетворять определенным стандартам качества, чтобы его можно было добавить в дерево исходного кода. За счет выполнения ряда тестов снижается количество сбоев сборки и повышается ее качество.

Существует несколько типичных сценариев сборки ПО. Как правило, это касается крупных проектов и связано с одной из следующих ситуаций.

1. Разработка проекта с внешними зависимостями требует использования ссылок проекта на другие проекты внешних систем, чтобы сборка работала на сервере сборки без каких-либо дополнительных действий. Если для ссылок на внешние зависимости используется сопоставление рабочей области на клиентской стороне, сопоставление необходимо поддерживать на сервере сборки, чтобы она выполнялась успешно.

2. Разработка проекта установки. Для компиляции данного проекта и копирования двоичных файлов в место накопления используется собственный послесборочный этап.

При работе в крупных проектах следует учитывать следующие отличия больших команд:

- требуют более сложной структуры ветвления и слияния;
- часто требуют управления зависимостями между разными решениями и командными проектами;
- чаще возникает необходимость поддержки нескольких сборок для компонентов и команд разработчиков.

С точки зрения крупных команд разработчиков имеют значение такие особенности:

- в крупных командах сборка обычно занимает больше времени. При этом частота выполнения сборки с непрерывной интеграцией не должна превышать оптимальное время сборки во избежание очередей и увеличения нагрузки на сервер сборки;
- если в команде есть подразделения, следует использовать отдельный сервер сборки для каждого подразделения. Это позволит каждому подразделению сосредоточиться на выполнении определенной задачи, например, на интеграции или разработке;
- подразделения, занимающиеся интеграцией, обычно работают только с плановыми сборками. Подразделения, занимающиеся разработкой, могут работать как с плановыми сборками, так и с непрерывной интеграцией.

5.15.2. Настройка плановой сборки

Частота создания сборок – одно из самых важных решений, принимаемых при настройке плановых сборок. Сборки могут выполняться каждый час, каждый день или каждую неделю.

Ежечасные сборки. Если в проекте достаточно возвратов, чтобы значительные изменения происходили в течение часа, но непрерывная интеграция не используется, нужно настроить ежечасное создание сборок. Это позволит разработчикам быстро получить информацию о качестве кода. Ежечасные сборки можно также предоставлять тестировщикам и другим членам команды.

Ежедневные сборки. Это наиболее распространенная частота сборки. При этом разработчики каждое утро получают

готовую к тестированию сборку со всеми вчерашними изменениями.

Еженедельные сборки. В крупных и сложных проектах сборка может длиться днями, и тогда лучшим вариантом становится еженедельная сборка. При этом команда тестировщиков в начале каждой недели будет получать сборку, включающую все изменения, внесенные на прошлой неделе.

5.15.3. Настройка непрерывной интеграции

Рассмотрим один из наиболее распространенных сценариев сборки ПО – непрерывную интеграцию.

Непрерывная интеграция (Continuous Integration) – процесс создания сборок при каждом возврате кода в систему управления исходным кодом.

Перечислим различные стратегии непрерывной интеграции.

1. Сборка при каждом возврате. Это простейшая стратегия непрерывной интеграции, позволяющая получать информацию о качестве кода наиболее оперативным способом. Однако, если возвраты происходят слишком часто и сервер сборки не успевает их обрабатывать, следует использовать другой подход, при котором сборка выполняется после определенного числа возвратов или по истечении определенного периода времени. Чтобы принять обоснованное решение, важно определить следующие параметры:

- время выполнения командной сборки;
- среднее время между возвратами кода;
- интервал времени, в котором возвраты осуществляются особенно часто.

Если время сборки превышает среднее время между возвратами, сборки начнут выстраиваться в очередь, поскольку одна сборка еще не будет выполнена, когда произойдет возврат, инициирующий выполнение следующей сборки. Чрезмерная длина очереди сборок может повлиять на производительность сервера и заблокировать начало других сборок, например

плановых. Актуальной задачей становится выявление временного окна, в котором часто выполняются возвраты, и определение, достаточно ли времени, чтобы очередь успела очиститься после окончания периода высокой нагрузки.

2. Сборка после определенного числа возвратов. Если в результате создания сборок после каждого возврата сервер оказался перегружен, следует выполнять сборку через определенное число возвратов. Это самый простой сценарий скользящего выполнения сборки, но у него есть серьезный недостаток. Поскольку должно произойти определенное число возвратов, прежде чем сервер выполнит сборку, последний возврат рабочего дня почти гарантированно сборку не инициирует, а значит, информация о качестве кода будет получена с задержкой.

3. Сборка по истечении определенного промежутка времени. Создание сборок через определенные промежутки времени после возврата – более совершенный способ. Он позволяет гарантированно получить сборку с небольшой задержкой относительно последнего возврата. При этом с каждой сборкой может быть связано разное число возвратов – от одного до нескольких. Чем больше возвратов, тем сложнее определить, который из них вызвал сбой.

4. Сборка по обоим условиям. Создание сборок по обоим условиям – после определенного числа возвратов или по истечении определенного промежутка времени – позволяет обеспечить максимальную согласованность сборок, по-прежнему не перегружая сервер. Одним из вариантов является так называемая *скользящая сборка*, которая применяется в случае, если при непрерывной интеграции возникает большая очередь сборок, обработка которой требует много времени. При этом следует использовать средний интервал между возвратами, чтобы задать число возвратов, которое должен получить сервер, прежде чем выполнить сборку. Для того чтобы гарантировать выполнение сборки даже в том случае, если получено небольшое число возвратов, используется таймаут.

Для определения оптимального интервала выполнения сборки нужно разделить среднее время между возвратами на длительность процесса сборки. Допустим, сборка занимает 10 мин., а среднее время между возвратами – 5 мин. Установите выполнение сборки после двух возвратов и таймаут 10 мин. При этом сборка в любом случае завершится к тому времени, когда будет инициирована следующая сборка. Если сервер все равно чрезмерно перегружается, эти значения можно увеличить.

5. Непрерывная интеграция. По умолчанию в СУ ЖЦ не включено решение непрерывной интеграции, но в нем имеется среда, позволяющая реализовать такое решение самостоятельно. Для настройки непрерывной интеграции в СУ ЖЦ используется решение, предоставленное командой разработчиков Visual Studio Team System. Оно устанавливает веб-службу, работающую от имени учетной записи, имеющей доступ к серверу СУ ЖЦ. Сервер СУ ЖЦ способен отправлять сообщения электронной почты или вызывать веб-службу при наступлении определенного события. Предлагаемое решение непрерывной интеграции использует механизм событий для связывания веб-службы с событием Checkin Event. В результате каждый раз, когда происходит возврат, веб-служба инициирует выполнение сборки.

6. УПРАВЛЕНИЕ ПРОЕКТАМИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Управление проектами – это весьма масштабная деятельность по решению задач и достижению поставленных целей проекта. В этой главе рассматривается частный случай управления программными проектами.

Управление проектом – один из организационных процессов ЖЦ ПО, связан с вопросами планирования и организации работ, создания коллективов разработчиков и контроля за сроками и качеством работ.

Процессы управления проектом – это неосновные процессы ЖЦ ПО. Управление проектом предполагает планирование по следующей схеме.

1. Концептуальное описание проекта. На этом этапе формируется представление о том, какой конечный результат проекта хотят получить все заинтересованные стороны. Заинтересованные лица: заказчик и разработчики. Причем заказчик может быть представлен непосредственным заказчиком, тем организационным объектом, который будет эксплуатировать программную систему, а также другими стейкхолдерами, например, клиентами или поставщиками этого заказчика, которые будут работать с разрабатываемыми сервисами. Все должны изложить свою точку зрения, затем планировщик будет выстраивать план – концептуальные представления, описания. Надо понять, что в итоге хотят получить потребители: какой интерфейс, какие функциональные особенности приложения и пр. Идеальным вариантом является сбалансированный документ по графической части, поэтому это должно быть и схемное, и текстовое описание.

2. Формулирование сценариев. На этом этапе с участием заказчика формулируется начальный набор сценариев использования ПО. Здесь последовательно выстраивается основной сценарий работы приложения в нормальном режиме. Также строится альтернативный(е) сценарий(и), когда возникают

отклонения от нормы, хватает каких-то данных или нет сетевого соединения, имеет место сбой базы данных либо другие редкие типовые ситуации, которые могут возникнуть и привести к отклонениям от нормы.

Заказчик оценивает готовый сценарий с позиции собственных интересов. После того как сценарий проанализирован, можно построить UML для иллюстрации функционала хотя бы в статике в виде моделей классов. Это нужно, чтобы понять, какие поля в этих классах есть, и выстроить прообраз будущего хранилища данных. Далее более подробно выписываются функциональные возможности приложения и так далее по жизненному циклу.

3. Формирование набора функциональных возможностей для реализации выбранных сценариев. На этом этапе сценарии разбиваются на отдельные компоненты, важные с точки зрения заказчика. Благодаря этому появляется возможность обсуждать ожидания заказчика в отношении этих компонентов. Здесь можно уже строить детализированную схему и на каждый компонент обращать пристальное внимание, обсуждать детали. Когда они обсуждены, можно сформировать набор элементов и построить структуру на физическом уровне в виде модели компонентов UML.

4. Формирование набора рабочих элементов. Сценарии и компоненты разбиваются на конкретные задачи, между которыми можно распределить ресурсы, в том числе трудовые. Завершение рабочего элемента означает реализацию соответствующей функции или сценария. У каждого рабочего элемента есть начало и конец. Завершение рабочего элемента означает реализацию соответствующих функций или сценария. Поэтому каждый рабочий элемент имеет свою линию жизни и в принципе все это можно уже положить в основу плана, после чего остается только распределить по областям.

5. Распределение задач по областям. Области выделяются на основе компонентов или в зависимости от организации конкретной команды. Задачи могут быть по-разному

распределены между членами команды. В итоге имеем распределение работ по членам команды.

6. Создание плана работ. На этом этапе либо составляется полный график выполнения работ, либо производится разбиение работы на итерации, потому что, как правило, это итеративный процесс разработки. Планируется ряд итераций, на них выделяется время и бюджет. Дальнейшая детализация ведется при планировании отдельной итерации. В частности, каждый этап на предстоящей итерации должен тоже быть выверен с точки зрения временных и финансовых ресурсов исходя из бюджета итерации и ее временного задела. К тому же набор рабочих элементов требует распределения задач по областям разработки, т.е. по тем же этапам жизненного цикла. Так и получается итоговый план.

В итоге вся схема заканчивается сборками. В идеале должна быть поддержка системы управления жизненным циклом. Варианты с различными видами сборок – это очень гибкое решение, во многом определяет эффективность программного процесса и качество программного продукта.

6.1. ТИПИЧНЫЕ ПРОБЛЕМЫ УПРАВЛЕНИЯ ПРОЕКТАМИ

В распоряжении руководителя проекта имеется множество разнообразных инструментов для управления процессом разработки ПО. Как правило, эти инструменты практически никак не интегрированы с инструментами, используемыми для разработки ПО. Отсутствие интеграции и взаимодействия между инструментарием и командами разработчиков – главная проблема руководителя проекта. Обычно ему приходится сталкиваться со следующими трудностями.

1. Работа с разрозненными источниками информации. Инструменты для управления проектами обычно используются изолированно, что приводит к появлению разрозненных источников информации, которые не так просто объединить. Кроме того, обычно бывает сложно объединить данные по управлению проектом с данными, предоставляемыми другими

членами команды, например, учесть несоответствие между ожидаемым и фактическим поведением системы. Такой диссонанс может пагубно сказаться на программном процессе.

Здесь корнем является сама процедура планирования проекта, потому что у сложных проектов довольно сложная структура, десятки разработчиков, сотни автоматизированных функций, миллионы пользователей. При необходимости автоматизации программного процесса у руководителя проекта имеется инструментарий, а у отдельных действующих лиц – какие-то фрагменты этого комплекса. Идеальным вариантом является использование системы управления жизненным циклом, интегрирующим все инструменты. Инструменты управления процессом разработки ПО и самой разработкой должны быть строго взаимосвязаны.

2. Сложности со сбором показателей проекта. Показатели проекта очень важны для контроля за его состоянием, принятия обоснованных решений и получения ответа на фундаментальные вопросы, например, «Будет ли проект поставлен вовремя и будет ли выполнен бюджет?». Отвечая на ключевые вопросы, руководитель проекта обычно полагается на данные Microsoft Office Project или системы отслеживания ошибок, используемой группами разработки и тестирования. Объединить данные, предоставляемые этими несопоставимыми системами, сложно, на это уходит много времени, к тому же в процессе сведения данных легко допустить ошибки. Для большинства показателей, формируемых инструментарием, не существует унифицированной системы хранения и доступа. Создание отчетов по таким данным – это обычно напряженный ручной труд, связанный с многократным копированием и вставкой информации из разных источников.

3. Сложности с выполнением требований. Это распространенная проблема не только в сфере управления проектами, но и в сфере их разработки. Зачастую работа, запланированная для группы разработки, требования заказчика и основные нефункциональные требования к создаваемой системе существенно отличаются друг от друга. Запланированный и

фактический объемы работы могут значительно отличаться. Из-за этих несоответствий теряются важные данные, что приводит к невыполнению требований.

Идеальный вариант, когда они полностью совпадают, все идет по плану. Но реальность такова, что могут возникать недоразумения или форс-мажорная ситуация, обстоятельства, которые выбивают из колеи коллектив. Из-за таких несоответствий теряются важные данные, что приводит к невыполнению требований. Для того чтобы отслеживать выполнение требований качественно, необходима единая система управления ЖЦ (СУ ЖЦ). Как уже неоднократно обсуждалось ранее, требования используются по всему жизненному циклу, начиная с их формирования еще на начальных этапах жизненного цикла и завершая эксплуатацией, где по-прежнему требования пересматриваются, и, опираясь на них, производится доработка системы. Все эти сложности должны выявляться, т.е. необходима активная работа по выявлению этих сложностей. Любые недоразумения, связанные с управлением требованиями, должны сразу же оперативно устраняться, чтобы минимизировать риски возникновения проблемы или провала проекта. Для этого есть проработанная технология управления требованиями – один из основных столпов программной инженерии, которая рассмотрена в главе 3. Технология управления требованиями должна быть качественно поставлена и автоматизирована.

4. Управление процессами и изменения в них. Еще один краеугольный камень программного процесса – его изменчивость. Процессуальные компоненты проектной деятельности достаточно динамичны сами по себе и поэтому склонны изменяться. Объяснить команде, каких процессов она должна придерживаться, очень сложно. Еще сложнее внести изменения в процессы для решения возникающих проблем, не снизив производительности команды. Для этого необходимо иметь строгий регламент. Однако стандартный унифицированный процесс – это не всегда хорошо, так как чем больше унификации, тем больше шаблонности в процессе. Между тем разработка ПО – это творческий процесс и уложить

его в прокрустово ложе каких-либо шаблонов, унифицированных механизмов – достаточно неблагоприятное дело. С одной стороны, это способствует максимальному порядку в делах, с другой – может сильно исказить (упростить) процесс, лишить его оригинальности, что негативно скажется на конечном продукте – он просто утратит свою эксклюзивность и будет похож на типовое решение, что, конечно, вовсе не украшает его, напротив, делает его серым, безликим и неконкурентоспособным на рынке, непривлекательным для заказчика. Здесь нужен баланс, и поэтому процессы не всегда являются строго регламентированными.

5. Своевременность внесения изменений в процессы без снижения при этом производительности команды – еще одна проблема, с которой сталкивается разработчик. Зачастую изменения приходится делать на ходу, и чтобы команда не страдала от этих изменений, необходимо максимально автоматизировать процесс. Вместе с тем автоматизировать нестандартное решение крайне сложно, тут надо найти баланс и сделать процедуру изменений максимально прозрачной, чтобы все были в курсе принимаемых решений, а не получали эти изменения внезапно. Ещё тяжелее, если по ходу проекта появляется новый инструментарий. В процессе менять или перестраивать инструментарий нехорошо, хотя бывает так, что это неизбежно. Но в любом случае надо минимизировать риски. Добиваются этого интеграцией, прозрачностью, что характерно для интегрированных систем управления ЖЦ.

6. Недостаток учитываемого обмена информацией и отслеживания задач. Взаимодействие и слаженность команды обычно обеспечиваются при помощи совещаний, списков задач, текущего положения дел, что позволяет правильно выбрать приоритеты. При этом отслеживать процесс выполнения отдельной задачи может быть очень сложно. Все это сказывается на обмене и отслеживании задач. Чтобы не возникало проблем, важно наладить качественное отслеживание процесса выполнения каждой задачи, для чего нужна автоматизация таких процессов. Кроме того, руководители проектов часто тратят

массу ценного времени на извлечение информации о состоянии работ из разных планов и списков. Члены команды тратят время на отчеты о состоянии и обновлении различных документов и форм. Ждать обновления различных документов – это тоже рутина, которую требуется автоматизировать.

7. Контроль качества – это процедура, которая осуществляется по всему циклу. Надо контролировать качество требований, рабочих элементов, проектных решений, тестов, сборок и т.д. Это довольно сложно. Предсказать количество и серьезность ошибок в создаваемом ПО трудно, поэтому обычно плановые оценки и затраты основываются на оптимистических предположениях. Эти оценки традиционно делаются с запасом, величина которого зависит от степени уверенности руководителя в текущем состоянии проекта. При этом важна объективная информация. Чтобы ее получать, нужна система управления качеством, которая тоже должна быть максимально автоматизирована. Все тесты необходимо сделать автоматическими, остальное – максимально ответственно тестировать вручную.

Система СУ ЖЦ призвана помочь в решении многих традиционных проблем, с которыми сталкиваются руководители проектов. Интегрированные в нее инструменты помогают командам совершенствовать процесс разработки ПО, а руководителям проектов – лучше контролировать этот процесс, обеспечить прозрачность процедур, процессуальных компонентов деятельности и наладить высококачественный программный процесс.

6.2. ФУНКЦИИ УПРАВЛЕНИЯ ПРОЕКТАМИ

Рассмотрим основные функции по управлению проектами.

1. Управление процессами. СУ ЖЦ включает руководство по процессу Microsoft Solution Framework (MSF, см. параграф 2.2), а также шаблоны процессов, благодаря которым новые командные проекты обеспечены типами рабочих элементов, отчетами, порталом проекта SharePoint и настройками

системы управления исходным кодом. Эти инструменты способствуют обмену данными через сети и их хранению на сервере в соответствующих репозиториях, куда можно сохранять всю информацию по проекту: показатели, документы, рабочие элементы проекта, настройки, а также обеспечить управление версиями, сборками и качеством. Всё это централизованно управляется на сервере.

2. Безопасность и разрешения. В новые проекты включены стандартные группы и разрешения, соответствующие типичным ролям команды разработки. Чтобы свести к минимуму риски, связанные с человеческим фактором, вводятся пользовательские роли. Как правило, проектом предусмотрены стандартные роли и группы пользователей. Выдаются разрешения, соответствующие типичным ролям команды разработки: одни только читают данные, другие изменяют и отправляют на сервер, третьи их извлекают оттуда и применяют в процессах, четвертые только наблюдают за этими процессами. Поэтому, согласно правам пользователей, разработчики получают необходимые инструменты и возможности для эффективного управления процессами, которыми они обязаны заниматься согласно своей роли в проекте, и отслеживания своих рабочих элементов, результатов, которые они создают. Например, разработчики пишут и потом могут отслеживать компоненты на дальнейших этапах ЖЦ (как они тестируются, собираются и т.д.), и как только возникает проблема (например, выявляется несоответствие), то разработчики оперативно узнают об этом. Еще до того, как им пришлют отчет, они уже знают, где есть проблема, и начинают размышлять над её устранением.

3. Централизованное управление рабочими элементами. Рабочие элементы, в том числе ошибки, риски, задачи, сценарии и требования к качеству обслуживания (quality of service, QoS), централизованно регистрируются, управляются и обслуживаются в БД рабочих элементов. Всё это прозрачно для пользователей, обладающих соответствующими правами. Централизованное хранение упрощает доступ и просмотр этих элементов всеми членами команды согласно их правам доступа.

4. Показатели и составление отчетов. В СУ ЖЦ включена возможность создания и отображения отчетов, которые превращают рабочие данные (например, рабочие элементы, результаты сборки и результаты тестирования) в показатели, хранящиеся в хранилище данных. Благодаря встроенным отчетам есть возможность запрашивать разнообразные показатели состояния и качества проекта.

5. Порталы проекта. Для каждого командного проекта СУ ЖЦ создает ассоциированный портал Microsoft Windows SharePoint®. Портал используется для управления документацией проекта, быстрого просмотра ключевых отчетов и оценки текущего состояния проекта.

Преимущества управления проектами из СУ ЖЦ следующие:

- централизованное управление – единое хранилище, интеграция, централизованное управление руководителем проекта; в крупных проектах, где модель группы не работает, нужна иерархическая модель команды и централизованное управление, здесь это предусмотрено;

- возможность отслеживать взаимосвязи рабочих элементов – всё в проекте связано и эти связи прозрачно отслеживаются в системе;

- интегрированное планирование и составление графика проекта – всё в проекте интегрировано, график у каждого, каждый видит свою задачу и её метаданные;

- расширенные возможности управления процессами – возможности управления настройками своих процессов в рамках прав, предоставленных ролями;

- расширенные возможности обмена информацией и взаимодействия внутри команды – эффективная коммуникация и регламентированное влияние на процессы смежных работников, с которыми осуществляется взаимодействие;

- подробные отчеты о ходе выполнения работ.

6.3. СОЗДАНИЕ ПРОЕКТОВ И УПРАВЛЕНИЕ ИМИ

В целом при создании проекта в СУ ЖЦ выделяют 11 этапов.

1. Выбор шаблона процесса. Можно использовать стандартный шаблон или создать собственный.
2. Создание командного проекта. В основу проекта ляжет выбранный шаблон.
3. Добавление в командный проект групп и членов.
4. Разработка итерационного цикла проекта. Включает создание итераций.
5. Преобразование сценариев в рабочие элементы.
6. Выбор сценариев, которые должны быть завершены в каждой итерации.
7. Определение требований QoS. Связывание требований со сценариями.
8. Разбиение сценариев с целью обеспечить разработчиков управляемыми рабочими элементами. Разработчики предварительно оценивают сроки выполнения для каждого рабочего элемента.
9. Создание графика проекта.
10. Определение критериев приемки. Критерии завершенности рабочих элементов согласовываются с требованиями QoS.
11. Определение требований к отчетам. Определяются ключевые показатели выполнения проекта и оговариваются требования к отчетам.

Безопасность и разрешения. При создании проекта в СУ ЖЦ независимо от выбранного шаблона создаются четыре группы, каждая из которых по умолчанию располагает теми или иными правами, определяющими, что разрешается делать членам этой группы:

- администратор проекта;
- участник;
- читатель;
- сервисы сборки.

Чтобы максимально удовлетворить требованиям по безопасности конкретной организации, в проекте можно создавать группы доступа самостоятельно. Это эффективный способ предоставить группе пользователей в командном проекте конкретный набор полномочий. Для повышения безопасности проекта следует давать группе лишь минимально необходимые права и вводить в нее только тех пользователей или группы, которым эти права необходимы.

При создании группы добавить ее в список, предоставить ей соответствующие права и включить в нее членов. По умолчанию вновь созданная группа командного проекта не получает никаких прав.

6.4. СТРАТЕГИИ КОМАНДНЫХ ПРОЕКТОВ

Чтобы начать работать с СУ ЖЦ, необходим по крайней мере один командный проект. Руководитель сразу должен понять, какого типа проект ему нужен. При создании нового проекта одновременно создается его веб-сайт с библиотеками документов, в которых содержатся шаблоны документов и отчеты. Также создается база данных рабочих элементов для отслеживания всех работ по проекту. Устанавливается шаблон методологии, определяющий правила, политику, группы доступа и запросы для всех работ по проекту. В системе управления исходным кодом создается ветвь исходного кода.

Выбирая структуру командного проекта, в первую очередь необходимо руководствоваться требованиями к ПО, сложностью проекта, его масштабом. Структура проекта может меняться в процессе работы. Существует три основные стратегии создания нового проекта, которые могут использоваться по отдельности или в сочетании друг с другом.

1. Один проект на приложение. Это самая распространенная стратегия создания командных проектов. Такой подход уместен при разработке как крупных, так и небольших приложений, а также для параллельной разработки нескольких версий одного

приложения. При таком подходе для каждого приложения, находящегося в разработке, создается по одному проекту.

2. Один проект на выпускаемую версию. Этот подход удобен для больших команд, работающих над долгосрочными проектами. Новый проект создается после выпуска каждой новой версии, и все начинается с нуля. При этом не приходится беспокоиться о переносе ресурсов предыдущего проекта, включая рабочие элементы. Этот подход позволяет улучшить шаблоны процессов или использовать новые на основании приобретенного опыта и знаний.

3. Один проект на команду. Этот подход удобен в больших проектах, над которыми работают несколько команд, там, где особенно важны централизованное управление и мониторинг процесса разработки. При таком подходе отдельный проект создается для каждой команды. Каждая команда соотносится с последовательностями операций, которые определяются типами рабочих элементов СУ ЖЦ, и располагает единым блоком для создания отчетов. Здесь выстраивается глубокая иерархическая структура управления, в которой выделяются руководители группы для каждого подпроекта, и руководитель проекта, который управляет всеми руководителями группы. У каждого руководителя есть своя подсистема контроля.

6.5. УПРАВЛЕНИЕ ПРОЦЕССАМИ

Ввиду того что процессы ЖЦ разработки интегрированы в СУ ЖЦ, обмен рабочими элементами между членами команды можно в значительной степени автоматизировать.

В СУ ЖЦ для определения набора инструкций и артефактов, например, руководств по процессу, шаблонов документов, стандартных рабочих элементов и т. д., используются шаблоны процессов. Шаблон процесса – это независимый набор инструкций, описывающих методологию разработки ПО для команд разработчиков. В шаблон процесса включаются следующие элементы.

1. Руководство по процессу. Предоставляется для каждого шаблона и содержит контекстно-зависимую информацию, справку и указания членам команды, нуждающимся в помощи и понимании определенной деятельности. Руководство по выполнению процесса интегрировано в Visual Studio Help System.

2. Шаблоны документов. Позволяют членам команды единообразно создавать новые документы (спецификации, оценки рисков и планы проектов).

3. Рабочие элементы и последовательность операций. У каждого типа рабочих элементов имеется собственный набор атрибутов и правил, определяющих последовательность операций рабочего элемента и распределение работ между членами команды.

4. Группы безопасности. Используются для определения прав по управлению и изменению отчетов, результатов работы, например, исходного кода и документации, и рабочих элементов. Администрировать группы безопасности проекта может его руководитель, для этого ему не обязательно быть администратором Windows.

5. Политики возврата после правки. Используются для применения правил и порогов качества ко всему коду, возвращаемому в систему управления исходным кодом. Так, можно поставить условие, что весь возвращаемый код должен отвечать определенному критерию (например, соответствовать корпоративным стандартам написания кода) или должен подвергаться модульному тестированию.

6. Отчеты. Используются для мониторинга исполнения процессов и текущего состояния проекта. В СУ ЖЦ встроено множество отчетов, включая отчеты о качестве кода, о соблюдении графика работ, об эффективности тестирования и др. Можно создавать собственные отчеты и настраивать существующие.

6.6. УПРАВЛЕНИЕ РАБОЧИМИ ЭЛЕМЕНТАМИ

Рабочие элементы (Work Item) используются в качестве блоков работы для обмена информацией и координации совместной деятельности в команде. Выбранный шаблон процесса предоставляет исходный набор типов рабочих элементов.

Рабочие элементы – основной инструмент руководителя проекта и ведущих разработчиков для отслеживания работ по проекту: что уже сделано и что еще предстоит сделать. Члены команды используют рабочие элементы для отслеживания собственной последовательности задач и для распределения работ, например, в форме ошибок или задач.

Обычно рабочие элементы в проектах используются в следующих целях:

- формирование требований пользователей или требований к приложению;
- контроль соответствия процессов разработки и тестирования этим требованиям;
- создание задач разработки, представляющих действия, которые необходимо выполнить для реализации компонентов и функций приложения;
- создание ошибок для представления дефектов в реализации компонентов и функций приложения;
- сортировка ошибок и задач для расстановки приоритетов и распределения между членами команды;
- отслеживание задач разработки для оценки темпов продвижения к завершению кода;
- отслеживание ошибок и других показателей качества для определения качества приложения и его готовности к поставке.

Для лучшей управляемости можно устанавливать связи между рабочими элементами, например, связать определенную задачу с соответствующим ей сценарием или требованием QoS (Quality of Service requirement – качество обслуживания требований).

Использование рабочих элементов в проекте определяется тем, какие типы рабочих элементов заданы в проекте. Описания рабочих элементов хранятся в шаблоне процесса, выбранном при создании проекта. Можно выбрать один из двух стандартных шаблонов – MSF Agile или MSF CMMI, или настроить рабочие элементы соответственно конкретным требованиям.

В шаблоне процесса имеются описания типов рабочих элементов, включая набор полей для каждого типа. Правильный выбор шаблона очень важен, поскольку в ходе выполнения проекта изменить его уже нельзя. При необходимости шаблон процесса можно дополнительно настроить, включив в него все необходимые типы рабочих элементов, отсутствующие в базовом шаблоне.

В шаблонах MSF Agile и MSF CMMI предусмотрен стандартный набор рабочих элементов с задачами, достаточными для начала процесса разработки ПО.

Шаблон MSF Agile. MSF ориентирован преимущественно на сложные проекты. Поэтому в этом шаблоне имеются следующие типы рабочих элементов.

1. Сценарий (scenario) используется для представления взаимодействия пользователя с приложением. Описывает конкретные шаги, необходимые для достижения цели. Сценарии должны быть конкретными, поскольку возможных способов действия может быть несколько.

2. Задача (task) используется для представления блока работы. У каждой роли свои требования к задачам. Например, разработчик использует для распределения работ задачи разработки, тестировщик планирует свои работы по тестированию, у сборщика – задачи подготовки сборки, контроля сборки и т.д.

3. Требование QoS (Quality of Service requirement) документирует характеристики системы: что система должна делать (функциональные требования) и как, какими свойствами должна обладать (нефункциональные требования).

4. Ошибка (bug) используется для информирования о потенциальной проблеме в системе. Проблема – несоответствие

ожидаемого поведения системы и ее фактического поведения. По итогу формируется отчет о несоответствии (Bug Report), который передается для разбирательства другим действующим лицам.

5. Риск (risk) используется для выявления и управления рисками в проекте. Актуально для сложных проектов, где риски высокие. Идентификация и эффективное управление рисками позволяет избежать многих проблем и на этапах ЖЦ при разработке системы, и при её эксплуатации для решения реальных задач. Риски устранить нельзя, но можно свести к минимуму, чтобы убрать их из критической области, чтобы они уже не так ярко были выражены. Даже если случится рисковое событие, то в результате предварительного риск-менеджмента будут применены выработанные стратегии превентивных мероприятий, которые позволят упредить возможные последствия рисков. Управление рисками программного процесса подробно рассмотрено в главе 7.

Шаблон MSF CMMI. Модель CMMI описывает зрелость стандартного процесса разработки ПО (подробнее см. параграф 2.2). Поэтому данный шаблон подходит для крупных и средних компаний разработки ПО, которые стремятся «дозреть» до высокого уровня. Находясь на верхнем, пятом уровне, компания может успешно конкурировать с другими компаниями и способна выдавать качественные продукты с высокой вероятностью успеха. В этом шаблоне имеются следующие типы рабочих элементов.

1. Требование (requirement). В отличие от шаблона MSF for Agile, здесь представляют и фиксируют главным образом функциональные требования, определенные на этапе анализа предметной области.

2. Запрос на изменение (change request) фиксирует любые запросы на внесение изменений, возникающие после этапа сбора требований, изменения которых в крупных проектах неизбежны и могут быть весьма значительны. Также меняются процесс, инструментарий, коллектив, ресурсы. Поэтому запрос на изменения здесь фиксирует, кристаллизует изменения. Как только появились изменения, фиксируется снимок и помещается

на рабочей доске (dashboard). Так отслеживаются все запросы, упорядочивается работа в условиях перманентных изменений.

3. Проблемы (issue), исправление которых необходимо отслеживать, детально рассмотрены ранее в этой главе. Для каждой проблемы создается отдельный рабочий элемент. Очень удобно сразу поставить проблему на учет и работать с ней, как с каким-то подучетным объектом. Совсем по-другому рассматривается проблема, когда она стоит на учете, и каждое утро на совещании происходит её текущий анализ:

- Что по этой проблеме сделано за истекшие сутки?
- Что мы будем делать с ней сегодня?

Такой прагматичный подход позволяет эффективно разрешать проблемы, исправления которых необходимо отслеживать, причем постоянно в режиме реального времени.

4. Задача (task) имеет тот же смысл, что и в отношении шаблона MSF for Agile. Используется для представления блока и идентификации объема работы. Специфика может быть связана с аспектами достижения заданного уровня зрелости программного процесса, который декомпозируется на эти задачи.

5. Рецензия (review) представляет собой блок работы по составлению отзывов, например, рецензии исходного кода, проекта и пр. Это очень полезный рабочий элемент, потому что фиксирует дополнительную информацию по каждому рабочему элементу, дающую дополнительный материал для обсуждений. Эти рецензии, как правило, размещаются в открытом доступе в соответствующем репозитории. К ним обычно имеют доступ все пользователи, но только автор может его менять, а посмотреть, прокомментировать, делать заметки могут все. Все это можно связывать с проблемами, запросами на изменение, с требованиями. Фактически выстраивается многоуровневая структура, которая, будучи грамотно организованной, дает достаточно хороший прагматичный эффект.

6. Ошибка (bug) используется для информирования о потенциальной проблеме в системе.

7. Риск (risk) используется для выявления и управления рисками в проекте.

В отличие от шаблона MSF for Agile, в данном шаблоне отсутствует элемент «Сценарий». В целом СММІ и так достаточно четко структурированная модель, которая сама по себе является сценарием и включает ряд сценариев по достижению заданного уровня зрелости программного процесса.

Структура рабочего элемента. Каждый тип рабочего элемента имеет типовую структуру:

- он имеет назначение и предполагаемое использование (например, ошибки используются для отслеживания дефектов качества, задачи – для отслеживания запланированных работ, требования QoS – для описания аспектов, не связанных с функциональностью, например, требований к безопасности, производительности и т. д.);

- с ним связана последовательность операций, описываемая посредством состояний и переходов. Например, возможны состояния “Opened”, “Resolved” и “Closed”;

- у него есть набор полей, например, поля Priority (приоритет), Status (состояние) и Iteration (итерация), которые можно задавать, опрашивать и использовать при создании отчетов.

Последовательность операций рабочего элемента представляет все возможные состояния рабочего элемента, а также переходы между ними. Каждое состояние обычно ассоциируется с ролью СУ ЖЦ. Например, когда тестировщик открывает в MSF Agile новую ошибку, соответствующий рабочий элемент переходит в состояние Active. Когда разработчик исправляет ошибку, состояние рабочего элемента меняется на Resolved. Когда тестировщик подтверждает исправление ошибки, состояние меняется на Closed. Далее приведены примеры последовательностей операций для двух самых распространенных типов рабочих элементов.

Настройка рабочих элементов. Существует несколько ситуаций, в которых может потребоваться изменение типов рабочих элементов MSF Agile или MSF CMMI:

- в рабочем элементе нет поля, которое необходимо в процессе разработки;

- последовательность операций рабочего элемента не соответствует схеме работы команды;

- необходим новый тип рабочего элемента.

Для разрешения этих проблем в СУ ЖЦ можно сделать следующее:

- добавить (или удалить) тип рабочих элементов;
- изменить поля существующих рабочих элементов;
- изменить состояния и переходы существующих рабочих элементов.

6.7. КОНТРОЛЬ ВЫПОЛНЕНИЯ РАБОТ И ОТЧЕТЫ

Отчеты предназначены для оперативного оценивания состояния проекта, качества разрабатываемого ПО, а также определения этапа, на котором находится проект. Отчеты формируются на основе данных из хранилища СУ ЖЦ и содержат показатели, поступающие от рабочих элементов, системы управления исходным кодом, тестирования и сборок. Например, с помощью отчетов можно выяснить темпы работы команды на основании ее фактической деятельности. Задача системы создания и отображения отчетов СУ ЖЦ – предоставление интегрированных данных по всем компонентам СУ ЖЦ, чтобы руководители проекта и члены команды могли оценить состояние проекта и предпринять соответствующие действия для его успешного выполнения.

Отчеты, доступные по умолчанию, определяются используемым шаблоном проекта, но существует также возможность создания собственных отчетов. Содержимое и использование каждого отчета из шаблона процесса объясняется в руководстве для этого шаблона. Azure DevOps Server основан на Microsoft SQL Server и использует SQL Server для хранения всех данных, связанных с рабочими элементами, атрибутами качества, тестированием, результатами тестирования и результатами сборок. Для объединения и анализа этих данных и создания отчетов в СУ ЖЦ используются службы SQL Server Analysis Services. Отчеты, созданные шаблоном процесса или

отдельными членами команды с помощью Microsoft Office Excel или Visual Studio 2005 Report Designer, доступны при помощи SQL Server 2005 Reporting Services и портала Share Point команды.

Возможности системы контроля версий. Рабочие элементы проекта могут быть разных версий в зависимости от версии продукта. Миграция рабочих элементов в другие подпроекты приводит к появлению новой версии. Копии элементов могут множиться в арифметической прогрессии. В случае, если параллельно ведутся несколько сложных проектов и много версий реализуется, то довольно сложно поддерживать порядок. Поэтому возникает задача контроля версий. СУ ЖЦ включает централизованную систему контроля версий, которая предоставляет следующие возможности.

1. Атомарные возвраты – изменения, вносимые в файлы, упаковываются в «набор изменений». Возврат файла в наборе изменений представляет собой неделимую транзакцию. Этим гарантируется согласованность базы кода. Эта транзакция тоже фиксируется, а метаданные сохраняются в БД проекта, и все это прозрачно отслеживается в соответствии с правами всеми действующими лицами. Примечания при возврате позволяют формировать метаданные о возврате. Возможность одновременного извлечения позволяет нескольким членам команды одновременно редактировать файл с последующим объединением изменений.

2. Ассоциация операций возврата с рабочими элементами, что предоставляет возможность отслеживать требования от начальной функции до задач и возврата в систему контроля версий кода, реализующего эту функцию. По сути это не что иное как целостное восприятие всех элементов, связанных между собой в цепочку: требования → задача → постановка задачи на создание кода → проектные решения по коду → собственно код → тесты к нему → сборка. Все это связано между собой ассоциативными связями.

3. Ветвление и объединение при разработке кода. Это характерная особенность системы контроля версий, потому что

есть разные ветви проекта. Они возникают, когда изменения вносятся в код, любое изменение создает новую ветвь. Поэтому все эти ветви дифференцированно рассматриваются, сохраняются, маркируются и объединяются в единый проект при разработке кода. Это диалектическая аномалия: с одной стороны, разделение проекта на ветви, с другой – рассмотрение всех ветвей дерева как единого проекта.

4. Наборы отложенных изменений позволяют создавать копии кода, не записывая их в главное хранилище системы контроля версий, чтобы лишний раз не загромождать это хранилище. Создаются копии кода, и таким образом решается две проблемы: 1) не загромождается хранилище; 2) выполняется предварительный анализ этих копий, и если они по каким-то причинам команду не устраивают по качеству, можно их дополнительно куда-то отправить, экономя таким образом критические системные ресурсы, которые для крупных проектов достаточно критичны.

5. Политики возврата, которые предоставляют возможность выполнить код для проверки допустимости возврата. Если возврат допустим, то тогда он реализуется. Если нет (например, если еще не накоплено достаточно много изменений), то тогда код не запускается, чтобы не загромождать репозитории лишний раз, не загружать сервер сборки. Сборки могут длиться в крупных проектах часы, дни. Важно принимать сбалансированное, компромиссное решение по этому поводу.

6. Прокси-сервер позволяет оптимизировать работу с региональными центрами разработки данных о ходе проекта.

7. УПРАВЛЕНИЕ РИСКАМИ ПРОГРАММНОГО ПРОЦЕССА

Процесс управления рисками выполняется с целью гарантировать, что каждый риск, идентифицированный в проектном окружении, оформляется документально, доводится до сведения участников и минимизируется соответствующим образом.

Управление рисками – неотъемлемая часть процесса разработки программного обеспечения. В современном мире, где ПО становится все более сложным и важным, эффективное управление рисками играет решающую роль в успехе проектов. Технологии проектирования ПО предоставляют инструменты и подходы, которые помогают управлять рисками на протяжении всего жизненного цикла разработки ПО. В данной главе рассматривается важность задачи управления рисками программного процесса и её взаимосвязь с технологиями разработки ПО.

Риски программного процесса могут быть определены как потенциальные события или обстоятельства, которые могут негативно повлиять на достижение целей проекта по разработке ПО, характеризующиеся некоторой величиной неопределенности. При этом сами события или обстоятельства правильнее относить к *рисковым факторам*.

Риск – степень неопределённости, сопровождающая во времени деятельность активного субъекта в ситуации, когда имеется возможность отклонения её последствий от предполагаемой цели.

Риск всегда характеризуется наличием неопределенности и является разновидностью неопределенности, когда возникает вероятность события и она может быть установлена.

Неопределенность – это неполнота или неточность информации об условиях реализации проекта, в том числе о связанных с ними затратах и результатах.

К факторам неопределенности относятся:

- неполное знание – неполнота или неточность информации о параметрах и обстоятельствах проекта, о ситуациях, требующих выбора оптимального решения; невозможность адекватного и точного учета всей, даже доступной информации; наличие вероятностных характеристик поведения среды проекта;

- факторы случайности – факторы, возникновение которых невозможно предусмотреть и спрогнозировать даже в вероятностной оценке;

- субъективные факторы противодействия – факторы, возникающие в ситуации взаимодействия партнеров, имеющих противоположные или несовпадающие интересы.

В программной инженерии риск – это любое событие, которое вероятно окажет неблагоприятное влияние на способность проекта достигнуть поставленных целей. Эти риски могут включать такие рисковые факторы, как технические проблемы, изменения требований, задержки в расписании, неудовлетворение пользовательских ожиданий и другие негативные последствия.

Риск – сочетание вероятности события и его последствий (ГОСТ Р 51897-2002 «Менеджмент риска. Термины и определения»).

Основные характеристики риска:

- непосредственная причина или источник – явление, обстоятельство, обуславливающее наступление риска;

- симптомы риска (триггеры) – указание на то, что событие риска произошло или вот-вот произойдет;

- последствия риска – проблема или возможность, которая может реализоваться в проекте в результате произошедшего риска;

- влияние риска – влияние реализовавшегося риска на возможность достижения целей проекта.

Рисковые факторы – это объективные обстоятельства с неопределенностью, которые могут иметь как положительное,

так и отрицательное влияние на проект. Как правило, устранить такие обстоятельства невозможно, однако возможно и необходимо их контролировать и в известной степени ими управлять. Это включает максимизацию вероятности наступления положительных и минимизацию вероятности наступления отрицательных по отношению к целям проекта событий.

Управление рисками – это совокупность методов анализа и нейтрализации факторов рисков, объединенных в систему планирования, мониторинга и корректирующих воздействий.

Понятие риска всегда связано с возможностью выбора того или иного варианта развития событий. Риск неразрывно связан с понятием альтернативности и корректно работает в определенных границах неопределенности (рис. 7.1).



Рис. 7.1. Схема границ управления рисками с позиций неопределенности

Управление рисками программного процесса направлено на идентификацию, анализ, планирование и контроль рисков для минимизации их влияния на проект. Это максимизация вероятности наступления положительных и минимизация вероятности наступления отрицательных по отношению к целям проекта событий.

7.1. ОСНОВНЫЕ РИСКИ ПРОГРАММНОГО ПРОЦЕССА

Современная программная инженерия сопряжена с высоким уровнем рисков. Для эффективного управления рисками необходимо умение идентифицировать их на каждом этапе жизненного цикла ПО.

Технологии проектирования программного обеспечения предоставляют в числе прочего набор методов, инструментов и подходов, которые помогают разработчикам эффективно управлять рисками программного процесса. Рассмотрим некоторые из ключевых технологий проектирования программного обеспечения.

1. Моделирование и анализ. Использование формальных моделей и методов анализа помогает разработчикам предвидеть потенциальные проблемы и риски в программном процессе. Моделирование позволяет выявить слабые места и недостатки в проектировании еще на ранних стадиях разработки.

2. Функции безопасности. Интеграция механизмов безопасности в архитектуру ПО помогает предотвратить уязвимости и риски, связанные с нарушением принципов информационной безопасности при эксплуатации ПО. Технологии, такие как Secure Software Development Lifecycle (SSDLC), помогают инженерам разрабатывать безопасные программные продукты и системы на протяжении всего жизненного цикла ПО.

3. Тестирование и верификация. Применение техник тестирования и верификации помогает обнаружить дефекты и проблемы в программном обеспечении, которые могут стать источником рисков. Автоматизированные тесты, модульное тестирование, интеграционное тестирование и другие методы обеспечивают раннее выявление проблем и большую уверенность в качестве продукта.

Управление рисками проекта можно разделить на три части:

- 1) идентификация, анализ и оценка рисков проекта;
- 2) определение мероприятий по управлению рисками;
- 3) контроль и мониторинг рисков проекта.

Но простого разделения на части процесса управления рисками недостаточно. Управление рисками имеет под собой некую цикличность, когда в зависимости от возникающих изменений требуется выполнять одни и те же процессы управления. Данный цикл процесса управления рисками изображен на рис. 7.2.



Рис. 7.2. Цикл процесса управления рисками

Изучение ГОСТ Р ИСО/МЭК 12207-99 «Информационная технология. Процессы жизненного цикла программных средств» выявило основные процессы разработки программного обеспечения (подробнее см. параграф 2.3):

- 1) выявление и анализ требований;
- 2) проектирование программного обеспечения;
- 3) программирование;
- 4) тестирование программного обеспечения.

Выявление и анализ требований играют важную роль в успехе проекта разработки ПО, так как большая часть ошибок возникает на ранних стадиях разработки. Это происходит потому, что каждая последующая фаза работы строится на результатах предыдущих. Например, проектирование основывается на требованиях, программирование – на основе проектных моделей, а тестирование – на написанном коде программы.

Двусмысленные требования. Одна из проблем заключается в том, что одно и то же требование может быть истолковано пользователями по-разному. Также возможно неоднозначное понимание требования разными читателями.

Проектирование. Риски, связанные с проектированием, играют важную роль в разработке программного обеспечения и требуют особого внимания, так как основные технические вопросы решаются именно на этом этапе.

Сложность архитектуры ПО. Архитекторы ПО иногда не следуют правилу «не создавать лишних сущностей». Однако именно простота, понятность и концептуальная целостность обеспечивают эффективную реализацию системы.

Неудобный пользовательский интерфейс. Разработка удобного пользовательского интерфейса – важная часть процесса создания ПО. Пользовательский интерфейс представляет собой точку взаимодействия между человеком и программой, которая часто обладает сложной функциональностью. От удобства и понятности интерфейса зависит мнение пользователей о программе в целом, и они часто принимают решение о его использовании исходя из этого. Таким образом, успех продукта во многом зависит от того, насколько удобным будет разработанный пользовательский интерфейс.

Неправильная структура базы данных. Проектирование базы данных требует особого внимания и ответственности, так как ошибки на этом этапе могут иметь серьезные последствия. К возможным проблемам на этапе проектирования базы данных относятся:

- неправильная схема базы данных, не соответствующая предметной области;

- несоответствие аппаратным ограничениям;
- сложность работы с базой данных;
- невозможность поддержки и сопровождения.

Неоптимальный выбор структур данных. Выбор структур данных влияет на эффективность алгоритмов и, соответственно, на производительность ПО.

Неоптимальный выбор языка программирования. При разработке ПО необходимо учесть ряд факторов при выборе языка программирования, таких как целевая платформа, гибкость языка, время реализации, производительность, сопровождение, предметная область, использование библиотек и опыт разработчиков.

Программирование. Основные риски на этапе программирования:

- изобретение «велосипеда» – написание кода без использования возможностей языка программирования и существующих библиотек;
- нечитаемый код – трудности в изменении и сопровождении ПО, вызванные нечитаемым кодом;
- создание программных закладок¹ – неконтролируемое использование программных закладок может привести к несанкционированным действиям и угрозам безопасности;
- нерегулярное резервное копирование кода – неправильное или нерегулярное создание резервных копий кода может привести к потере данных и серьезным проблемам восстановления.

Тестирование. На этапе тестирования возможны следующие риски:

- неэффективный выбор методов тестирования, что может привести к неполному или недостаточному тестированию ПО;

¹ Программные закладки – это функциональные объекты, внедренные в программное обеспечение, которые могут инициировать выполнение неописанных функций при определенных условиях.

- неупорядоченные требования, которые не позволяют определить приоритеты тестирования;
- неправильное использование протоколов тестирования, что может привести к пропуску определенных видов тестов или неполному охвату функциональности ПО.

Управление рисками в процессе разработки ПО включает идентификацию, анализ и управление рисками на каждом этапе разработки. Это может включать применение стратегий снижения рисков, использование стандартов и лучших практик, обучение и подготовку персонала, а также систематический мониторинг и оценку рисков на протяжении всего проекта.

7.2. ПРОЦЕСС УПРАВЛЕНИЯ РИСКАМИ РАЗРАБОТКИ ПО

Управление рисками программного процесса включает несколько ключевых этапов.

1. Идентификация рисков. Это предварительный этап риск-менеджмента, на котором проводится анализ проекта для выявления потенциальных рисков. Идентификация рисков должна быть систематической и включать в себя участие всех заинтересованных сторон.

2. Анализ рисков. На этом этапе рассматривается вероятность возникновения рисков и их влияние на проект. Риски классифицируются по степени важности и приоритету для последующего планирования и управления.

3. Планирование. На данном этапе разрабатываются стратегии управления рисками. Это включает определение мер по предотвращению рисков, снижению их влияния и контролю исполнения указанных мер.

4. Мониторинг. После планирования рисков важно осуществлять мониторинг и контроль изменения рискового фона. Мониторинг осуществляется непрерывно (в идеале – в режиме реального времени) на протяжении всего проекта. Это позволяет своевременно выявлять новые риски и переоценивать критичность выявленных рисков, оценивать эффективность

применяемых стратегий и корректировать планы, если необходимо.

Преимущества эффективного управления рисками.

1. Сокращение вероятности срыва проекта. Адекватное управление рисками помогает предотвратить срыв проекта или минимизировать его проблемность. Это способствует соблюдению сроков, бюджета и качества разработки.

2. Улучшение качества продукта. Идентификация и управление рисками способствуют раннему обнаружению проблем в проекте и их эффективному решению. Это приводит к повышению качества продукта и удовлетворению потребностей пользователей.

3. Оптимизация ресурсов. Эффективное управление рисками позволяет оптимизировать распределение ресурсов проекта. Средства и усилия могут быть сосредоточены на наиболее критических аспектах разработки, что позволяет достичь лучших результатов.

4. Улучшение коммуникации и сотрудничества. Управление рисками способствует улучшению коммуникации и сотрудничества внутри команды разработчиков. Раннее обнаружение и обсуждение рисков позволяет разработчикам работать вместе над их решением, что способствует более эффективному и гармоничному процессу разработки.

Для эффективного управления рисками программного процесса необходимо интегрировать его в процесс разработки ПО. Рассмотрим несколько ключевых аспектов интеграции.

1. Раннее управление рисками. Управление рисками должно начинаться на ранних стадиях разработки. Важно провести анализ потенциальных рисков еще на этапе определения требований и планирования проекта. Это позволяет принять соответствующие меры для предотвращения рисков и учитывает их во время проектирования.

2. Вовлечение всех заинтересованных сторон. Управление рисками должно быть задействовано во всех этапах разработки ПО. Разработчики, менеджеры проекта, заказчики и другие заинтересованные стороны должны активно участвовать в

идентификации и анализе рисков. Это способствует получению полной и достоверной информации о потенциальных рисках, параметрах их критичности и разработке эффективных стратегий риск-менеджмента.

3. Непрерывный мониторинг рискового фона. Управление рисками не является статическим процессом. Риски могут меняться и возникать на протяжении всего проекта. Поэтому важно осуществлять непрерывный мониторинг и анализ рисков. Это позволяет своевременно реагировать на новые риски и принимать соответствующие меры.

4. Обучение и обмен опытом. Обучение разработчиков и профессионалов по управлению рисками является важным аспектом успешного внедрения управления рисками в процесс разработки ПО. Также важно осуществлять обмен опытом и передачу знаний между командами разработчиков. Это позволяет извлекать уроки из предыдущих проектов и применять опыт для более эффективного управления рисками.

7.3. ТЕХНОЛОГИЯ УПРАВЛЕНИЯ РИСКАМИ ПРОГРАММНОГО ПРОЦЕССА

Для успешного управления рисками программного процесса существует ряд методик, моделей и инструментов, обеспечивающих управление рисками, а также стандартов и методологий, определяющих саму деятельность риск-менеджмента.

Можно отметить следующие **стандарты и методологии**, которые могут быть применены для управления рисками программного процесса.

1. ISO 31000:2018 «Менеджмент рисков» – международный стандарт, описывающий принципы и руководство по управлению рисками безотносительно к отрасли или сектору. Он может быть применен для управления рисками программного процесса, предоставляя общий фреймворк и методы.

2. Project Management Body of Knowledge (PMBOK) – это руководство по управлению проектами, которое включает в себя

процессы управления рисками. Применение PMBOK может быть полезно для интеграции управления рисками в общий процесс управления проектом разработки программного обеспечения.

3. Agile-методологии разработки ПО, такие как MSF, Scrum и Kanban, предоставляют гибкие подходы к управлению проектами и рисками. Они акцентируют внимание на постоянном взаимодействии с заказчиком, быстрой адаптации к изменениям и управлении рисками на протяжении всего процесса разработки.

4. Модель CMMI представляет собой модель для оценки и улучшения процессов разработки ПО (подробнее см. параграф 2.2). Она включает также аспекты управления рисками, позволяет оценивать и улучшать процессы управления рисками программного процесса в организации-разработчике ПО.

Рассмотренные стандарты и методологии регламентируют применение инструментов и методик риск-менеджмента. Рассмотрим некоторые **инструменты и методики**, которые могут быть эффективны в программном процессе.

1. Матрица рисков – это инструмент, который позволяет классифицировать риски по их вероятности и существенности последствий. Он помогает определить приоритеты и разработать соответствующие стратегии управления рисками.

2. SWOT-анализ (анализ сильных и слабых сторон, возможностей и угроз) может быть применен на этапе идентификации рисков программного процесса. Он помогает выявить внутренние и внешние факторы, которые могут повлиять на успешность проекта.

3. Анализ дерева решений позволяет оценить различные альтернативы и выбрать наиболее оптимальные варианты управления рисками. Он основан на построении дерева возможных событий и оценке их вероятности и воздействия.

4. Инструменты моделирования и симуляции позволяют провести виртуальные эксперименты с различными сценариями и альтернативами управления рисками. Это позволяет оценить эффективность стратегий управления рисками и принять более обоснованные решения.

5. Специализированные системы управления рисками предоставляют набор инструментов и функциональность для идентификации, анализа, планирования и контроля рисков. Эти системы работают на всех этапах ЖЦ ПО и позволяют централизованно управлять рисками и обеспечивать прозрачность процесса риск-менеджмента.

6. Автоматизированные инструменты тестирования помогают выявлять уязвимости и проблемы, собственно, в самом ПО, которые могут стать источником рисков проекта. Они позволяют проводить обширное тестирование и анализ кода, а также оценивать безопасность и надежность ПО.

Управление рисками программного процесса должно осуществляться на различных фазах разработки. Рассмотрим **ключевые фазы**, где управление рисками играет важную роль.

1. Фаза планирования. На этой фазе проводится анализ рисков, разрабатывается план управления рисками и определяются стратегии и меры для снижения рисков. Также определяются роли и ответственности в рамках управления рисками.

2. Фаза разработки. В процессе разработки проводится постоянный мониторинг и анализ рисков. Разработчики должны быть вовлечены в идентификацию и решение рисков, а также применять соответствующие техники и инструменты для снижения рисков.

3. Фаза тестирования и отладки. На этой фазе проводится тестирование ПО для выявления потенциальных проблем и рисков. Результаты тестирования помогают идентифицировать и оценивать риски, связанные с неполадками и некорректной работой ПО.

4. Фазы внедрения и эксплуатации. При внедрении и эксплуатации ПО могут возникать риски, связанные с интеграцией системы, безопасностью и производительностью. Управление рисками на этой фазе включает мониторинг функционирования ПО, регулярное обновление и обслуживание, а также реагирование на новые риски.

5. Фаза поддержки и сопровождения. В этой фазе управление рисками связано с обеспечением непрерывной работы и поддержкой ПО. Риски могут возникать в виде сбоев ПО, несоответствия требованиям клиентов или изменений в окружающей среде. Управление рисками включает в себя мониторинг и анализ проблем, разработку и реализацию планов риск-менеджмента, обеспечение своевременной поддержки пользователей и обновление ПО.

7.4. РИСКООБРАЗУЮЩИЕ ФАКТОРЫ ПРОГРАММНОГО ПРОЦЕССА

Учитывая логическую взаимосвязь между целями проекта и возможными рисками, можно предположить, что при разработке программного проекта могут возникнуть четыре типа (категории) рисков:

- срыв плановых сроков проекта;
- превышение стоимости (бюджета) проекта;
- критические отклонения по составу и содержанию проекта (невыполнение функциональных требований);
- критическое отклонение по показателям качества проекта (невыполнение нефункциональных требований).

Появление каждого из рисков возможно при наличии причин (процессов или явлений), способствующих его возникновению и поясняющих, почему наступление риска неизбежно. Такие явления принято называть *рискообразующими факторами*.

Рискообразующий фактор – это совокупность процессов или явлений, определяющих содержание и характерные черты риска.

При этом возникает понятие «источник риска», который определяется как состояние рискообразующего фактора, ведущее к возникновению риска.

Для стандартизации и унификации терминологии рискообразующих факторов, оценки их влияния на конкретные цели и фазы программного процесса необходима систематизация

и классификация факторов по определенным признакам. В данном случае для классификации факторов риска используется иерархический метод классификации, при котором множество рискообразующих факторов последовательно, в соответствии с выбранными основаниями классификации, разбивается на подмножества (рис. 7.3).



Рис. 7.3. Классификация факторов риска программного проекта

На первом уровне классификатора в качестве основания классификации используется модель жизненного цикла программного проекта: инициация – разработка – продвижение – внедрение. На втором уровне для каждого этапа ЖЦ ПО можно выделить внешние и внутренние факторы. Внешние факторы – это события, которые лежат за пределами контроля и влияния команды проекта. Внутренние факторы (специфичные для конкретной компании) определяют способность самой организации успешно реализовать проект. На третьем уровне

проявление внешних факторов обуславливается как политикой государства в отношении бизнеса малых ИТ-компаний, так и различными ситуациями на продуктовом, финансовом рынках и рынке труда. Классификатор внутренних факторов определяется составом системной модели деятельности: средствами деятельности, предметами деятельности, кадрами, технологией. Четвертый уровень представляет собой набор первичных факторов риска. При этом допускается возможность принадлежности одного и того же фактора разным основаниям классификации.

Внутренние факторы риска классифицируют по продукту, персоналу и технологии реализации продукта.

1. Продукт:

- нереальные сроки выхода на планируемые объемы продаж;
- ошибки в расчетах трудоемкости и финансовых затрат на разработку и продвижение программного продукта;
- появление «забытых» работ.

2. Персонал:

- отсутствие опыта, необходимого для разработки и реализации программы;
- саботаж отдельных членов команды;
- недостатки во внутренней организации работ;
- неумение работать в реальном времени.

3. Технологии реализации продукта:

- ошибки при выборе программно-аппаратной платформы;
- ошибки выбора каналов и инструментов продвижения;
- отсутствие эффективного взаимодействия с потенциальными пользователями.

Внешние факторы риска: государство, финансовый рынок, рынок труда, продуктовый рынок (потребители), продуктовый рынок (конкуренты).

1. Государство:

- изменение нормативно-правовых механизмов ведения бизнеса в ИТ-отрасли;

- отсутствие устоявшейся законодательческой практики по защите авторских и имущественных прав ПП.

2. Финансовый рынок:

- колебания курса валют;
- изменение ставок по кредитам.

3. Рынок труда:

- отсутствие специалистов требуемой квалификации.

4. Продуктовый рынок (потребители):

- несоответствие функциональных характеристик ПП потребностям потребителей;

- несовместимость предлагаемого продукта с программно-аппаратной средой потребителей;

- несоответствие рыночной цены ПП возможностям потенциальных потребителей;

- низкий уровень подготовки пользователей у потенциальных потребителей ПП.

5. Продуктовый рынок (партнеры, конкуренты):

- появление на рынке новых аналогичных продуктов;
- непредсказуемое поведение конкурентов;
- пиратское распространение копий ПП.

7.5. ИДЕНТИФИКАЦИЯ РИСКОВ ПРОГРАММНОГО ПРОЦЕССА

Идентификация рисков – это первый этап риск-менеджмента, базирующийся на обследовании объекта управления. Выполняется в отношении каждого риска.

Идентификация риска – процедура, позволяющая выявить и коллективно обсудить возможность проявления риска и рискообразующих факторов, способных повлиять на цели проекта, документально описать результаты в виде логически увязанных характеристик.

Идентификация риска – процесс нахождения, составления перечня и описания элементов риска. Элементы риска могут включать в себя источники, события и их последствия.

Идентификация риска дает возможность любому члену команды проекта идентифицировать риск в проекте. При этом инициатор риска идентифицирует риск, присущий конкретному аспекту проекта (например, содержанию, результатам, срокам или ресурсам), после чего заполняет бланк идентификации риска и направляет его менеджеру проекта.

Основные источники сбора данных о рисках:

- архивы предыдущих проектов (база знаний организации);
- информация из открытых источников;
- исследовательские и научные работы в данной области;
- контрольные списки рисков проектов в данной области.

Описание факторов риска следует проводить на естественном языке по следующей схеме:

1) условия возникновения (содержит описание причины, которая может сделать проект убыточным);

2) последствия проявления (описывает нежелательную ситуацию при наступлении рискообразующего фактора, которую следует избегать);

3) влияние на цели программы продвижения (отражает негативные изменения характеристик проекта в плане его продвижения потребителю).

Формат описаний рисков должен быть последовательным для обеспечения четкого и недвусмысленного понимания каждого риска с целью поддержки результативного анализа и разработки плана реагирования. Кроме того, описание рисков должно поддерживать возможность сравнивать относительное воздействие на проект одного риска с относительным воздействием других рисков. Пример идентификации риска при выводе ПП на рынок с описанием факторов представлен в табл. 7.1.

Рассмотрим риски, которые можно идентифицировать на разных этапах ЖЦ ПО. На ранних этапах разработки, в процессе выявления и анализа требований могут появляться двусмысленные требования. Пользователь может интерпретировать одно и то же требование по-разному, либо у

нескольких человек возникает разное понимание того, что означает требование.

Таблица 7.1

Фрагмент описания схемы рискообразующих факторов

Факторы риска	Описание фактора		
	Условие возникновения	Последствия проявления	Воздействие на цели программы продвижения
Изменение экономической ситуации при выводе ПП на рынок	Экономический кризис	Изменение платежеспособности потребителей	Сокращение объемов продаж
Появление новых аналогичных продуктов	Выход на рынок новых аналогичных продуктов	Усиление конкуренции	Сокращение объемов продаж
Ошибки выбора каналов и инструментов коммуникаций	Снижение необходимого уровня информирования целевой аудитории	Несоответствие плановых и фактических показателей результативности программы продвижения	Сокращение объемов продаж

Риски на этапе проектирования играют важную роль в процессе разработки ПО и требуют особого внимания, так как основные технические вопросы решаются именно на этом этапе, на котором выделяются следующие риски:

1. Сложность архитектуры программного обеспечения. Простота, понятность и единая концептуальная целостность обеспечивают эффективную реализацию системы. Однако есть риск, что разработчик архитектуры ПО неоправданно усложнит проект.

2. Неудобный пользовательский интерфейс. Разработка пользовательского интерфейса – часть любого проекта,

связанного с созданием ПО. Интерфейс пользователя является точкой взаимодействия человека и программы, зачастую имеющей сложную функциональность. Именно по интерфейсу пользователь судит о программе в целом. Более того, часто решение об использовании ПО пользователь принимает по тому, насколько ему удобен и понятен интерфейс. Следовательно, от того, насколько удобным будет разработанный интерфейс пользователя, будет зависеть и успех продукта.

3. Неправильная структура базы данных. При разработке ПО проектирование базы данных требует особого внимания и ответственности, так как стоимость допущенных на этом этапе ошибок особенно велика. К проблемам, которые могут произойти на этапе проектирования базы данных, относятся:

- некорректность схемы базы данных по отношению к предметной области;
- несоответствие аппаратным ограничениям;
- несоответствие типов данных полей БД и переменных программ;
- сложность и неудобство работы с базой данных;
- невозможность поддержки и сопровождения.

4. Неоптимальный выбор структур данных. Структуры данных влияют на эффективность алгоритмов и, следовательно, на производительность ПО.

5. Неоптимальный выбор языка программирования. При разработке ПО имеется широкий выбор языков программирования. Для того чтобы выбор языка оказал благоприятное влияние на реализацию системы, необходимо учитывать следующие факторы:

- целевая платформа;
- гибкость языка;
- время реализации;
- производительность;
- сопровождение программного обеспечения;
- предметная область разрабатываемой системы;
- необходимость в использовании библиотек;
- опыт разработчиков.

Риски на этапе кодирования также разнообразны и характеризуют качество реализации требований и проектных решений.

6. Неиспользование актуальных возможностей языка программирования. Написание кода без использования возможностей языка и существующих библиотек. Это делает код костным и хрупким, понижает модифицируемость, затрудняет переносимость и сопровождаемость ПО.

7. Нечитаемый код. Последствия этого риска – низкая понимаемость, трудность в модификации, расширении и сопровождении ПО. Чаще всего такой код оказывается нежизнеспособным и ЖЦ такого ПО завершается досрочно.

8. Создание программных закладок – весьма неприятный фактор, который может иметь тяжелые последствия для ПО.

На этапе разработки программистам достаточно легко внедрить в код программную закладку, которые впоследствии могут быть использованы злоумышленниками и обнаружены только после активации.

9. Нерегулярное резервное копирование кода. Последствие этого риска очень масштабно. На этапе тестирования продукта отметим следующие риски:

- неэффективный выбор методов тестирования;
- требования не ранжированы по приоритетам;
- не используются протоколы тестирования.

7.6. СТРАТЕГИИ УПРАВЛЕНИЯ РИСКАМИ

Процессы планирования мероприятий по реагированию на риски предполагают выбор стратегии по снижению угроз и разработку планов мероприятий по реализации стратегий. Возможны четыре вида таких стратегий: уклонение от риска, передача риска, снижение риска, принятие риска.

Уклонение от риска предполагает разработку комплекса мероприятий по нейтрализации критических рискообразующих факторов, т. е. изменение плана управления проектом таким образом, чтобы исключить влияние негативных факторов на цели

проекта или скорректировать целевые показатели, находящиеся под угрозой, например, отказаться от реализации рискованного функционального требования.

Передача риска подразумевает переложение негативных последствий от проявления рискообразующего фактора на третью сторону (но риск при этом остается), например, заказать разработку рискованного компонента «на стороне».

Снижение риска предполагает понижение вероятности и/или последствий негативного проявления рискообразующего фактора до приемлемых пределов, например, увеличить сроки выполнения проекта, понизить значения ряда показателей качества ПО. Принятие предупредительных мер по снижению вероятности наступления фактора или его последствий зачастую оказывается более эффективным, нежели действия по устранению негативных последствий, предпринимаемые после наступления события.

Принятие риска предполагается в тех случаях, когда избежать проявления рискообразующих факторов маловероятно и команда проекта не нашла эффективных мероприятий реагирования на риски. Реализация данной стратегии возможна в двух вариантах: активном либо пассивном. Пассивное принятие данной стратегии не предполагает проведения каких-либо предупредительных мероприятий, оставляя команде проекта право действовать по собственному усмотрению в случае наступления негативных событий. Наиболее распространенной формой активного принятия данной стратегии является создание резерва на непредвиденные обстоятельства в виде возможности привлечения дополнительных финансовых и/или трудовых ресурсов либо корректировки сроков реализации проекта.

Приведем примеры мероприятий, направленных на снижение ряда рискообразующих факторов:

- изменение стоимости и сроков реализации проекта при каждом добавлении или корректировке требований;
- согласование с заказчиком подробного перечня требований и включение этого списка в контракт на разработку ПО;

- использование моделей ЖЦ, позволяющих периодическое уточнение требований;

- введение буферных работ с соответствующими ресурсами и длительностью выполнения.

Применение управления рисками в различных фазах программного процесса минимизирует потенциальные угрозы и проблемы, а также повышает вероятность успешного завершения проекта. Это обеспечивает более эффективное планирование, управление ресурсами и снижение возможных финансовых потерь. Кроме того, управление рисками способствует улучшению качества программного обеспечения и повышению удовлетворенности клиентов.

Управление рисками программного процесса в контексте технологий проектирования ПО – неотъемлемая часть успешной разработки программных продуктов. Риски могут возникать на каждом этапе разработки, и их негативное влияние может привести к задержкам, проблемам с качеством и бюджетом проекта.

Осознание рисков и активное управление ими позволяют предотвратить или минимизировать потенциальные проблемы, связанные с программным обеспечением. Важно проводить раннюю и систематическую идентификацию рисков, их анализ и планирование мер по снижению рисков.

Интеграция управления рисками в процесс разработки программного обеспечения, применение соответствующих стандартов, методологий и инструментов, а также постоянный мониторинг и анализ рисков способствуют повышению эффективности и успешному завершению проектов.

Управление рисками программного процесса позволяет не только минимизировать отрицательные последствия возможных проблем, но также создает возможности для инноваций, улучшений и достижения высокого качества программного обеспечения, способствует достижению поставленных целей проекта, улучшению управляемости и уверенности заказчика, а также повышению конкурентоспособности организации на рынке разработки ПО.

7.7. МЕТОДИКА РИСК-МЕНЕДЖМЕНТА ПРОЕКТА РАЗРАБОТКИ ПО

При разработке реального проекта программного продукта важно понимать, насколько разрабатываемые и реализуемые решения на объекте управления подвержены рискам различного вида и/или их порождают. Для этого, в первую очередь, необходимо изучить процесс управления рисками, который в них реализуется, по крайней мере, опосредованно (например, в форме функционирования системы менеджмента качества). По результатам данного исследования необходимо предложить решения по обеспечению культуры риск-менеджмента на объекте управления в контексте внедрения и риск-устойчивой реализации проектируемого программного решения. Для этого необходимо:

1. Идентифицировать потенциальные риски для решения поставленной задачи и применения полученных результатов для приведения объекта в заданное (нормальное) состояние. При этом можно разделить риски, например, на три категории, представленные в табл. 7.1: риски разработки, риски реализации и риски внедрения разработанных программных решений. Рекомендуются выявить 3–5 рисков каждой из трех категорий, актуальных для объекта управления. Так, примеры рисков событий показаны в табл. 7.2. Описание рисков, их возможных последствий и факторов возникновения следует свести в таблицу, аналогичную табл. 7.3.

2. Установить характеристики (предпочтительно – количественные) шкал измерения основных параметров рисков событий, а именно вероятности их возникновения и степени существенности последствий их реализации, например, это показано в табл. 7.4, 7.5.

3. Используя установленные шкалы, провести оценивание рисков посредством:

- оценивания вероятности возникновения риска;
- оценивания степени существенности последствий реализации риска.

Оценивание риска – процесс присвоения значений вероятности и последствий риска.

Таблица 7.2

Пример классификационной модели рисков

Класс риска	Подкласс риска	Пример рискового события
Риски, возникающие на этапе разработки программно-информационных решений	–	1. Увеличение нагрузки на работников объекта на этапе его обследования 2. Некорректные требования к ПО, сформулированные заказчиком 3. Недостаточное финансирование проекта
Риски, возникающие на этапе реализации и внедрения разработанных программно-информационных решений	Риски нарушения функционала реализации разработанных программно-информационных решений	1. Загрузка недостоверных данных для анализа либо их отсутствие 2. Нестабильная работа Интернета 3. Недостаточность вычислительной мощности
	Риски применения на объекте разработанных программно-информационных решений	1. Слабая синхронизация бизнес-процессов и разработанных решений 2. Недостаточная квалификация пользователей 3. Отказ от реализации управленческих решений

Таблица 7.3

Описание рисковых событий

Наименование риска	Факторы возникновения риска	Последствия реализации риска

Таблица 7.4

**Шкала оценивания вероятности возникновения рисковог
события**

Оценка вероятности возникновения риска	Характеристика оценки

Таблица 7.5

**Шкала оценивания степени существенности последствий
рискового события**

Оценка существенности последствий реализации риска	Характеристика оценки

Результаты оценивания рекомендуется представить в табличном (табл. 7.6) и графическом форматах (карта рисков, рис. 7.3).

Таблица 7.6

Оценка рисковых событий

Наименование риска	Вероятность возникновения риска	Степень существенности последствий реализации риска

На рис. 7.4 представлен частный пример карты рисков. На данной карте рисков вероятность (частота) отображается по горизонтальной оси, а существенность последствий – по вертикальной оси. В этом случае вероятность появления риска увеличивается слева направо по горизонтальной оси, а существенность последствий риска увеличивается сверху вниз по направлению вертикальной оси.

Штриховкой в клетку на карте выделены области критических рисков – рисков в «красной зоне»: риски, у которых высокая вероятность возникновения и значительная

существенность последствий реализации риска. На эти риски необходимо обратить внимание в первую очередь: либо смягчить последствия, либо исключить возможность их возникновения.

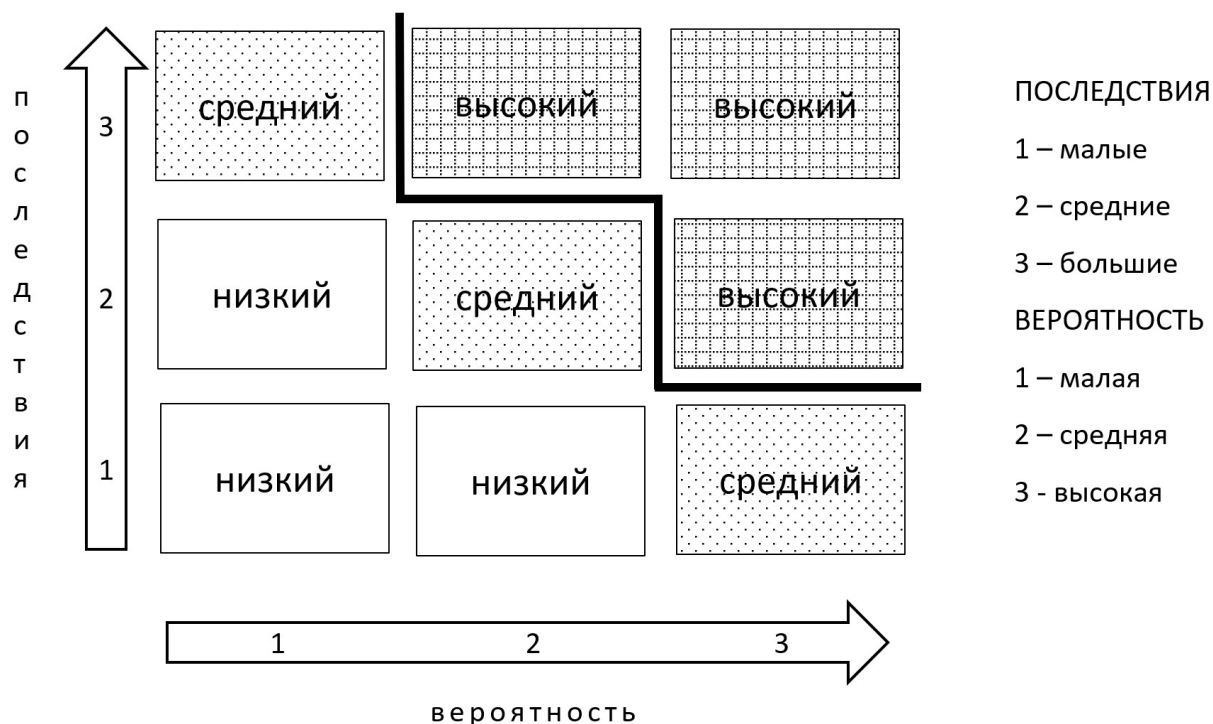


Рис. 7.4. Частный пример карты рисков

Черная жирная линия над областью допустимых рисков – это граница толерантности или критическая граница терпимости к риску. Риски, лежащие на границе толерантности, необходимо смещать в область управляемых рисков, чтобы они не перешли в область критических рисков.

Точечной штриховкой выделены риски границы терпимости – риски в «жёлтой зоне», у которых средняя частота появления и «терпимые» последствия реализации рискового события.

Зоны без штриховки – риски, которые редко реализуются и имеют незначительные последствия – риски в «зелёной зоне». С этими рисками работают в последнюю очередь, так как они не несут больших потерь и являются управляемыми в рабочем режиме.

4. На основе составленной карты рисков необходимо провести их ранжирование. Наиболее высокий ранг присваивается рискам из критической области. Далее следуют риски, лежащие в допустимой области, и, наконец, риски, которые являются управляемыми в рабочем режиме.

5. На основании полученного материала исследования рискованного фона необходимо сделать заключение о подготовке и реализации мероприятий, необходимых для приведения объекта управления к состоянию умеренного рискованного профиля, находящегося ниже линии толерантности. Заключение должно быть развернутым и содержать вывод о целесообразности применения полученных в ВКР решений, прогноз изменения карты рисков в перспективе, а также решения по возможным мероприятиям, улучшающий рискованную профилем объекта управления. Предлагаемые мероприятия целесообразно представить в виде табл. 7.7.

Таблица 7.7

Мероприятия для улучшения рискованного профиля объекта
управления

Наименование риска	Мероприятия по управлению риском

Особое внимание при анализе рисков следует уделить так называемым системным рискам как возможным рискам нарушения функционала системы либо появления системных дисфункций предложенных решений. Как правило, такие риски наиболее критичны, поскольку приводят к частичной либо полной потере эффективности и результативности. Также следует тщательно проанализировать область применения предложенных решений. При этом необходимо выявить риски, связанные с низкой эффективностью принятия решений и/или высокой субъективностью результатов реализации принятых управленческих решений.

В заключение остается лишь отметить, что реализация качественной и высокоэффективной системы риск-менеджмента

– задача сложная и требует применения нетривиального математического аппарата и построения математической модели оценки, мониторинга и регулирования рискового профиля объекта управления.

7.8. РОЛЕВАЯ МОДЕЛЬ РИСК-МЕНЕДЖМЕНТА

Ролевая модель риск-менеджмента проекта разработки ПО включает в себя следующие роли:

- инициатор риска;
- менеджера проекта;
- группа по анализу проекта;
- команда проекта.

Инициатор риска идентифицирует риск и формально информирует менеджера проекта о риске. Инициатор риска отвечает за следующие задачи:

- идентификацию рисков в проекте;
- документальное оформление риска (посредством заполнения бланка (листка) идентификации риска);
- отправку бланка идентификации риска менеджеру проекта на рассмотрение.

Менеджер проекта получает каждый бланк идентификации риска, вносит запись и выполняет мониторинг процесса работы со всеми рисками в проекте. Менеджер проекта отвечает за следующие задачи:

- получение всех бланков идентификации риска и определение подверженности проекта риску;
- внесение всех рисков в реестр рисков;
- отправка всех рисков группе по анализу проекта;
- информирование заинтересованных сторон обо всех решениях, принятых группой по анализу проекта;
- мониторинг процесса выполнения всех мероприятий по минимизации рисков.

Группа по анализу проекта подтверждает вероятность возникновения и влияния и планирует проведение мероприятий

по минимизации рисков при необходимости. Группа по анализу проекта отвечает за следующие задачи:

- регулярный анализ всех рисков в реестре рисков;
- идентификацию запросов на изменение, которые необходимы для минимизации выявленных рисков;
- назначение мероприятий по минимизации рисков;
- закрытие рисков, которые больше не оказывают влияния на проект.

Команда проекта выполняет все мероприятия по минимизации рисков, запланированных группой по анализу проекта.

ЗАКЛЮЧЕНИЕ

Учебное пособие отражает жизненный цикл разработки ПО методами программной инженерии. Рассмотренные методы и модели ориентированы главным образом на эффективное и риск-устойчивое управление программным процессом. Проработка данных методов и моделей формирует у студентов представление о базовых принципах производства ПО, а также средствах реализации этих методов и моделей на практике.

Материал пособия сформирован на основе учебных курсов «Программная инженерия», «Технология проектирования ПО» и «Тестирование ПО», реализуемых в КубГУ, читаемых авторами на протяжении последних лет. Также при подготовке учебного пособия использован опыт авторов в практической разработке ПО на производстве. Сочетание теоретической базы по указанным дисциплинам с опытом авторов в применении данных знаний в учебном процессе в разных образовательных организациях позволило создать учебное пособие, ориентирующее студентов на реализацию полного жизненного цикла программного процесса, включая качественную его организацию и управление рисками проекта. Опыт авторов в области создания и применения ПО разного назначения, основанный на знаниях, позволяет использовать при создании ПО методы и подходы информатики, программной и системной инженерии, а также бизнес-инжиниринга, получая таким образом качественные программные продукты и системы.

Вместе с тем необходимо помнить, что программная инженерия продолжает развиваться в соответствии с потребностями рынка и пользователей, с появлением новых технологий и возможностей. За более чем полвека существования программная инженерия претерпела значительные изменения и достигла высокого уровня зрелости. Авторы надеются, что благодарные читатели последуют за своими предшественниками и в свое время также внесут свой вклад в столь важную и необходимую для цифровой трансформации учебную и научную дисциплину – программную инженерию.

РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

Бейзер Б. Тестирование черного ящика. Технология функционального тестирования программного обеспечения и систем. СПб.: Питер, 2004.

Блэк Р. Ключевые процессы тестирования. М.: Лори, 2006.

Буч Г. Объектно-ориентированный анализ и проектирование с примерами приложений на С++. М.: Бином, 1999.

Вигерс К.И. Разработка требований к программному обеспечению. М.: Русская редакция, 2004.

Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. СПб.: Питер, 2001.

Гвоздева В. А. Введение в специальность программиста: учебник. 2-е изд., испр. и доп. М.: ИД «ФОРУМ»; ИНФРА-М, 2007.

Гецци К., Джазайери М., Мандриоли Д. Основы инженерии программного обеспечения. СПб.: БХВ-Петербург, 2005.

Грехем И. Объектно-ориентированные методы. Принципы и практика. М.: Вильямс, 2004.

Дастин Э., Рэшка Д., Пол Д. Автоматизированное тестирование программного обеспечения. Внедрение, управление и эксплуатация. М.: Лори, 2003.

Дмитриева М.А., Крылов А.А. Психология труда и инженерная психология. М.: Дельфа, 2005.

Доррер Г. А. Методология программной инженерии: учеб. пособие. Красноярск: Сибирский государственный университет науки и технологий имени академика М.Ф. Решетнева, 2021.

Ехлаков Ю.П. Управление программными проектами: учебник. Томск: Томский государственный университет систем управления и радиоэлектроники, 2015.

Канер С., Фолк Д., Нгуен Е. Тестирование программного обеспечения. М.: ДиаСофт, 2001.

Кнут Д. Искусство программирования. 3-е изд., М.: Изд. дом «Вильямс», 2000. Т. 1.

Кознов Д. В. Введение в программную инженерию: учеб. пособие. 3-е изд. М.: Интернет-университет информационных технологий (ИНТУИТ), 2020.

Кронрод А. С. Беседы о программировании. 2-е изд., стереотип. М.: УРСС, 2004.

Кьюу Дж., Джеанини М. Объектно-ориентированное программирование.: учебный курс. СПб. Питер, 2005.

Леоненков А.В. Язык UML 2 в анализе и проектировании программных систем и бизнес-процессов. М.: ИНТУИТ.ру, 2009.

Леффингуэлл Д., Уидриг Д. Принципы работы с требованиями к программному обеспечению. Унифицированный подход. М.: Вильямс, 2002.

Липаев В.В. Программная инженерия сложных заказных программных продуктов: учеб. пособие. М.: МАКС Пресс, 2014.

Липаев В.В. Программная инженерия. Методологические основы: учебник. М.: ТЕИС, 2006.

Мартин Р. Чистый код: создание, анализ и рефакторинг. СПб.: Библиотека программиста. Питер, 2013.

Моделирование бизнес-процессов с использованием нотаций UML: учеб. пособие / Д.В. Шлаев [и др.]. Ставрополь: Издательство «АГРУС», 2023.

Муртазина М. Ш. Управление проектами в сфере информационных технологий: учеб. пособие. Новосибирск: Новосибирский государственный технический университет, 2022.

Полетайкин А. Н. Социальные и экономические информационные системы: законы функционирования и принципы построения: учеб. пособие. Новосибирск: СибГУТИ, 2016.

Суханов М.Б. Программная инженерия: учеб. пособие. СПб.: Санкт-Петербургский государственный университет промышленных технологий и дизайна, 2018.

ПРИЛОЖЕНИЯ

Перечень нефункциональных требований¹
(согласно ГОСТ Р ИСО/МЭК 9126-93)

1. Целостность – способность системы противостоять (сохранять информационное содержание и однозначность интерпретации смысла) изменениям, искажениям или порче при возникновении сбоев или ошибок.

2. Рациональность – подразумевает, что при функционировании оптимальным образом используются имеющиеся в распоряжении системы ресурсы: время, оборудование (память), люди.

3. Численность задействованного персонала – количество лиц, задействованных в эксплуатации и обслуживании системы.

4. Работоспособность – способность системы выполнять свои функции с эксплуатационными показателями не ниже заданных значений.

5. Производительность – характеристика системы, отражающая ее способность производить определенный объем работ в единицу времени, например, пропускная способность, время ответа, доступность, число продуктов, полученная прибыль, быстроедействие.

6. Гибкость – возможность модификации обеспечивающей части системы, обычно возникает по двум причинам: чтобы отразить в системе изменение требований или чтобы исправить ошибки, внесённые ранее в процессе разработки. Гибкость заключается в возможности адаптации, наращивания, изменения средств.

7. Открытость – прозрачность функциональной части системы и возможность ее модификации без нарушения процесса функционирования.

8. Защищенность (безопасность) – способность обеспечения защиты данных от разрушения, искажения или

¹ URL: <https://docs.cntd.ru/document/1200009076>

преднамеренных фальсификаций злоумышленником. Характеризует возможное отсутствие риска, связанного с нанесением некоторого ущерба.

9. Потенциальная управляемость – возможность компенсировать возмущение быстрее, чем успеют измениться эти возмущения.

10. Наблюдаемость основных параметров управляемого процесса – характеризует степень и глубину отслеживания параметров функционирования системы и основных характеристик объекта управления на всех циклах и этапах работы системы / объекта.

11. Надёжность – свойство системы сохранять во времени в установленных пределах значения всех параметров, характеризующих способность системы выполнять требуемые функции в заданных режимах и эксплуатации.

12. Степень оперативности и надёжности управления – характеризует способность системы и субъекта управления выполнять поставленные задачи в сроки, обеспечивающие эффективное управление объектом. Управление является оперативным, если время, затрачиваемое на решение системой задач, в совокупности со временем, необходимым персоналу на подготовку к реализации управленческих решений, не будет превышать критического времени, диктуемого условиями сложившейся ситуации на объекте.

13. Безотказность – свойство системы сохранять работоспособность в течение требуемого интервала времени непрерывно без вынужденных перерывов. Безотказность является наиболее важной компонентой надёжности, так как она отражает способность длительное время функционировать без отказов. Один из показателей надёжности системы.

14. Живучесть – свойство системы сохранять работоспособность в условиях возмущающих воздействий внешней среды и отказов компонентов системы с минимальной частотой отказов, а в случае их возникновения эффективно восстанавливать утраченные функции и ресурсы.

15. Прогрессивность компьютерных и информационных технологий, задействованных в системе.

16. Наглядность интерфейса – должен быть удобным, интуитивно понятным и продуманным с точки зрения инженерной психологии, эргономики и методов технической эстетики.

17. Долговечность – свойство системы сохранять работоспособность до наступления предельного состояния. Зависит от долговечности технических средств и подверженности системы моральному старению.

18. Сопровождаемость – отражает возможность проведения конкретных изменений (модификаций) системы. Изменение может включать исправления, усовершенствования или адаптацию компонентов системы к изменениям в окружающей обстановке, требованиях и условиях функционирования.

19. Стоимость – овеществленный в товаре труд разработчиков. Определяется количеством труда, материала, энергии и информации, затраченных на производство товаров, и измеряется количеством, например эквивалента рабочего времени.

20. Эффективность – получение от функционирования системы существенного технико-экономического, социального или другого эффекта.

21. Мобильность – возможность перенесения системы из одного окружения в другое. Окружающая обстановка может включать организационное, техническое или программное окружение, а также условия эксплуатации.

22. Наличие технической поддержки. Введенная в эксплуатацию готовая система требует определенной технической поддержки. Это обусловлено, прежде всего, динамичностью информационных процессов: совершенствованием документооборота, появлением дополнительных структур данных и автоматизированных функций, что является обычным явлением для любых развивающихся систем.

Приложение Б

Профессия программного инженера¹

Программный инженер (инженер-программист) – это специалист, который имеет высшее техническое образование и занимается разработкой программных продуктов в области вычислительной техники. Программный инженер решает сложные задачи, связанные с проектированием, реализацией, тестированием и сопровождением программного обеспечения, а также изучает принципы, методы, технологии и инструменты для создания качественного и эффективного ПО.

Программный инженер должен обладать следующими навыками и знаниями:

- знание основ информатики, математики, физики и логики;
- знание нескольких языков программирования и парадигм программирования;
- знание основных стандартов, методологий и процессов разработки ПО;
- знание основных инструментов для анализа, проектирования, тестирования и сопровождения ПО;
- умение анализировать требования к ПО и проектировать архитектуру ПО;
- умение реализовывать ПО с использованием различных технологий и платформ;
- умение тестировать ПО на наличие ошибок и уязвимостей;
- умение развертывать и сопровождать ПО в различных средах;
- умение работать в команде и общаться с заказчиками и пользователями.

Программный инженер может работать в различных областях применения ПО, таких как веб-разработка, мобильная

¹ URL: <https://www.profguide.io/professions/inzhener-programmist.html>

разработка, игровая разработка, встроенные системы и т.д. Программный инженер может работать как на постоянной основе в IT-компаниях или организациях, так и на фрилансе или удаленно.

Программная инженерия – это высокооплачиваемая и востребованная профессия в современном мире. По данным сайта HeadHunter, средняя зарплата программного инженера в России составляет 120 000 р. в месяц. По данным сайта Glassdoor, средняя зарплата программного инженера в США составляет 107 000 дол. в год.

Программная инженерия – это также интересная и перспективная профессия, которая позволяет решать разнообразные задачи, создавать полезные и инновационные продукты, постоянно учиться новому и развиваться профессионально.

Требования к программному инженеру могут варьироваться в зависимости от конкретной позиции, проекта или организации. Однако обычно от него требуется:

- высшее образование по специальности «Программная инженерия», «Информатика» или смежной области;
- опыт работы в разработке программного обеспечения от 1 до 5 лет или более;
- сертификаты или портфолио, подтверждающие знание языков программирования и технологий;
- знание английского языка на уровне не ниже среднего.

Программный инженер отличается от программиста тем, что он не только пишет код, но и занимается всем жизненным циклом программного обеспечения от анализа требований до сопровождения. Программный инженер также использует более систематический и дисциплинированный подход к разработке программного обеспечения, основанный на принципах инженерии. Программист же может быть более специализированным в определенном языке программирования или технологии и фокусироваться на реализации кода.

Кодекс этики программного инженера: полная версия¹

Кодекс этики и профессиональной деятельности в области программной инженерии (Software Engineering Code of Ethics and Professional Practice) (версия 5.2).

Рекомендован ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices и совместно одобрен ACM и IEEE-CS в качестве стандарта обучения и работы в области программной инженерии.

Введение

Компьютеры играют центральную и все возрастающую роль в торговле, промышленности, управлении, медицине, образовании, досуге и в жизни общества в целом. Программные инженеры – это те, кто вносит свой вклад либо непосредственно, либо через обучение в анализ, разработку спецификаций, проектирование, реализацию, сертификацию, поддержку и тестирование программных систем. Играя важную роль в разработке программных систем, программные инженеры имеют значительные возможности творить добро или причинять зло, позволять другим творить добро или причинять зло, либо влиять на тех, кто творит добро или причиняет зло. Чтобы обеспечить, насколько это возможно, что их усилия будут использованы в благих целях, программные инженеры должны неуклонно превращать программную инженерию в полезную и уважаемую профессию. Для этого программисты должны твердо придерживаться следующего Кодекса профессиональной этики.

Кодекс содержит восемь принципов, влияющих на линию поведения и выбор решения программными инженерами, включая практиков, преподавателей, менеджеров и высшее руководство, а также учащихся и студентов. Принципы

¹ URL: <https://club.shelek.ru/viewart.php?id=277>

определяют этику отношений между отдельными инженерами, группами и организациями, а также связанные с этим обязательства. В каждый принцип включены иллюстрации некоторых обязательств, налагаемых этими отношениями. Эти обязательства основываются на гуманности профессии программного инженера, особом внимании по отношению к людям, на которых оказывает влияние деятельность программных инженеров, и уникальности этой деятельности. Кодекс провозглашает эти обязательства для всех, кто относит себя к программным инженерам или собирается им стать.

Отдельные части Кодекса не могут быть использованы изолированно от других для оправдания упущений и проступков. Перечень принципов и положений не является исчерпывающим. Положения не могут рассматриваться как разделяющие профессиональное поведение на приемлемое и неприемлемое во всех реальных ситуациях. Кодекс не является простым этическим алгоритмом, генерирующим этические решения. В некоторых ситуациях стандарты могут противоречить друг другу или другим стандартам. Такие ситуации требуют от программного инженера действий в соответствии с духом Кодекса профессиональной этики в зависимости от конкретных обстоятельств.

Следует вдумчиво использовать основные положения этики, а не слепо полагаться на ее подробные указания. Эти принципы побуждают программных инженеров осознать, на кого оказывает влияние выполняемая ими работа; разобраться, относятся ли они и их коллеги к окружающим с должным уважением; принять во внимание, как общество, будучи информированным должным образом, отнеслось бы к их решениям; наконец, оценить, соответствуют ли их профессиональные действия идеалам программной инженерии. Во всех этих оценках забота о благополучии, безопасности и процветании общества первична; т.е. интересы общества являются центральными в данном Кодексе.

Динамичный и требовательный контекст программной инженерии требует кодекса, который можно приспособить к

новым ситуациям по мере их появления. Однако даже в таком общем виде Кодекс обеспечивает поддержку программным инженерам и их руководителям, которые нуждаются в правильном выборе действий в специфических условиях, путем документирования профессиональных этических установок. Кодекс обеспечивает этическую базу, к которой могут обращаться как отдельные члены команд, так и команды в целом. Кодекс позволяет определить действия, которые этически неуместно требовать от программных инженеров или их команд.

Данный кодекс предназначен не только для оценки спорных действий; он имеет также важное образовательное значение. Поскольку в нем выражено общее мнение относительно этической стороны профессии, он является средством, позволяющим довести до сведения как общества, так и профессионалов этические обязательства всех программных инженеров.

Принципы

Принцип 1: ОБЩЕСТВО.

ОБЩЕСТВО – программные инженеры должны действовать неукоснительно в интересах общества. В частности, программные инженеры должны:

- 1) нести полную ответственность за свою работу;
- 2) ограничивать интересы программных инженеров, работодателей, клиентов и пользователей пользой для общества в целом;
- 3) одобрять программное обеспечение лишь в случае, если они твердо убеждены в том, что оно безопасно, соответствует спецификациям, прошло соответствующее тестирование и не угрожает качеству жизни, не нарушает приватности и не вредит окружающей среде. Результат работы должен безусловно служить на благо обществу;
- 4) доводить до сведения уполномоченных лиц и организаций действительную или потенциальную опасность для пользователей, общества или окружающей среды, которая, по их

мнению, связана с использованием программного обеспечения или сопутствующей ему документации;

5) принимать участие в работе над проблемами, вызывающими тревогу в обществе, касающимися программного обеспечения, его инсталляции, развития, поддержки или документирования;

6) быть честным и не допускать лжи во всех высказываниях, особенно публичных, в отношении программного обеспечения или связанных с ним документации, методик и инструментов;

7) обращать внимание на проблемы, связанные с физическими недостатками, распределением ресурсов, экономической отсталостью и другими факторами, способными ограничить доступ к пользованию программным обеспечением;

8) быть готовым добровольно использовать свое профессиональное мастерство для общего блага и способствовать распространению знаний о своей профессии.

Принцип 2: КЛИЕНТ И РАБОТОДАТЕЛЬ.

Программные инженеры должны действовать согласно интересам клиента и работодателя, если они не противоречат интересам общества. В частности, программные инженеры должны:

1) предоставлять услуги в пределах своей компетентности, быть честными и не скрывать ограниченности своего образования и опыта;

2) не использовать программное обеспечение, полученное либо заведомо нелегальным, либо неэтичным путем;

3) пользоваться собственностью клиента или работодателя только надлежащим образом и с их ведома;

4) убедиться, что все используемые ими документы, которые должны быть утверждены, действительно утверждены уполномоченным лицом;

5) хранить втайне любую конфиденциальную информацию, полученную при исполнении профессиональных обязанностей,

если это не противоречит интересам общества и законодательству;

6) идентифицировать, документировать, собирать факты и немедленно оповещать клиента или работодателя, если, по их мнению, проект близок к провалу, оказывается чересчур дорогим, нарушает закон об интеллектуальной собственности или может повлечь другие проблемы;

7) идентифицировать, документировать и докладывать работодателю или клиенту о социальных проблемах, связанных с программной и сопутствующей документацией, о которых им стало известно;

8) не принимать предложений побочной работы, которая может нанести ущерб работе, выполняемой для основного работодателя;

9) не действовать против интересов работодателя или клиента, за исключением случаев, когда это противоречит более высоким этическим соображениям; в этом случае следует информировать работодателя или другое уполномоченное лицо об этих соображениях.

Принцип 3: ПРОДУКТ.

Программные инженеры должны обеспечивать соответствие качества своих продуктов и их модификаций наивысшим возможным профессиональным стандартам. В частности, программные инженеры должны:

1) стремиться к высокому качеству, приемлемой стоимости и разумным срокам выполнения проектов, доводя существенные альтернативы до сведения работодателя и клиента, заручившись их согласием с выбором, а также ставя пользователей и общество в известность о них;

2) обеспечивать адекватность и достижимость целей и направленности для всех проектов, над которыми они работают или намереваются работать;

3) выявлять, определять и принимать меры в отношении проблем, связанных с проектом, над которым они работают, и

имеющих отношение к этике, экономике, культуре, законности и окружающей среде;

4) гарантировать, что их образование, подготовка и опыт достаточны для всех проектов, над которыми они работают или намереваются работать;

5) гарантировать, что во всех проектах, над которыми они работают или намереваются работать, используются надлежащие методики;

6) работать, следуя наиболее подходящим профессиональным стандартам и отступая от них лишь в тех случаях, когда это оправдано по этическим либо техническим причинам;

7) стремиться к полному пониманию спецификаций программного обеспечения, над которым они работают;

8) гарантировать, что спецификации на программное обеспечение, над которым они работают, хорошо документированы, соответствуют требованиям пользователей и утверждены должным образом;

9) гарантировать реалистичность количественных оценок стоимости, сроков выполнения, трудозатрат, качества и затрат по всем проектам, над которыми они работают или намереваются работать, а также неопределенности этих оценок;

10) гарантировать адекватность тестирования, отладки и ревизий программного обеспечения и сопутствующей документации, над которыми они работают;

11) гарантировать адекватность документации, включая обнаруженные проблемы и их одобренные решения, для всех проектов, над которыми они работают;

12) разрабатывать программное обеспечение и сопутствующую документацию, относясь с уважением к приватности в отношении тех, чьи интересы затрагивает данное программное обеспечение;

13) использовать только надежные данные, полученные приемлемыми с точки зрения морали и закона средствами, и использовать их только надлежащим образом;

14) поддерживать целостность данных, подверженных устареванию и потере актуальности;

15) относиться ко всем видам поддержки программного обеспечения с тем же профессионализмом, что и к новым разработкам.

Принцип 4: ОЦЕНКИ.

Программные инженеры должны поддерживать целостность и независимость своих профессиональных оценок. В частности, программные инженеры должны:

1) направлять все технические суждения на службу человеческим ценностям;

2) рекомендовать лишь те документы, которые либо разработаны под их контролем, либо находятся в области их компетентности и с содержанием которых они согласны;

3) соблюдать профессиональную объективность по отношению к программному обеспечению или сопутствующей документации, которые их попросили оценить;

4) не принимать участия в финансовых махинациях, таких как подкуп, двойная оплата и прочие незаконные финансовые действия;

5) раскрывать всем заинтересованным сторонам конфликты интересов, которых невозможно избежать разумными средствами;

6) отказываться от участия в качестве члена команды или советника в частных, правительственных или профессиональных мероприятиях, связанных с программным обеспечением, из-за которых может быть нанесен потенциальный ущерб их собственным интересам, интересам их работодателей или клиентов.

Принцип 5: МЕНЕДЖМЕНТ.

Программные инженеры-менеджеры и ведущие сотрудники должны придерживаться этических подходов к управлению разработкой и поддержкой программного обеспечения и

продвигать эти подходы. В частности, руководители и ведущие специалисты в области программной инженерии должны:

1) гарантировать качественное управление всеми проектами, над которыми они работают, включая эффективные процедуры повышения качества и уменьшения риска;

2) гарантировать, что программные инженеры обучены стандартам перед тем, как намереваются следовать им;

3) гарантировать, что программные инженеры знают политики и процедуры работодателя в отношении защиты паролей, файлов и конфиденциальной информации, касающейся работодателя или иных лиц;

4) распределять работу только после выяснения образования и опыта сотрудника, учитывая его желание совершенствовать свои образование и опыт;

5) гарантировать реалистичность количественных оценок стоимости, сроков выполнения, трудозатрат, качества и прибыли по всем проектам, над которыми они работают или намереваются работать, а также неопределенности этих оценок;

6) привлекать к работе программных инженеров только после того, как им предоставлено полное и точное описание условий работы;

7) предлагать справедливое вознаграждение за труд;

8) не препятствовать беспричинно назначению сотрудника на должность, для которой он имеет подходящую квалификацию;

9) гарантировать справедливое соглашение относительно прав собственности на любое программное обеспечение, технологию, исследования, рукописи и прочую интеллектуальную собственность, в которую программный инженер внес свой вклад;

10) должным образом сообщать об ответственности за нарушение политики работодателя или данного Кодекса;

11) не требовать от программного инженера ничего противоречащего данному Кодексу;

12) не наказывать никого, кто выражает озабоченность в связи с этическими проблемами, связанными с проектом.

Принцип 6: ПРОФЕССИЯ.

Программные инженеры должны поднимать престиж и репутацию своей профессии в интересах общества. В частности, программные инженеры должны:

1) способствовать созданию в организации атмосферы, способствующей этичному поведению;

2) распространять знания в области программной инженерии;

3) расширять знания в области программной инженерии путем участия в профессиональных организациях и собраниях, а также своими публикациями;

4) поддерживать других коллег, стремящихся следовать данному Кодексу;

5) не ставить собственные интересы выше профессиональных интересов, интересов клиента или работодателя;

6) подчиняться всем законам, регулирующим их работу, за исключением особых ситуаций, когда это противоречит интересам общества;

7) быть точным в оценках программного обеспечения, над которым они работают, избегая не только заведомо лживых обещаний, но и обещаний, которые справедливо могут быть восприняты как спекулятивные, необоснованные, вводящие в заблуждение, сбивающие с толку или сомнительные;

8) принимать ответственность за обнаружение, исправление и оповещение об ошибках в программном обеспечении и связанной с ним документации, над которыми он работает;

9) поставить в известность клиентов, работодателей и руководство о том, что программные инженеры следуют данному Кодексу этики, и о последствиях этого;

10) избегать организаций, которые находятся в конфликте с данным Кодексом;

11) осознавать, что нарушения данного Кодекса несовместимы с принадлежностью к профессиональным программным инженерам;

12) выражать свою озабоченность в случае существенного нарушения данного Кодекса людям, причастным к этому, за исключением случаев, когда это невозможно, приводит к серьезным конфликтам или опасно;

13) сообщать о случаях существенного нарушения данного Кодекса в соответствующие инстанции, если очевидно, что диалог с причастными к этому людьми невозможен, приводит к серьезным конфликтам или опасен.

Принцип 7: КОЛЛЕГИ.

Программные инженеры должны быть справедливы по отношению к своим коллегам, помогать им и поддерживать. В частности, программные инженеры должны:

- 1) призывать коллег придерживаться данного Кодекса;
- 2) помогать коллегам в профессиональном росте;
- 3) уважать работу других, но воздерживаться от необоснованного доверия к ней;
- 4) обзирать работу других объективно, непредубежденно, документируя должным образом;
- 5) прислушиваться к мнению, озабоченности или жалобам коллег;
- 6) помогать коллегам в освоении текущих рабочих стандартов, включая политики и процедуры защиты паролей, файлов и другой конфиденциальной информации, а также мер безопасности в целом;
- 7) не вмешиваться без необходимости в рабочие дела коллег; однако искренняя забота об интересах работодателя, клиента или общества могут вынудить программного инженера поставить под сомнение компетентность коллеги;
- 8) в ситуациях, выходящих за пределы их собственной компетентности, спрашивать мнение других профессионалов, компетентных в данной области.

Принцип 8: ЛИЧНАЯ ОТВЕТСТВЕННОСТЬ.

Программные инженеры должны постоянно учиться навыкам своей профессии и способствовать продвижению

этического подхода к своей деятельности. В частности, программные инженеры должны непрерывно стремиться к следующему:

1) углублять свои знания в области анализа, спецификации, проектирования, разработки, поддержки и тестирования программного обеспечения и сопутствующей документации, а также управления процессом разработки;

2) совершенствовать свои способности к созданию безопасного, надежного и функционального качественного программного обеспечения по разумной цене и в разумные сроки;

3) совершенствовать свои способности к производству точной, информативной, качественно написанной документации;

4) совершенствовать знание программного обеспечения и сопутствующей документации, над которой они работают, а также среды, в которой они будут использоваться;

5) совершенствовать знания подходящих стандартов и законов, регулирующих программное обеспечение и сопутствующую документацию, над которыми они работают;

6) совершенствовать знание данного Кодекса, его интерпретацию и использование в своей работе;

7) не допускать несправедливого обращения с кем-либо по причине не относящихся к делу предубеждений;

8) не подстрекать других к действиям, нарушающим данный Кодекс;

9) осознавать, что личное нарушение данного Кодекса несовместимо с принадлежностью к профессиональным программным инженерам.

Стандарты и инструменты программной инженерии

Стандарты программной инженерии – это документы, которые определяют требования, рекомендации, правила и практики для разработки, функционирования и сопровождения программного обеспечения. Стандарты программной инженерии помогают обеспечить качество, совместимость, безопасность и эффективность ПО, а также упростить коммуникацию и сотрудничество между разработчиками и другими заинтересованными сторонами.

Существует множество стандартов программной инженерии, разработанных различными организациями, такими как Международная организация по стандартизации (ИСО), Международная электротехническая комиссия (МЭК), Институт инженеров электротехники и электроники (ИИЭЕ), Ассоциация вычислительной техники (АСМ) и т.д. Некоторые из наиболее известных международных стандартов программной инженерии:

- ISO/IEC 15288 – Systems and Software Engineering – System Life Cycle Processes – Процессы жизненного цикла систем;

- ISO/IEC 25010 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation – Требования к качеству систем и программного обеспечения и оценка;

- ISO/IEC 29119 – Software Testing – Тестирование программного обеспечения;

- ISO/IEC 42010 – Systems and Software Engineering – Architecture Description – Описание архитектуры систем и программного обеспечения;

- ISO/IEC 9126 – Software Engineering – Product Quality – Качество продукта программной инженерии;

- ISO/IEC 15939 – Systems and Software Engineering – Measurement Process – Процесс измерения систем и программного обеспечения;

- ISO/IEC 20000 – Information Technology – Service Management – управление услугами информационных технологий;
- ISO/IEC 27000 – Information Technology – Security Techniques – Техники безопасности информационных технологий;
- IEEE 830 – Recommended Practice for Software Requirements Specifications – Рекомендуемая практика для спецификации требований к программному обеспечению;
- IEEE 1016 – Standard for Information Technology – Systems Design – Software Design Descriptions – Стандарт для информационных технологий – Проектирование систем – Описания проектирования программного обеспечения;
- IEEE 1028 – Standard for Software Reviews and Audits – Стандарт для ревью и аудита программного обеспечения;
- IEEE 1044 – Standard Classification for Software Anomalies – Стандартная классификация аномалий программного обеспечения;
- IEEE 1058 – Standard for Software Project Management Plans – Стандарт для планов управления проектами по разработке программного обеспечения;
- IEEE 1061 – Standard for a Software Quality Metrics Methodology – Стандарт для методологии метрик качества программного обеспечения;
- IEEE 1074 – Standard for Developing Software Life Cycle Processes – Стандарт для разработки процессов жизненного цикла программного обеспечения;
- IEEE 1219 – Standard for Software Maintenance – Стандарт для сопровождения программного обеспечения;
- IEEE 1233 – Guide for Developing System Requirements Specifications – Руководство по разработке спецификации требований к системе;
- IEEE 1471 (ISO/IEC 42010) – Recommended Practice for Architectural Description of Software-intensive Systems –

Рекомендуемая практика для описания архитектуры систем с интенсивным использованием программного обеспечения;

– CMMI (Capability Maturity Model Integration) – Интегрированная модель зрелости возможностей – модель оценки и улучшения процессов разработки ПО.

Инструменты программной инженерии – это программные средства, которые помогают разработчикам выполнять различные задачи в рамках процессов жизненного цикла ПО. Инструменты программной инженерии могут быть классифицированы по различным критериям, таким как функциональность, тип поддерживаемого ПО, стадия жизненного цикла ПО, тип интерфейса и т.д.

Некоторые примеры инструментов программной инженерии:

– инструменты для анализа требований к ПО, например, Rational RequisitePro, DOORS, CaliberRM и т.д.;

– инструменты для проектирования ПО, например, Rational Rose, Enterprise Architect, Visual Paradigm и т.д.;

– инструменты для реализации ПО, например, Visual Studio, Eclipse, NetBeans и т.д.;

– инструменты для тестирования ПО, например, JUnit, TestNG, Selenium и т.д.;

– инструменты для развертывания ПО, например, Jenkins, Docker, Kubernetes и т.д.;

– инструменты для сопровождения ПО, например, Bugzilla, Jira, Redmine и т.д.;

– инструменты для управления проектами по разработке ПО, например, Microsoft Project, Trello, Asana и т.д.;

– инструменты для контроля версий ПО, например, Git, SVN, CVS и т.д.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. ОСНОВНЫЕ ПОНЯТИЯ ПРОГРАММНОЙ ИНЖЕНЕРИИ....	7
1.1. Происхождение программной инженерии	7
1.2. Понятие программной инженерии.....	12
1.3. Программный процесс	19
1.4. Программное обеспечение.....	21
1.5. Методы программной инженерии	22
1.6. CASE-средства	24
2. СТАНДАРТНЫЙ ПРОЦЕСС РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.....	27
2.1. Понятие процесса разработки ПО	27
2.2. Зрелость процесса разработки ПО	29
2.3. Жизненный цикл ПО	34
2.4. Модель процесса разработки ПО.....	37
2.5. ФАЗЫ И ВИДЫ ДЕЯТЕЛЬНОСТИ	38
2.6. Каскадная модель процесса разработки ПО.....	40
2.7. Спиральная модель жизненного цикла ПО	42
2.8. V-образная модель жизненного цикла ПО	47
2.9. Архитектура ПО. Множественность точек зрения	51
3. УПРАВЛЕНИЕ ТРЕБОВАНИЯМИ	54
3.1. Задача управления требованиями к ПО	54
3.2. Виды требований к ПО	56
3.3. Основные трудности при формировании требований к ПО	63
3.4. Свойства требований к ПО	64
3.5. Варианты формализации требований.....	66
3.6. Ошибки при документировании требований	68
3.7. Цикл работы с требованиями.....	72
3.8. Этические требования к программистам.....	75
4. КОНФИГУРАЦИОННОЕ УПРАВЛЕНИЕ И ВЕРСИОННЫЙ КОНТРОЛЬ.....	79
4.1. Понятие конфигурационного управления.....	79
4.2. Объекты конфигурационного управления	81
4.3. Управление версиями составных конфигурационных	

ОБЪЕКТОВ	83
4.4. ПОНЯТИЕ BASELINE.....	84
4.5. ВОЗМОЖНОСТИ СРЕДСТВ ВЕРСИОННОГО КОНТРОЛЯ ПО.....	86
4.6. РАСШИРЕНИЯ И ИНСТРУМЕНТЫ ДЛЯ ВЕРСИОННОГО КОНТРОЛЯ ПО.....	87
5. ТЕХНОЛОГИИ ОБЕСПЕЧЕНИЯ КАЧЕСТВА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	88
5.1. СТАНДАРТИЗАЦИЯ КАЧЕСТВА	88
5.2. МЕТОДЫ ОБЕСПЕЧЕНИЯ КАЧЕСТВА ПО.....	92
5.3. КРИТЕРИИ КАЧЕСТВА ПРОГРАММНОГО ПРОДУКТА	94
5.4. ПОНЯТИЕ ТЕСТИРОВАНИЯ ПО.....	98
5.5. ЦЕЛИ И ПРИНЦИПЫ ТЕСТИРОВАНИЯ ПО	101
5.6. КЛАССИФИКАЦИЯ ВИДОВ ТЕСТИРОВАНИЯ	102
5.7. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ, ОБЛАСТЬ ПРИМЕНЕНИЯ МОДУЛЬНЫХ ТЕСТОВ	113
5.8. НАГРУЗОЧНОЕ И СТРЕСС-ТЕСТИРОВАНИЕ.....	114
5.9. ТЕХНИКИ ТЕСТ-ДИЗАЙНА	118
5.10. АВТОМАТИЗИРОВАННОЕ ТЕСТИРОВАНИЕ	123
5.11. КОМПЬЮТЕРНЫЕ СРЕДСТВА АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ ПО.....	126
5.12. РАБОТА С ОШИБКАМИ	134
5.13. ПЯТЬ ПРИНЦИПОВ ЧИСТЫХ ТЕСТОВ.....	136
5.14. РЕФАКТОРИНГ. ДИЗАЙН КОДА	137
5.15. УПРАВЛЕНИЕ СБОРКАМИ	146
6. УПРАВЛЕНИЕ ПРОЕКТАМИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	154
6.1. ТИПИЧНЫЕ ПРОБЛЕМЫ УПРАВЛЕНИЯ ПРОЕКТАМИ.....	156
6.2. ФУНКЦИИ УПРАВЛЕНИЯ ПРОЕКТАМИ	160
6.3. СОЗДАНИЕ ПРОЕКТОВ И УПРАВЛЕНИЕ ИМИ	163
6.4. СТРАТЕГИИ КОМАНДНЫХ ПРОЕКТОВ.....	164
6.5. УПРАВЛЕНИЕ ПРОЦЕССАМИ.....	165
6.6. УПРАВЛЕНИЕ РАБОЧИМИ ЭЛЕМЕНТАМИ	167
6.7. КОНТРОЛЬ ВЫПОЛНЕНИЯ РАБОТ И ОТЧЕТЫ.....	172
7. УПРАВЛЕНИЕ РИСКАМИ ПРОГРАММНОГО ПРОЦЕССА	175
7.1. ОСНОВНЫЕ РИСКИ ПРОГРАММНОГО ПРОЦЕССА	178

7.2. ПРОЦЕСС УПРАВЛЕНИЯ РИСКАМИ РАЗРАБОТКИ ПО.....	182
7.3. ТЕХНОЛОГИЯ УПРАВЛЕНИЯ РИСКАМИ ПРОГРАММНОГО ПРОЦЕССА	184
7.4. РИСКООБРАЗУЮЩИЕ ФАКТОРЫ ПРОГРАММНОГО ПРОЦЕССА	187
7.5. ИДЕНТИФИКАЦИЯ РИСКОВ ПРОГРАММНОГО ПРОЦЕССА	190
7.6. СТРАТЕГИИ УПРАВЛЕНИЯ РИСКАМИ	194
7.7. МЕТОДИКА РИСК-МЕНЕДЖМЕНТА ПРОЕКТА РАЗРАБОТКИ ПО	197
7.8. РОЛЕВАЯ МОДЕЛЬ РИСК-МЕНЕДЖМЕНТА	202
ЗАКЛЮЧЕНИЕ	204
РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА	205
ПРИЛОЖЕНИЯ	207

Учебное издание

Полетайкин Алексей Николаевич
Добровольская Наталья Юрьевна

МЕТОДОЛОГИЯ И ТЕХНОЛОГИЯ РАЗРАБОТКИ
ПРОГРАММНЫХ СИСТЕМ: МЕТОДЫ И МОДЕЛИ
ПРОГРАММНОЙ ИНЖЕНЕРИИ

Учебное пособие

Подписано в печать 27.02.2025. Выход в свет 05.03.2025.

Печать цифровая. Формат 60×84 ¹/₁₆.

Уч.-изд. л. 14,5. Тираж 500 экз. Заказ № 6014.

Кубанский государственный университет
350040, г. Краснодар, ул. Ставропольская, 149.

Издательско-полиграфический центр
Кубанского государственного университета
350040, г. Краснодар, ул. Ставропольская, 149.